

ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
-----o0o-----



ĐỒ ÁN TỐT NGHIỆP

PHÁT HIỆN KHIẾM KHUYẾT BỀ MẶT VẬT LIỆU SỬ DỤNG KIT DE10

GVHD: PGS. TS. Trương Quang Vinh

SVTH: Hà Xuân Cát _ 2210342

Nguyễn Quốc Tín _ 2213496

TP. HỒ CHÍ MINH, THÁNG 09 NĂM 2026

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc Lập – Tự Do – Hạnh Phúc

KHOA ĐIỆN – ĐIỆN TỬ		NHIỆM VỤ KHÓA LUẬN TỐT NGHIỆP	
(Chú ý: sinh viên phải dán tờ này vào trang thứ nhất của bản thuyết minh)			
HỌ VÀ TÊN:	HÀ XUÂN CÁT NGUYỄN QUỐC TÍN	MSSV:	2210342 2213496
NGÀNH:	ĐIỆN TỬ - VIỄN THÔNG	LỚP:	DD22DVT1 DD22DVT4
1. Đầu đề khóa luận:			
Phát hiện khiếm khuyết bề mặt vật liệu sử dụng KIT DE10			
Surface Defect Detection for Materials Using the DE10 Development Kit			
2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):			
-	Tìm hiểu giải thuật CNN, mô hình VGG.		
-	Thực hiện phần cứng cho CNN.		
-	Huấn luyện mô hình để có được các trọng số cho phần cứng.		
-	Thực hiện phần mềm cho vi xử lý ARM để điều khiển phần cứng.		
-	Thực hiện hệ thống SoC để phát hiện khiếm khuyết bề mặt vật liệu.		
-	Đánh giá và phân tích mô hình.		
3. Ngày giao nhiệm vụ khóa luận:	14 / 06 / 2026		
4. Ngày hoàn thành nhiệm vụ:	24 / 09 / 2026		
5. Họ và tên người hướng dẫn:			Phần hướng dẫn:
-	PGS.TS. Trương Quang Vinh		Toàn bộ
Nội dung và yêu cầu KLTN đã được thông qua Chủ trì ngành.			

Ngàytháng năm 2026

CHỦ TRÌ NGÀNH
(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH
(Ký và ghi rõ họ tên)

PHẦN DÀNH CHO KHOA

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ luận án:

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin được bày tỏ lòng biết ơn sâu sắc nhất đến PGS.TS. Trương Quang Vinh, giảng viên Bộ môn Điện – Điện tử, trường Đại học Bách Khoa – ĐHQG TP.HCM. Trong suốt quá trình học tập, nghiên cứu và thực hiện đề tài tốt nghiệp "**Phát hiện khiếm khuyết bề mặt vật liệu sử dụng kit DE10**". Thầy đã hướng dẫn, giúp đỡ, định hướng chuyên môn và đưa ra những lời khuyên, đóng góp vô cùng quý báu để chúng em có thể hoàn thiện đề tài một cách tốt nhất. Sự nghiêm túc, tâm huyết và tầm nhìn khoa học của Thầy không chỉ giúp em vượt qua những khó khăn trong kỹ thuật mà còn là bài học lớn cho em về tác phong làm việc chuyên nghiệp sau này.

Em cũng xin chân thành cảm ơn quý Thầy cô thuộc Bộ môn Điện – Điện tử, Khoa Điện – Điện tử trường Đại học Bách Khoa TP.HCM đã truyền đạt cho em những kiến thức nền tảng vững chắc, tạo mọi điều kiện thuận lợi về cơ sở vật chất cũng như phòng thí nghiệm trong suốt những năm tháng em học tập tại trường.

Mặc dù đã dành nhiều thời gian và nỗ lực để hoàn thành đồ án, song do kiến thức và kinh nghiệm thực tiễn của bản thân còn hạn chế, đề tài chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ quý Thầy cô trong Hội đồng chấm đồ án tốt nghiệp để đề tài của em được hoàn thiện hơn.

Cuối cùng, em xin kính chúc PGS.TS. Trương Quang Vinh cùng toàn thể quý Thầy cô Bộ môn Điện – Điện tử luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp trồng người cao quý.

Tp. Hồ Chí Minh, ngày tháng năm 2026 .

Sinh viên

TÓM TẮT ĐỒ ÁN

Đồ án này trình bày các nội dung liên quan đến thiết kế hệ thống SoC để thực hiện dự án “PHÁT HIỆN KHIẾM KHUYẾT BỀ MẶT VẬT LIỆU SỬ DỤNG KIT DE10”, nhóm triển khai thành các nội dung chính như sau:

Nội dung 1: Tìm hiểu nguyên lý, lý thuyết về mạng nơron tích chập (CNN), bài toán phân loại lỗi bề mặt vật liệu và IP CNN

Nội dung 2: Tìm hiểu cấu trúc phần cứng bo mạch Intel Cyclone V SoC, kiến trúc bus giao tiếp Avalon Memory-Mapped (Avalon MM) và các cầu nối AXI giữa HPS và FPGA.

Nội dung 3: Thu thập dữ liệu hình ảnh vật liệu lỗi, huấn luyện mô hình CNN rút gọn (VGG9) bằng Python (Keras/Tensorflow).

Nội dung 4: Xây dựng cấu trúc firmware chạy trên hệ điều hành Linux của CPU ARM để quản lý dữ liệu camera, thiết lập thanh ghi điều khiển lỗi IP và xử lý các tầng Fully-Connected cuối cùng.

Nội dung 5: Tích hợp toàn bộ hệ thống bằng công cụ Platform Designer (Qsys), biên dịch bằng phần mềm Quartus Prime, nạp bo mạch thực tế và đưa kết quả lên màn hình.

Nội dung 6: Tiến hành đánh giá về hoạt động của hệ thống bao gồm tài nguyên phần cứng đã sử dụng, thời gian đáp ứng và tốc độ khung hình, tính ứng dụng, tính thẩm mỹ của hệ thống.

Nội dung 7: Kết luận về hệ thống đồng thời đưa ra hướng phát triển đề tài trong tương lai.

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Đặt vấn đề	1
1.2 Tình hình nghiên cứu trong và ngoài nước	2
1.3 Mục tiêu và nhiệm vụ của đề tài	3
2. LÝ THUYẾT	4
2.1 Tổng quan về kiến trúc của mạng VGG	4
2.2 Tìm hiểu chi tiết về kiến trúc của mạng VGG	6
2.2.1 Các lớp tích chập	6
2.2.2 Hàm kích hoạt	10
2.2.3 Các lớp Fully-Connected	10
2.2.4 Hàm Softmax	11
2.3 Tìm hiểu về dòng board SoC FPGA DE10-Standard	11
2.3.1 Tổng quan về board DE10-Standard	11
2.3.2 Sơ đồ khối và thành phần board	13
2.3.3 Giao tiếp bus Avalon	15
2.3.3.1 Avalon Memory Mapped Slave	16
2.3.3.2 Avalon Memory Mapped Master	21
2.3.4 SoC trên board DE10 Standard	24
2.4 Tìm hiểu về Qsys và project GHRD từ intel	25
2.4.1 Qsys là gì?	25
2.4.2 Thành phần Qsys là gì?	27

2.4.3 Project GHRD (Golden Hardware Reference Design)	27
2.4.3.1 Hard Processing System	29
2.4.3.2 System ID Peripheral Intel FPGA IP	29
2.4.3.3 JTAG to Avalon Master Bridge.....	30
2.4.3.4 Avalon MM Pipeline Bridge	30
2.4.3.5 JTAG UART Intel FPGA IP	31
2.4.3.6 Altera Interrupt Latency Counter Intel FPGA IP	31
3. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG	32
3.1. Tổng quan hệ thống.....	32
3.2. Yêu cầu thiết kế.....	33
3.3. Thiết kế phần cứng.....	33
3.4. Thực hiện phần cứng.....	35
3.4.1 Cấu hình hệ thống FPGA	35
3.4.2 Biên dịch hệ thống FPGA	47
4. THỰC HIỆN PHẦN MỀM.....	51
4.1 Yêu cầu thiết kế.....	51
4.2 Thiết kế và thực hiện phần mềm cho hệ thống trên kit DE10	51
4.2.1 Thiết kế phần mềm	51
4.2.2 Thực hiện phần mềm	61
4.2.2.1 Khởi tạo vùng nhớ ảo	61
4.2.2.2 Khởi tạo trọng số	63
4.2.2.3 Đọc ảnh đầu vào bằng camera, xác định vùng ROI và thực hiện tiền xử lý ảnh	66

4.2.2.4 Thực hiện tiền xử lý ảnh đầu vào	68
4.2.2.5 Thực hiện điều khiển phân cứng	69
4.2.2.6 Thực hiện lớp dense và trả kết quả phân loại khiếm khuyết	72
5. THỰC HIỆN HỆ THỐNG.....	74
5.1 Huấn luyện mô hình Deep Learning trên Google Colab và chuyển đổi trọng số.....	76
5.1.1 Thực hiện huấn luyện mô hình.....	76
5.1.2 Chuyển đổi trọng số thành file .txt.....	78
5.1.3 Khởi tạo môi trường Linux trên board DE10.....	85
5.1.4 Thiết lập kết nối từ laptop đến board DE10	87
5.2 Vận hành hệ thống SoC	90
5.2.1 Vận hành phân cứng trên board DE10	90
5.2.2 Vận hành phần mềm trên board DE10	99
6. KẾT QUẢ THỰC HIỆN.....	103
6.1 Kết quả trực quan.....	103
6.2 Đánh giá hệ thống	107
7. KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN HỆ THỐNG.....	111
7.1 Kết luận.....	111
7.2 Hướng phát triển	112
8. TÀI LIỆU THAM KHẢO.....	113

DANH SÁCH HÌNH ẢNH

Hình 2.1 Kiến trúc CNN phổ biến	4
Hình 2.2 Kiến trúc của mạng VGG16.	5
Hình 2.3 Quá trình thực hiện tích chập dùng same padding.	8
Hình 2.4 Output feature map 4x4.	9
Hình 2.5 Output feature map dùng max pooling với stride = 2.....	9
Hình 2.6 Đồ thị của hàm ReLU.	10
Hình 2.7 Block diagram của board DE10 Standard. [1].....	12
Hình 2.8 Các chân và chức năng quan trọng của board DE10-Standard. [1]..	12
Hình 2.9 Sơ đồ khối của Kit DE10. [1]	13
Hình 2.10 Avalon MM Slave. [2].....	16
Hình 2.11 Avalon MM Master. [2].....	21
Hình 2.12 Sơ đồ khối thiết bị SoC của Cyclone V.....	24
Hình 2.13 Sơ đồ khối hệ thống Qsys mẫu được triển khai trên board FPGA..	26
Hình 2.14 Cấu trúc GHRD cung cấp từ Terrasic.	28
Hình 3.1 Sơ đồ khối hoạt động của hệ thống.....	32
Hình 3.2 Thiết kế phần cứng. [3].....	34
Hình 3.3 File cnn_ver3 trong thư mục ip	35
Hình 3.4 Các kết nối của GHRD trong Qsys.....	36
Hình 3.5 Mục New Component khi mở Qsys	37
Hình 3.6 Nhập tên thiết kế là “cnn_top_ver3”	37
Hình 3.7 Thêm code RTL và đặt file cnn_top.v là Top Design	38
Hình 3.8 Analyze Synthesis Files thành công	38

Hình 3.9 Signals & Interfaces của IP “cnn_top_ver3”	39
Hình 3.10 Block Symbol của IP “cnn_top_ver3”	40
Hình 3.11 IP “address_span_extender_0”	41
Hình 3.12 IP “address_span_extender_1”	41
Hình 3.13 Kết nối hoàn chỉnh hệ thống SoC trên Qsys.....	43
Hình 3.14 Giao diện Generate HDL	43
Hình 3.15 Tạo file .sopcinfo	44
Hình 3.16 Sau khi tạo file .sopcinfo	44
Hình 3.17 Nội dung script tạo file header .h.....	45
Hình 3.18 Quá trình chạy script.....	45
Hình 3.19 Sau khi tạo file .h	46
Hình 3.20 Nội dung file hps_0.h vừa tạo.....	46
Hình 3.21 Giao diện khi load file DE10_Standard_FB.qpf đã hoàn thành phần Qsys	47
Hình 3.22 Quartus bắt đầu compile hệ thống	48
Hình 3.23 Báo cáo quá trình biên dịch hệ thống FPGA	48
Hình 3.24 Báo cáo phân tích timing Slow 1100mV 85C của hệ thống.....	49
Hình 3.25 Báo cáo phân tích timing Slow 1100mV 0C của hệ thống.....	49
Hình 3.26 File .sof được tạo sau quá trình biên dịch.....	50
Hình 4.1 Các luồng dữ liệu giữa HPS và khối IP CNN	52
Hình 4.2 Phân vùng nhớ RAM DDR3	52
Hình 4.3 Máy trạng thái phần mềm	53
Hình 4.4 Lưu đồ phần mềm điều khiển phần cứng HPS trên vi xử lý ARM ..	54

Hình 4.5 Cấu trúc chương trình phần mềm	55
Hình 4.6 Đường dẫn của file hwlib.h	59
Hình 4.7 Đường dẫn của folder socal	59
Hình 4.8 Các file header có trong folder socal	60
Hình 4.9 Lưu đồ chương trình khởi tạo vùng nhớ ảo	62
Hình 4.10 Lưu đồ chương trình khởi tạo trọng số cho quá trình tích chập	64
Hình 4.11 Lưu đồ chương trình khởi tạo trọng số cho quá trình fully-connected	65
Hình 4.12 Lưu đồ chương trình đọc dữ liệu từ camera.....	66
Hình 4.13 Lưu đồ chương trình khoanh vùng dữ liệu.	67
Hình 4.14 Cơ chế hoạt động đọc của Input buffer.....	68
Hình 4.15 Giải thuật tiền xử lý ảnh đầu vào.....	69
Hình 4.16 Lưu đồ chương trình điều khiển phần cứng.....	69
Hình 4.17 Lưu đồ chương trình gửi thông tin điều khiển.....	71
Hình 4.18 Lưu đồ chương trình trả output feature map.	71
Hình 4.19 Lưu đồ thực hiện chương trình lớp dense và trả kết quả phân loại khiếm khuyết.....	72
Hình 5.1 Đường cong học tập được thực hiện.....	76
Hình 5.2 Ma trận nhầm lẫn.	77
Hình 5.3 File .hdf5 sau khi train	78
Hình 5.4 Bên trong file model.weights.hdf5 khi chưa gộp batch.....	78
Hình 5.5 Chạy file read_h5.py.....	79
Hình 5.6 Bên trong file model_e48_acc_0.9967.h5 sau khi gộp batch.....	80
Hình 5.7 File reshape_arr.m	80

Hình 5.8 File write_txt_c.m.....	81
Hình 5.9 File write_weight_vgg_keras.m	82
Hình 5.10 File testMatlab.m	82
Hình 5.11 Bias của lớp convolution 1 trong định dạng .txt.....	83
Hình 5.12 Trọng số của lớp convolution 1 trong định dạng .txt	83
Hình 5.13 Bias của lớp dense 2 trong định dạng .txt.....	84
Hình 5.14 Trọng số của lớp dense 2 trong định dạng .txt	84
Hình 5.15 Phần mềm SD Card Formatter.....	85
Hình 5.16 Phần mềm Win32 Disk Imager.....	86
Hình 5.17 Kết quả sau khi burn hệ điều hành.....	86
Hình 5.18 Thiết lập kết nối giữa laptop, máy tính và board	87
Hình 5.19 Kết nối laptop với board thông qua PuTTY	88
Hình 5.20 Đăng nhập vào root của board	88
Hình 5.21 Tạo folder làm việc mới.....	89
Hình 5.22 Truy cập vùng nhớ dev	89
Hình 5.23 Gán vùng nhớ trên thẻ microSD vào vùng nhớ trên board.....	89
Hình 5.24 Copy folder Software vào phân vùng root.....	90
Hình 5.25 Folder Software hiển thị trên màn hình máy tính	90
Hình 5.26 Quá trình tạo file .rbf	91
Hình 5.27 Quá trình tạo file .rbf	92
Hình 5.28 Sau khi tạo file .rbf	92
Hình 5.29 Quá trình tạo file .dts và .dtb	93
Hình 5.30 Sau khi tạo file .dts và .dtb	93

Hình 5.31 Các file trên thẻ nhớ dùng để boot.....	94
Hình 5.32 Kết nối trên board	94
Hình 5.33 Quá trình nạp phần cứng lên board.....	95
Hình 5.34 Quá trình nạp phần cứng lên board.....	95
Hình 5.35 Quá trình nạp phần cứng lên board.....	96
Hình 5.36 Quá trình nạp phần cứng lên board.....	96
Hình 5.37 Quá trình nạp phần cứng lên board.....	97
Hình 5.38 Quá trình nạp phần cứng lên board.....	97
Hình 5.39 Quá trình nạp phần cứng lên board.....	98
Hình 5.40 Quá trình nạp phần cứng lên board.....	98
Hình 5.41 Các file phần mềm vận hành phần cứng.....	99
Hình 5.42 Nội dung Makefile	99
Hình 5.43 Include thư viện OpenCV	100
Hình 5.44 Version của thư viện OpenCV	100
Hình 5.45 Đường dẫn đến thư viện OpenCV	100
Hình 5.46 Bên trong thư mục Software	101
Hình 5.47 Biên dịch và xuất file thực thi.....	101
Hình 5.48 File thực thi sau khi chạy Makefile	102
Hình 6.1 Hệ thống phát hiện khiếm khuyết bề mặt vật liệu dựa trên FPGA. 103	
Hình 6.2 Hệ thống phát hiện khiếm khuyết bề mặt vật liệu dựa trên FPGA. 104	
Hình 6.3 Phát hiện không có lỗi	104
Hình 6.4 Phát hiện bề mặt bị xước	105
Hình 6.5 Phát hiện bề mặt bị thủng lỗ	105

Hình 6.6 Phát hiện bề mặt bị rỉ sét.....	106
Hình 6.7 Phát hiện bề mặt bị nứt	106
Hình 6.8 Ma trận nhầm lẫn khi chạy tập test trên FPGA.	109

DANH SÁCH BẢNG

Bảng 2.1 Thông số của mạng VGG16.....	5
Bảng 2.2 Cụm tích chập đầu tiên của mạng VGG16.....	7
Bảng 2.3 Các tín hiệu có trong Avalon MM slave	17
Bảng 2.4 Các tín hiệu có trong Avalon MM master.....	22
Bảng 3.1 Tài nguyên hệ thống FPGA tiêu tốn.....	49
Bảng 4.1 Các hàm chính trong main.cpp.....	56
Bảng 4.2 Các hàm chính trong VideoMaterialDetector.cpp	56
Bảng 4.3 Các hàm chính trong vgg9.cpp.....	57
Bảng 4.4 Các hàm chính trong conv.cpp.....	57
Bảng 4.5 Các hàm chính trong dense.cpp.....	58
Bảng 4.6 Thông số của mạng nơron tích chập đề nghị.....	60
Bảng 4.7 Các địa chỉ ngoại vi sử dụng trong phần mềm.	62
Bảng 5.1 Thông số của mạng nơron tích chập đề nghị.....	75
Bảng 5.2 Các chỉ số đánh giá mô hình	77
Bảng 6.1 Tài nguyên phần cứng tiêu tốn	108
Bảng 6.2 Các chỉ số đánh giá mô hình nhận diện khiếm khuyết trên Google Colab	109

1. GIỚI THIỆU

1.1 Đặt vấn đề

Trong các dây chuyền sản xuất hiện đại, quy trình giám sát, kiểm định và phát hiện sớm các khiếm khuyết trên bề mặt vật liệu (như vết nứt, trầy xước, rỉ sét, rỗ khí,...) trong quá trình chế tạo có ý nghĩa quyết định đến độ bền, tính thẩm mỹ và độ an toàn của sản phẩm đầu ra. Nhiều thập kỷ qua, công việc này phần lớn vẫn phụ thuộc vào phương pháp thủ công bằng mắt người. Tuy nhiên, phương pháp truyền thống này bộc lộ những hạn chế không thể khắc phục: tốc độ xử lý chậm, độ chính xác không đồng đều, phụ thuộc hoàn toàn vào yếu tố chủ quan và dễ xảy ra sai sót.

Dựa vào sự bùng nổ của trí tuệ nhân tạo và kỹ thuật thị giác máy tính, cụ thể là mạng nơ-ron tích chập (gọi tắt là CNN), đã mở ra một cuộc cách mạng cho các hệ thống kiểm tra quang học tự động. Nhờ khả năng tự động trích xuất các đặc trưng hình ảnh phức tạp ở nhiều tầng cấu trúc, các mô hình CNN đạt được độ chính xác vượt trội so với các thuật toán xử lý ảnh truyền thống trong bài toán phân loại và phát hiện lỗi bề mặt vật liệu. Nhưng việc triển khai các mô hình này tại nhà xưởng đang gặp một nút thắt lớn đó là các kiến trúc Deep Learning thông thường đòi hỏi hạ tầng GPU đắt đỏ, cồng kềnh, trong khi việc chạy bằng phần mềm trên CPU nhúng lại gây ra latency rất lớn do xử lý tuần tự.

Do tài nguyên trên chip Cyclone V SoC FPGA bị giới hạn nghiêm ngặt (chỉ tích hợp 112 bộ DSP cứng và 5570 Kb On-chip memory), việc triển khai các cấu trúc mạng khổng lồ là bất khả thi nếu không có các chip phần cứng cao cấp. Chính vì vậy, ý tưởng cốt lõi của đề tài là tùy chỉnh thiết kế, huấn luyện một mô hình deep learning VGG9 gọn nhẹ phù hợp với tài nguyên hiện có của kit DE10. Cấu trúc phần cứng cho mạng tích chập này sẽ được thiết kế dựa trên các nguyên lý tối ưu hóa bộ đệm (Input/Output Buffer) và tối ưu hóa vòng lặp tính toán trích dẫn từ công trình nghiên cứu đi trước của tác giả Phạm Thế Vinh (2020).

Từ những nhu cầu thực tiễn và thách thức kỹ thuật nêu trên, đề tài "**Phát hiện khiếm khuyết bề mặt vật liệu sử dụng kit DE10**" đã được thực hiện hệ thống có khả năng thu thập luồng dữ liệu hình ảnh trực tiếp từ camera, tự động

phát hiện, phân loại khuyết tật bề mặt trong thời gian thực với độ trễ thấp và tiêu thụ ít năng lượng, mở ra hướng ứng dụng thực tế cho các dây chuyền sản xuất công nghiệp.

1.2 Tình hình nghiên cứu trong và ngoài nước

Ở trong nước, đã có không ít các đề tài nghiên cứu về lĩnh vực xử lý ảnh này. Đầu tiên là đề tài “Optimizing Convolutional Neural Network Accelerator on Low-Cost FPGA” của tác giả Trương Quang Vinh và Đinh Việt Hai [3]. Trong bài nghiên cứu, tác giả tập trung thực hiện bộ tăng tốc trên hệ thống FPGA giá rẻ, Intel Cyclone V SoC trên DE10-Standard. Tiếp theo có thể kể đến là đề tài “Thiết kế phần cứng cho mạng thần kinh tích chập” của tác giả Phạm Thế Vinh. Đề tài hướng đến việc thiết kế cấu trúc của một mạng tích chập bằng giải thuật CNN, đặc biệt là mạng VGG16, và sau đó là kiểm chứng và đánh giá hiệu suất của thiết kế. Hai bài nghiên cứu này sẽ là nền tảng để thực hiện đề tài đồ án lần này.

Với tình hình ngoài nước cũng đã có nhiều đề tài có sự quan tâm đến lĩnh vực này. Đáng chú ý là đề tài “FPGA-based Accelerators of Deep Learning Networks for Learning and Classification: A Review” đã và đang được các nghiên cứu sinh quan tâm bởi tính ứng dụng cao cũng như phạm vi nghiên cứu sâu và rộng. Ở đề tài này, các tác giả tập trung phân tích hiệu suất của các mô hình mạng tích chập phổ biến từ đó áp dụng lên FPGA và các ASIC hỗ trợ.

Từ những nghiên cứu này, có thể thấy phần nào nền tảng và cơ sở để triển khai đề tài liên quan đến hệ thống phát hiện khiếm khuyết bề mặt vật liệu sử dụng mạng thần kinh tích chập trên nền tảng FPGA trong bối cảnh những nỗ lực nghiên cứu và ứng dụng trong lĩnh vực này.

1.3 Mục tiêu và nhiệm vụ của đề tài

Thông qua việc tìm hiểu những vấn đề xoay quanh đề tài “Phát hiện khiếm khuyết bề mặt vật liệu sử dụng KIT DE10”, để vận hành và kiểm chứng được hoạt động của một mạng thần kinh tích chập trên board SoC-FPGA có thể nhận dạng và phân loại 5 loại khiếm khuyết bề mặt vật liệu, đồng thời gửi kết quả lên màn hình thì nhiệm vụ đề án bao gồm:

- Nghiên cứu lý thuyết về CNN và giải thuật nhận diện, phân loại khiếm khuyết bề mặt vật liệu.
- Huấn luyện mô hình mạng CNN theo kiến trúc VGG với 7 lớp tính chập.
- Hiểu và dùng được thiết kế kiến trúc phần cứng cho lõi tăng tốc IP Core CNN.
- Xây dựng hệ thống firmware trên lõi ARM (HPS) để xử lý việc đọc ảnh từ camera, truyền nhận bộ tham số/trọng số qua bộ nhớ dùng chung (Shared Memory DDR3), điều khiển FPGA khởi chạy và tính toán lớp Fully-Connected (FC) cuối cùng để đưa ra phân loại.
- Thực hiện ứng dụng cấu trúc CNN trên FPGA để phân loại khiếm khuyết.
- Đánh giá và phân tích hệ thống

2. LÝ THUYẾT

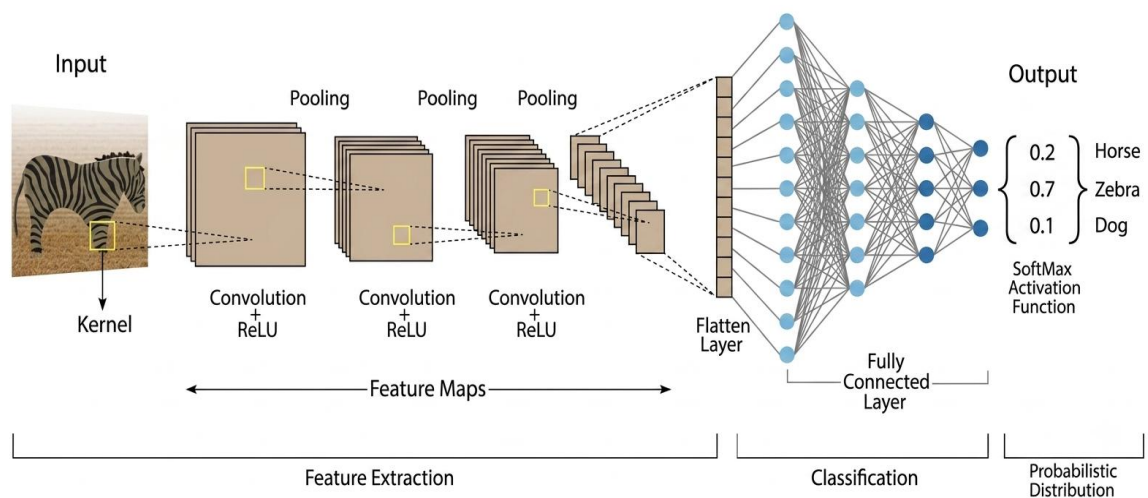
2.1 Tổng quan về kiến trúc của mạng VGG

Hiện nay, CNN đang được ứng dụng vô cùng rộng rãi trên nhiều lĩnh vực khác nhau. Hai tham số cốt lõi cấu thành mô hình này bao gồm trọng số weight và bias. Khởi đầu, lớp đầu tiên sẽ tiếp nhận ảnh đầu vào, qua xử lý sẽ tạo ra một tập hợp các feature map. Các lớp tiếp theo sẽ lần lượt lấy đầu ra của lớp trước đó làm đầu vào để tiếp tục trích xuất đặc trưng. Cuối cùng, một bộ phân loại sẽ đưa ra dự đoán về xác suất phân lớp (Softmax) của bức ảnh ban đầu.

Mạng VGG16 là một hình phổ biến và tiêu biểu nhờ khả năng phân loại đối tượng xuất sắc, dù sở hữu khối lượng thông số khá đồ sộ. Nên nhóm chúng em sẽ dựa trên mạng này để giải thích tổng quan hơn về kiến trúc CNN.

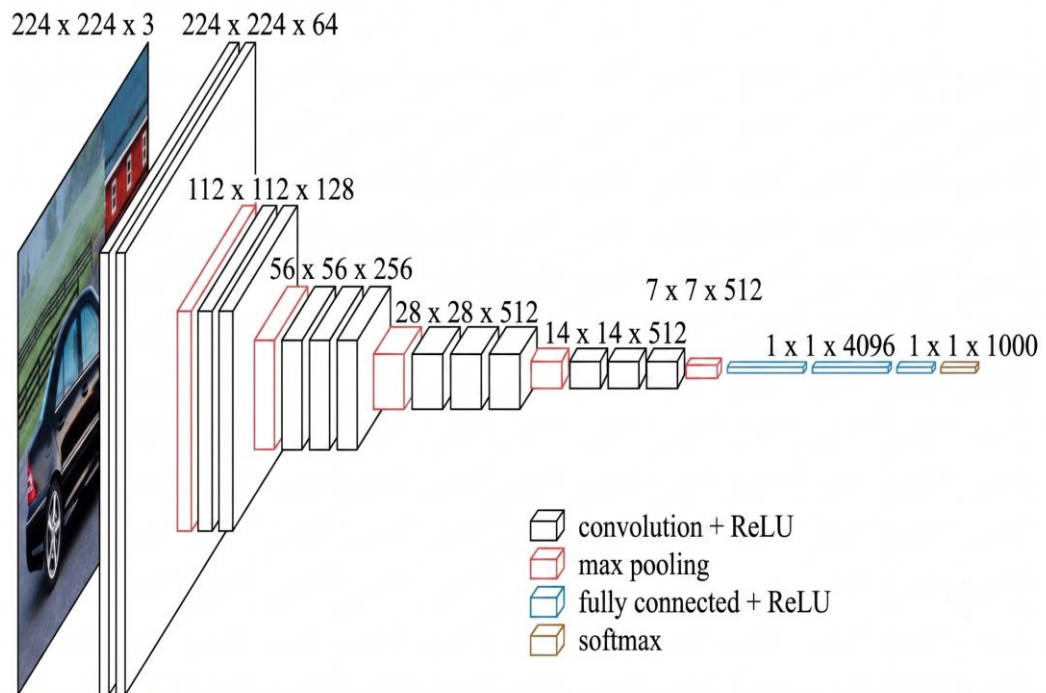
Lớp tích chập (viết tắt là CONV) và lớp Fully-Connected (viết tắt là FC) đóng vai trò nòng cốt trong kiến trúc CNN, thường được xen kẽ bởi các lớp Pooling.

Lớp CONV sẽ sử dụng các bộ lọc (kernel) để biến đổi các input feature map thành các output feature map.. Trong khi đó, **lớp Pooling** có nhiệm vụ chất lọc, giữ lại giá trị trung bình hoặc lớn nhất của mỗi vùng trên bản đồ đặc trưng. Đối với **lớp FC**, thuật toán thực hiện phép biến đổi tuyến tính trên một vector đặc trưng một chiều (gọi là Flatten), thay vì ma trận hai chiều như các lớp trước. Tổng quát về kiến trúc này thể hiện ở Hình 2.1.



Hình 2.1 Kiến trúc CNN phổ biến

Về tổng thể, mạng VGG16 sở hữu 16 lớp, bao gồm 13 lớp tích chập và 3 lớp fully-connect. Nó tiếp nhận ảnh đầu vào với độ phân giải 224x224 và phân loại kết quả vào một trong 1000 danh mục. Tổng quát, kiến trúc mạng VGG16 như sau:



Hình 2.2 Kiến trúc của mạng VGG16.

Bảng 2.1 Thông số của mạng VGG16.

CONV layer	Input size	Output size	N _{imap}	N _{omap}	Max pooling
CONV1	224x224	224x224	3	64	0
CONV2	224x224	112x112	64	64	1
CONV3	112x112	112x112	64	128	0
CONV4	112x112	56x56	128	128	1
CONV5	56x56	56x56	128	256	0

CONV6	56x56	56x56	256	256	0
CONV7	56x56	28x28	256	256	1
CONV8	28x28	28x28	256	512	0
CONV9	28x28	28x28	512	512	0
CONV10	28x28	14x14	512	512	1
CONV11	14x14	14x14	512	512	0
CONV12	14x14	14x14	512	512	0
CONV13	14x14	7x7	512	512	1
FC layer		Input size		Output size	
FC1		25088		4096	
FC2		4096		4096	
FC3		4096		1000	

2.2 Tìm hiểu chi tiết về kiến trúc của mạng VGG

2.2.1 Các lớp tích chập

Các lớp CONV đóng vai trò là nền tảng cốt lõi của mạng CNN. Về mặt toán học, quá trình tính toán tại lớp tích chập liên quan đến việc nhân chập input feature map với các bộ lọc (kernel) và cộng thêm bias để tạo ra output feature map.

Và công thức tính tích chập được mô tả như sau:

$$f_i^{out} = \sum_{j=1}^{n_{in}} f_j^{in} \otimes g_{i,j} + b_i \quad (1 \leq i \leq n_{out}) \quad (1)$$

Với:

- f_j^{in} : Input feature map thứ j.
- f_i^{out} : Output feature map thứ i.
- b_i : Bias thứ i của feature map.
- $g_{i,j}$: Là convolution kernel để input feature map thứ i thành output feature map thứ j.
- n_{in}, n_{out} là số lượng input và output feature map.

Trong mạng VGG16, lấy ví dụ ở cụm tích chập đầu tiên:

Bảng 2.2 Cụm tích chập đầu tiên của mạng VGG16.

CONV layer	Input size	Output size	N _{imap}	N _{omap}	Max pooling
CONV1	224x224	224x224	3	64	0
CONV2	224x224	112x112	64	64	1

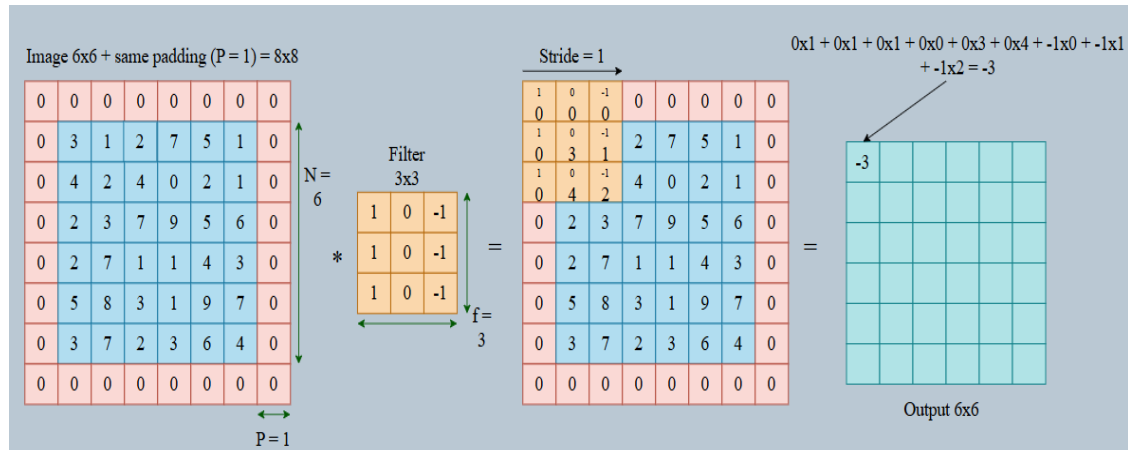
Tại lớp CONV1:

- Input: 224x224x3.
- Kernel: 3x3x3x64.
- Bias: 64.
- Pooling: 0.
- Output: 224x224x64.

Bước 1: Thực hiện chèn padding và tính tích chập:

Để bảo toàn nguyên vẹn độ phân giải của ảnh sau khi đi qua các bộ lọc, kỹ thuật chèn thêm viền (padding - ví dụ như chèn 'same padding' hoặc 'zero padding' được áp dụng. Sau khi thực hiện thao tác nhân chập và cộng bias lặp lại 64 lần, mô hình sẽ hoàn thành lớp tích chập đầu tiên và đảm bảo rằng kích

thước đầu ra sẽ bằng với đầu vào. Có thể hình dung bước 1 qua Hình 2.3 bên dưới:



Hình 2.3 Quá trình thực hiện tích chập dùng same padding.

Ta có thể tính toán được kích thước output qua công thức sau:

$$\text{The output size} = \frac{N - F + 2P}{S} + 1$$

Với:

- N: Kích thước ảnh đầu vào.
- F: Kích thước của filter.
- P: Lượng Padding thêm vào.
- S: Bước trượt.

Bước 2: Thực hiện cộng bias sau mỗi phép tích chập:

Ta có 64 lần thực hiện tích chập giữa input $224 \times 224 \times 3$ với filter $3 \times 3 \times 3$ nên kết quả sẽ cộng bias và ghi vào output $224 \times 224 \times i$ (với $i = 0, 1, \dots, 64$). Mà thực hiện tích chập 64 lần nên output sẽ là $224 \times 224 \times 64$.

Tại lớp CONV2:

- Input: $224 \times 224 \times 64$.
- Kernel: $3 \times 3 \times 64 \times 64$.
- Bias: 64.

- Pooling: 1.
- Output: 112x112x64.

Điểm đặc trưng của VGG16 là áp dụng hàm Max pooling sau mỗi cụm convolution để rút gọn kích thước dữ liệu. Thuật toán pooling này sẽ duyệt qua feature map và chọn ra giá trị lớn nhất trong từng cửa sổ 2x2 với bước trượt được cấu hình bằng 2.

Bước 1: Tính tích chập.

Bước 2: Thực hiện cộng bias sau mỗi phép tích chập.

Bước 3: Thực hiện pooling với stride = 2 để resize output còn 112x112x64.

Ví dụ về việc dùng max pooling với stride = 2 như sau. Giả sử output feature map thu được có giá trị sau:

0	4	3	5
1	9	7	3
3	6	4	7
8	2	9	0

Hình 2.4 Output feature map 4x4.

Áp dụng stride = 2 và max pooling để lấy giá trị lớn nhất trong vùng có kích thước cửa sổ trượt là 2x2, lúc này, output feature map giảm kích thước như sau:

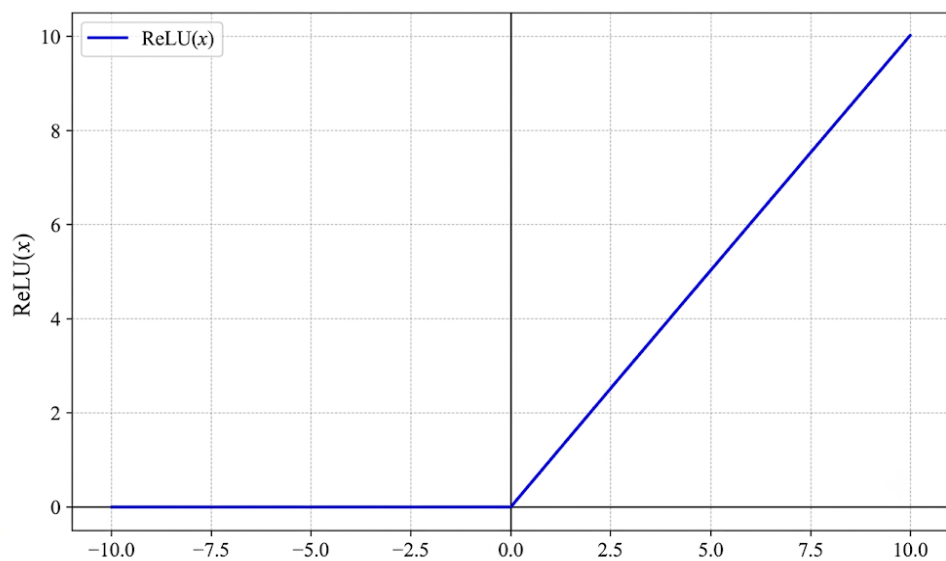
9	7
8	9

Hình 2.5 Output feature map dùng max pooling với stride = 2.

2.2.2 Hàm kích hoạt

Hàm kích hoạt đảm nhận nhiệm vụ đưa đặc tính phi tuyến vào mô hình, giúp biến đổi dữ liệu đầu ra của các neuron trước khi đẩy sang lớp kế tiếp. Mạng VGG16 sử dụng hàm ReLU (Rectified Linear Unit) với nguyên lý lọc tín hiệu rất đơn giản: $\text{ReLU}(x)$ giữ nguyên giá trị dương và đưa toàn bộ các giá trị âm về mức 0:

$$\text{ReLU}(x) = \max(0, x)$$



Hình 2.6 Đồ thị của hàm ReLU.

2.2.3 Các lớp Fully-Connected

Ngay trước khi tiến vào vùng không gian của các lớp FC, toàn bộ các bản đồ đặc trưng từ lớp CONV cuối cùng phải trải qua bước flatten nhằm biến đổi ma trận đa chiều thành dạng mảng 1 chiều.

Ví dụ: Một khối dữ liệu đầu vào kích thước $7 \times 7 \times 512$ sẽ được duỗi thẳng thành một vector chứa 25088 phần tử.

Tại tầng FC, hệ thống bắt đầu thực hiện phép nhân ma trận giữa vector đầu vào này với một mảng trọng số, kế đó là cộng dồn thêm bias. Giả sử lớp FC1 tiếp nhận vector 25088 phần tử và đối chiếu với kernel có kích thước tương ứng, kết quả xuất ra cuối cùng sẽ thu gọn thành một mảng $1 \times 1 \times 4096$.

2.2.4 Hàm Softmax

Được xem như một hàm trung bình mũ, Softmax đóng vai trò tính toán xác suất phân bố của các nhóm đối tượng mục tiêu. Bản chất của hàm này là đánh giá và quy đổi điểm số thành xác suất cho từng class dự đoán trong tổng thể các class có thể xảy ra, từ đó xác định nhãn hợp lý nhất cho đầu vào.

Công thức tính cơ bản của Softmax là lấy giá trị hàm mũ của một phần tử bất kỳ chia cho tổng các hàm mũ của toàn bộ phần tử trong tập hợp đang xét:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \forall i = 0, 1, \dots, n$$

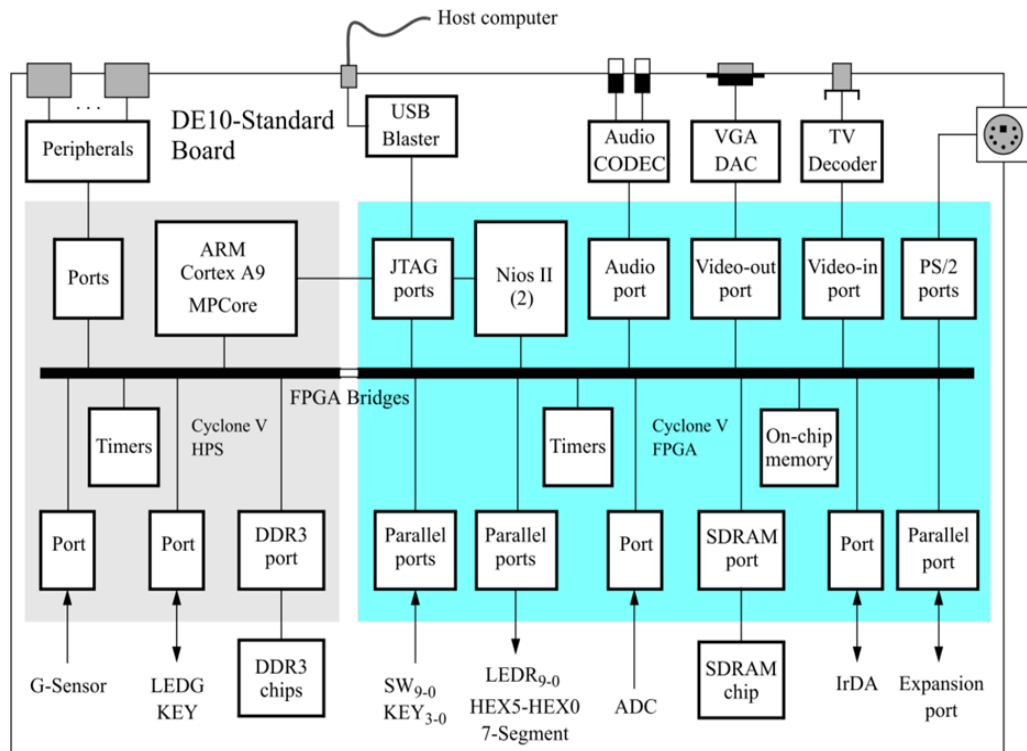
Trong đó:

- z_i : Đầu ra của neuron tại lớp fully connected thứ i .
- n : Số lớp phân loại đầu ra.

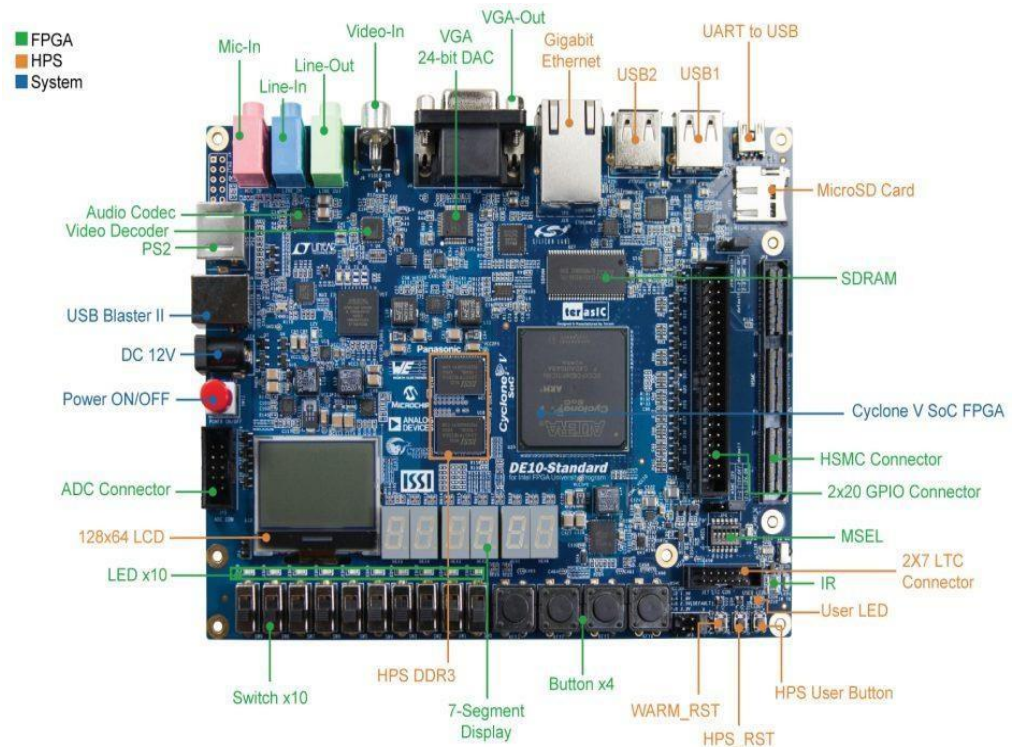
2.3 Tìm hiểu về dòng board SoC FPGA DE10-Standard

2.3.1 Tổng quan về board DE10-Standard

DE10-Standard là board System-on-Chip Field-Programmable Gate Array (gọi tắt là SoC FPGA) được phát triển trên dòng chip chi phí thấp của Altera là Cyclone V SoC, gồm hai phần chính: HPS và FPGA. Phần bên trái là Hard Processing System (HPS) bao gồm lõi vi xử lý cứng ARM, bộ điều khiển EMAC, bộ điều khiển SDRAM,... và bên phải là phần FPGA sẽ chứa các thiết kế phần cứng của người dùng. Tương tác giữa HPS và FPGA được thực hiện thông qua cầu nối AXI Bridge được cấu hình cho tương tác giữa bus Avalon (FPGA) và bus AXI (HPS), cho phép người dùng truy cập trực tiếp vào các thành phần của HPS (ví dụ như SDRAM) từ phía FPGA.

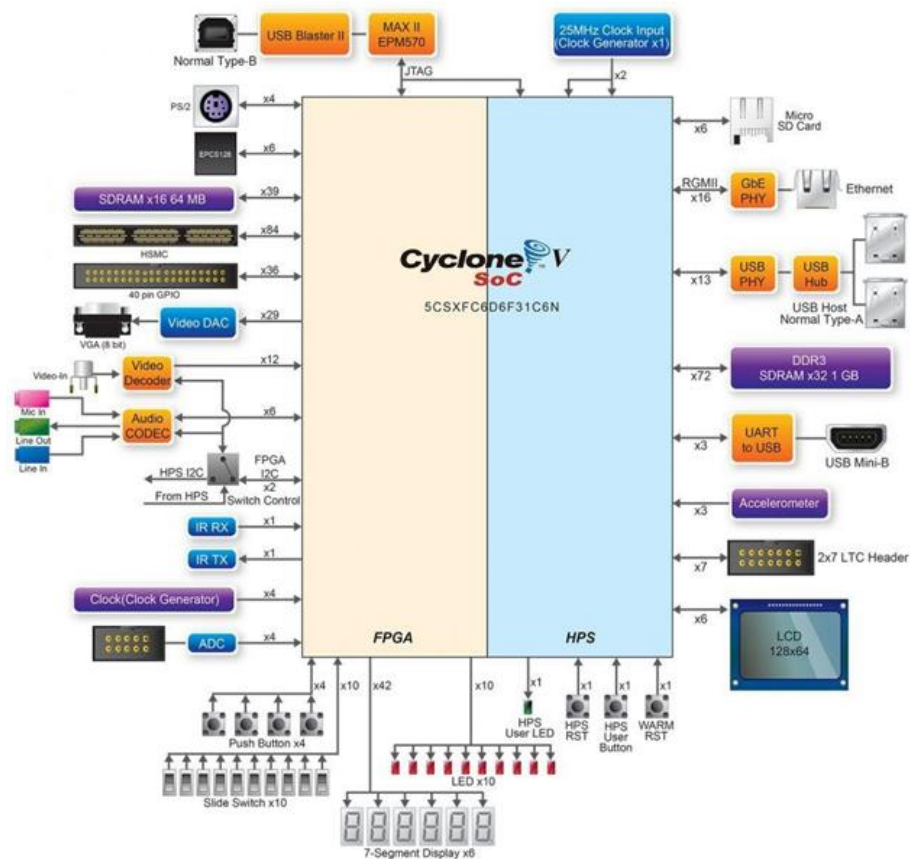


Hình 2.7 Block diagram của board DE10 Standard. [1]



Hình 2.8 Các chân và chức năng quan trọng của board DE10-Standard. [1]

2.3.2 Sơ đồ khối và thành phần board



Hình 2.9 Sơ đồ khối của Kit DE10. [1]

Thành phần [1]:

FPGA Device

- Cyclone V SoC 5CSXFC6D6F31C6N Device
- Dual-core ARM Cortex-A9 (HPS)
- 110K programmable logic elements
- 5,761 Kbits embedded memory
- 6 fractional PLLs
- 2 hard memory controllers

Configuration and Debug

- Quad serial configuration device – EPCS128 on FPGA
- Onboard USB-Blaster II (normal type B USB connector)

Memory Device

- 64MB (32Mx16) SDRAM on FPGA
- 1GB (2x256Mx16) DDR3 SDRAM on HPS
- Micro SD card socket on HPS

Communication

- Two port USB 2.0 Host (ULPI interface with USB type A connector)
- UART to USB (USB Mini-B connector)
- 10/100/1000 Ethernet
- PS/2 mouse/keyboard
- IR emitter/receiver
- I2C multiplexer

Connectors

- One HSMC (Configurable I/O standards 1.5/1.8/2.5/3.3V)
- One 40-pin expansion headers
- One 10-pin ADC input header
- One LTC connector (one Serial Peripheral Interface (SPI)

Master, one I2C and one GPIO interface)

Display

- 24-bit VGA DAC
- 128x64 dots LCD Module with Backlight

Audio

- 24-bit CODEC, Line-in, Line-out, and microphone-in jacks

Video Input

- TV decoder (NTSC/PAL/SECAM) and TV-in connector

ADC

- Interface: SPI
- Fast throughput rate: 500 KSPS
- Channel number: 8
- Resolution: 12-bit
- Analog input range : 0 ~ 4.096

Switches, Buttons, and Indicators

- 5 user Keys (FPGA x4, HPS x1)
- 10 user switches (FPGA x10)
- 11 user LEDs (FPGA x10, HPS x 1)
- 2 HPS reset buttons (HPS_RESET_n and HPS_WARM_RST_n)
- Six 7-segment displays

Sensors

- G-Sensor on HPS

Power

- 12V DC input

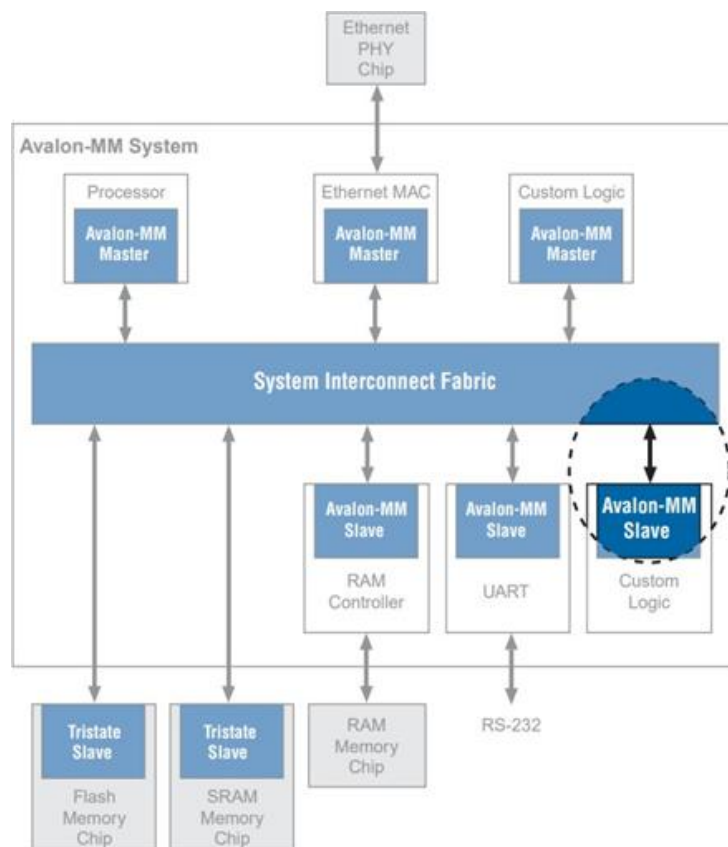
2.3.3 Giao tiếp bus Avalon**Avalon Memory Mapped**

Giao tiếp Avalon Memory-Mapped (Avalon MM) được sử dụng cho các trao đổi đọc/ghi dữ liệu giữa các thành phần master/slave trong hệ thống ánh xạ bộ nhớ dựa trên địa chỉ. Các thành phần này bao gồm bộ vi xử lý, bộ nhớ, UART và bộ định thời; chúng có giao thức master/slave được kết nối với nhau bởi một kết nối hệ thống chung. Giao tiếp Avalon MM có thể mô tả nhiều loại thành phần, từ một SRAM hỗ trợ đơn giản, cụ thể là việc trao đổi đọc/ghi dữ liệu trong chu kỳ cố định sang một giao tiếp pipeline phức tạp hơn, ngoài ra còn có khả năng trao đổi dữ liệu theo kiểu burst.

Các tính năng:

- Định nghĩa kết nối điểm với điểm
- Triển khai tự do các tập hợp con khi tín hiệu được yêu cầu
- Độ rộng dữ liệu có thể thay đổi: 8, 16, 24, ... 1024
- Tạo kết nối tự động

2.3.3.1 Avalon Memory Mapped Slave



Hình 2.10 Avalon MM Slave. [2]

Các loại tín hiệu có trong Avalon MM Slave được trình bày trong bảng sau [2]:

Bảng 2.3 Các tín hiệu có trong Avalon MM slave

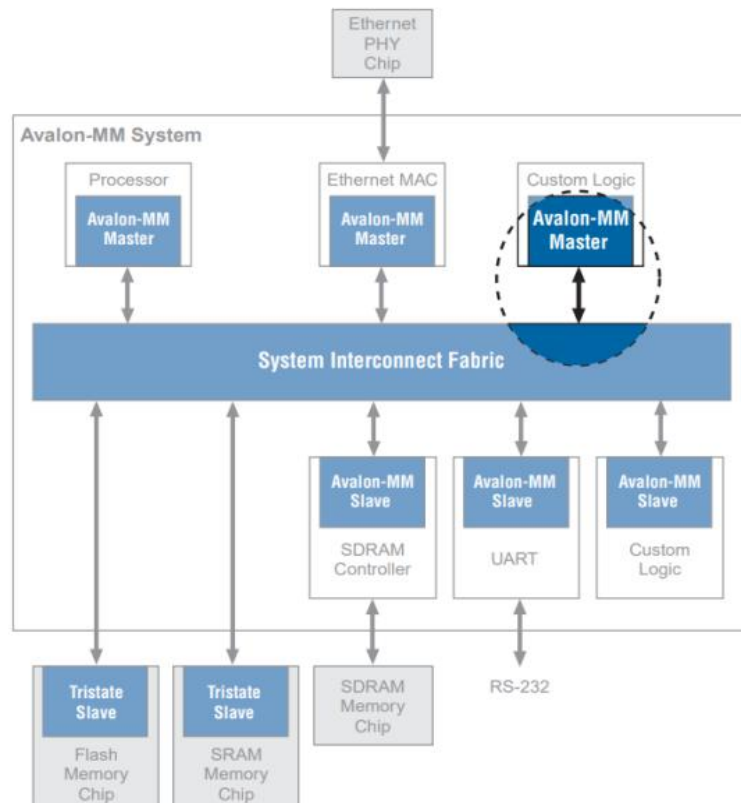
Loại tín hiệu	Độ rộng	Hướng	Yêu cầu	Mô tả
Tín hiệu cơ bản				
read read_n	1	In	No	Cho phép tín hiệu đọc, thường sẽ đi kèm với readdata.
write write_n	1	In	No	Cho phép tín hiệu ghi, thường sẽ đi kèm với writedata hoặc writebyteenable.
address	1-32	In	No	Địa chỉ để xác định không gian lưu trữ dữ liệu.
readdata	8, 16, 24, 32, 64, 128, 256, 512, 1024	Out	No	Dữ liệu được cung cấp khi có tín hiệu đọc.
writedata	8, 16, 24, 32, 64, 128, 512, 1024	In	No	Dữ liệu được nhận từ hệ thống khi có tín hiệu ghi.

byteenable byteenable_n	1, 2, 4, 8, 16, 32, 64, 128	In	No	Cho phép các lần byte cụ thể trong quá trình ghi. Nó sẽ chỉ định byte nào được ghi bằng cách đưa một dãy số 4 bit: 1111 ghi đủ 32 bits; 0011 ghi 2 bytes thấp; 1100 ghi 2 bytes cao; 0001 chỉ ghi byte 0.
begintransfer	1	In	No	Được xác nhận bởi kết nối hệ thống cho chu kỳ đầu tiên của mỗi lần chuyển bất kể yêu cầu chờ và các tín hiệu khác.
Tín hiệu trạng thái chờ				
waitrequest waitrequest_n	1	Out	No	Dùng để tạm ngưng kết nối bên trong hệ thống khi port slave không thể phản hồi tức thì.

arbiterlock arbiterlock_n	1	In	No	Tín hiệu phân xử chắc chắn khi một master giành được quyền phân xử, nó sẽ duy trì kết nối đến slave cho nhiều quá trình trao đổi.
Tín hiệu pipeline				
readdatavalid readdatavalid_n	1	Out	No	Dùng trong giao thức đọc pipeline với độ trễ có thể thay đổi được. Nó đánh dấu xung cạnh lên khi slave nhận được tín hiệu readdata phù hợp.
Tín hiệu burst				
burstcount	1-11	In	No	Dùng trong giao thức burst. Nó đại diện cho số lần trao đổi trong một burst. Thường sẽ đi kèm với tín hiệu waitrequest.

beginbursttransfer		In	No	Được xác nhận ở đầu chu kỳ khi giao thức burst bắt đầu.
Tín hiệu flow control				
readyfordata	1	Out	No	Được sử dụng trong giao thức flow control. Cho biết rằng ngoại vi đã sẵn sàng để thực hiện quá trình ghi.
dataavailable	1	Out	No	Được sử dụng trong giao thức flow control. Cho biết rằng ngoại vi đã sẵn sàng để thực hiện quá trình đọc.
Tín hiệu reset				
resetrequest resetrequest_n	1	Out	No	Cho phép ngoại vi thiết lập lại toàn bộ hệ thống Avalon MM.

2.3.3.2 Avalon Memory Mapped Master



Hình 2.11 Avalon MM Master. [2]

Ta có thể tăng thông lượng của hệ thống bằng cách bắt đầu đọc với nhiều thiết bị ngoại vi slave vì hệ thống tạo ra kết nối điểm – điểm giữa các cặp master/slave. Trong phản hồi các lần đọc, khi một thiết bị ngoại vi slave có dữ liệu hợp lệ, nó sẽ xác nhận `read_data_valid` và vài kết nối hệ thống cho phép kết nối giữa cặp master/slave.

Các loại tín hiệu được trình bày bảng dưới [2]:

Bảng 2.4 Các tín hiệu có trong Avalon MM master.

Loại tín hiệu	Độ rộng	Hướng	Yêu cầu	Mô tả
Tín hiệu cơ bản				
address	1-32	Out	Yes	Tín hiệu địa chỉ đại diện cho một byte địa chỉ, không phụ thuộc chiều rộng dữ liệu của master. Để viết một byte cụ thể trong một word, master phải sử dụng tín hiệu byteenable.
read read_n	1	Out	No	Đọc tín hiệu yêu cầu từ master. readdata phải có khi có tín hiệu đọc.
readdata	8, 16, 24, 32, 64, 128, 256, 512, 1024	In	No	Dữ liệu được cung cấp khi có tín hiệu đọc.
write write_n	1	Out	No	Tín hiệu ghi từ master. Thường sẽ đi cùng với writedata.
writedata	8, 16, 32, 64, 128, 256, 512, 1024	Out	No	Dữ liệu được nhận từ hệ thống khi có tín hiệu ghi. readdata và writedata phải có cùng chiều rộng.

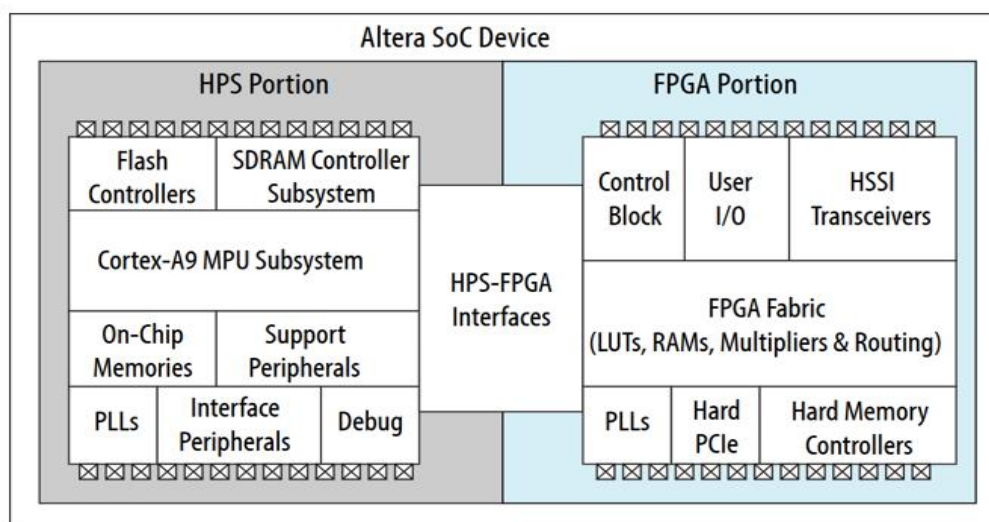
byteenable byteenable_n	1, 2, 4, 8, 16, 32, 64, 128	Out	No	Cho phép các lần byte cụ thể trong quá trình chuyển trên các cổng của chiều rộng lớn hơn 8 bit. Mỗi bit trong byteenable tương ứng với một lần byte trong writedata và readdata.
waitrequest waitrequest_n	1	In	Yes	Buộc thiết bị chính phải đợi cho đến khi hệ thống kết nối với nhau đã sẵn sàng để tiến hành trao đổi dữ liệu. Lúc bắt đầu quá trình trao đổi, master bắt đầu quá trình và đợi cho đến khi yêu cầu chờ được thực hiện. Master phải giữ nguyên các tín hiệu điều khiển của nó trong các chu kỳ tiếp theo nếu yêu cầu chờ đợi được xác nhận phân xử.
arbiterlock arbiterlock_n	1	Out	No	Khóa phân xử đảm bảo rằng khi master giành được quyền truy cập, nó sẽ duy trì kết nối truy cập vào nhiều slave cho nhiều chuyển giao.
Tín hiệu pipeline				
readdatavalid readdatavalid_n	1	In	No	Chỉ ra dữ liệu hợp lệ được chuyển đến dòng dữ liệu.

flush	1	Out	No	Dùng để xóa các giao tiếp đang chờ thực hiện trong một lần pipeline.
Tín hiệu burst				
burstcount	1-11	Out	No	Biểu thị số lần trao đổi trong một lần burst.
Tín hiệu reset				
resetrequest resetrequest_n	1	Out	No	Cho phép thành phần thiết lập lại toàn bộ hệ thống Avalon MM.

Về phần byteenable, khi nhiều hơn một lần byte được xác nhận, tất cả lần đã được xác nhận phải được đặt liền kề nhau. Số dòng liền kề phải là lũy thừa của 2 và số byte chỉ định phải được căn chỉnh trên một giới hạn địa chỉ phụ thuộc vào kích thước của dữ liệu.

2.3.4 SoC trên board DE10 Standard

Sơ đồ khối ở cấp độ high-level của SoC này được thể hiện trong hình sau:



Hình 2.12 Sơ đồ khối thiết bị SoC của Cyclone V.

Cấu trúc của SoC trên kit Altera gồm có 2 phần chính: HPS và FPGA. HPS sử dụng hệ thống bus AXI trong khi FPGA sử dụng hệ thống bus Avalon, vì thế cần có các cầu nối để giao tiếp giữa hai hệ thống này với nhau.

HPS Portion

HPS là hệ thống xử lý phần cứng của SoC Altera. Hệ thống xử lý này bao gồm các vi xử lý ARM và các ngoại vi đi kèm. Vi xử lý ARM trên SoC của Cyclone V là một cặp ARM Cortex-9. Ngoài MPU Subsystem của ARM Cortex-9, HPS còn bao gồm các ngoại vi khác như PLLs, bộ điều khiển Flash Controllers, SDRAM Controllers, bộ nhớ on-chip, v.v.

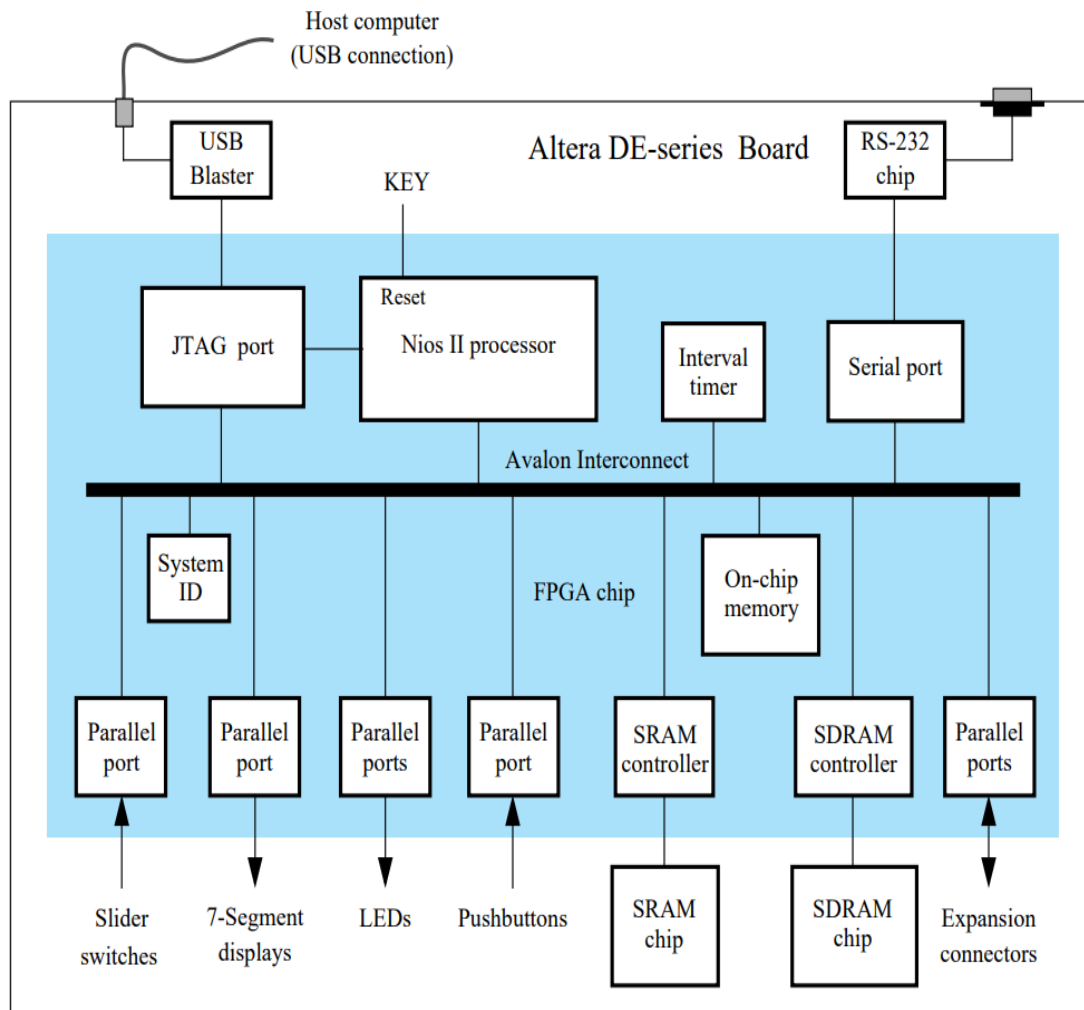
FPGA Portion

FPGA chính là loại vi mạch cho phép người dùng tùy chỉnh và lập trình lại cấu trúc logic bên trong, sau khi ta thiết kế RTL sẽ được kết nối vào hệ thống. Các IP được lập trình sẽ nằm ở phần này và ngoài ra còn có các ngắt và các cổng giao tiếp DIP Switch, JTAG UART, LED và BUTTON,...

2.4 Tìm hiểu về Qsys và project GHRD từ intel

2.4.1 Qsys là gì?

Công cụ Qsys cho phép người dùng kết nối một hệ thống với nhau bằng cách sử dụng các thành phần được tạo sẵn hoặc tùy chỉnh. Các hệ thống như vậy thường bao gồm một hoặc nhiều bộ xử lý, giao diện bộ nhớ, cổng I/O và phần cứng tùy chỉnh khác. Hệ thống do Qsys tạo ra có thể được đưa vào như một phần của mạch lớn hơn và được triển khai trên bảng FPGA, chẳng hạn như Altera DE-series. Ví dụ về một hệ thống như vậy được mô tả trong Hình bên dưới, trong đó phần của hệ thống được tạo bởi công cụ Qsys được đánh dấu bằng màu xanh dương.



Hình 2.13 Sơ đồ khối hệ thống Qsys mẫu được triển khai trên board FPGA.

Mỗi thành phần trong hệ thống, được gọi là thành phần Qsys, tuân theo ít nhất một trong các Giao diện Avalon được Qsys hỗ trợ. Với giao diện được xác định cho thành phần, Qsys có thể xây dựng một cấu trúc kết nối, được gọi là Avalon Interconnect, cho phép các thành phần trao đổi dữ liệu. Công cụ Qsys có thể tạo ra một hệ thống dựa trên tập hợp các thành phần và thông số người dùng đã chọn. Hệ thống được tạo chứa mã Verilog hoặc VHDL cho từng thành phần và cấu trúc kết nối, cho phép nó được tổng hợp, đặt và định tuyến cho thiết bị FPGA.

2.4.2 Thành phần Qsys là gì?

Thành phần Qsys là một mạch con phần cứng có sẵn như một thành phần thư viện để sử dụng trong công cụ Qsys. Thông thường, nó chứa hai phần: các mô-đun phần cứng bên trong và các Giao diện Avalon bên ngoài. Các mô-đun bên trong là các mạch thực hiện chức năng mong muốn của thành phần Qsys, trong khi các Giao diện Avalon được thành phần sử dụng để giao tiếp với các mô-đun phần cứng bên ngoài thành phần.

Các loại giao diện Avalon:

- Avalon Clock Interface – một giao diện phát hoặc nhận tín hiệu clock.
- Avalon Reset Interface – một giao diện cung cấp khả năng reset.
- Avalon Memory-Mapped Interface (Avalon MM) – một giao diện đọc/ghi dựa trên địa chỉ, vốn là đặc trưng của các kết nối kiểu Master-Slave.
- Avalon Streaming Interface (Avalon-ST) – một giao diện hỗ trợ luồng dữ liệu một chiều
- Avalon Conduit Interface – một giao diện hỗ trợ các tín hiệu đơn lẻ hoặc các nhóm tín hiệu không thuộc bất kỳ loại giao diện Avalon nào khác. Có thể xuất các tín hiệu conduit này để thực hiện kết nối ra bên ngoài hệ thống Qsys.

2.4.3 Project GHRD (Golden Hardware Reference Design)

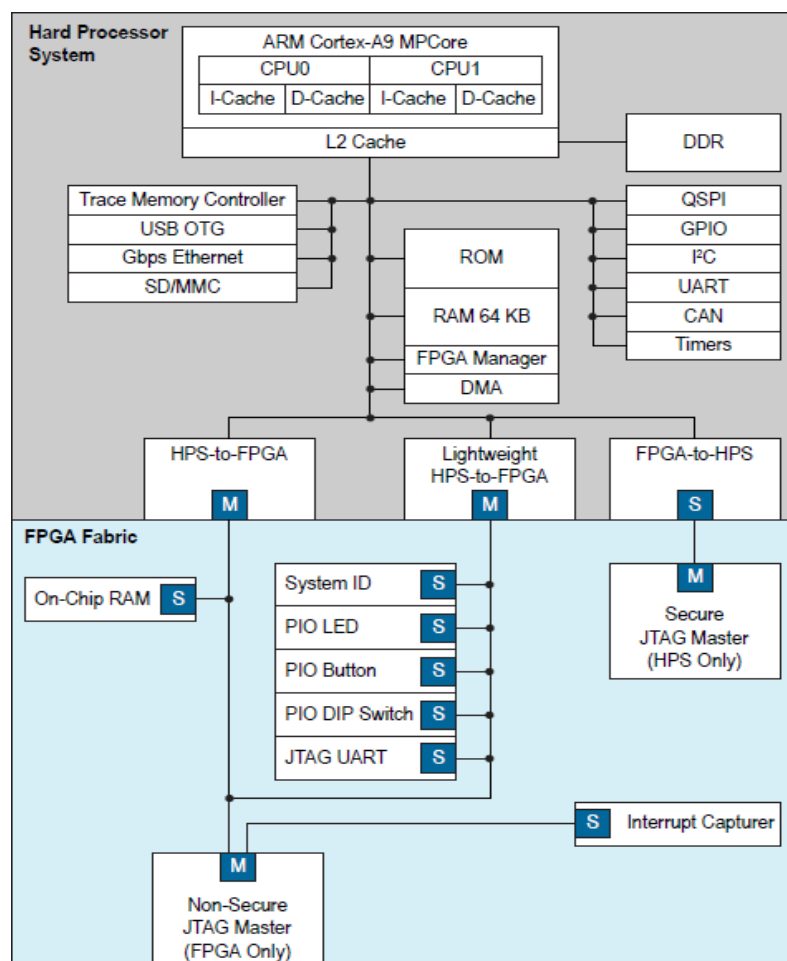
GHRD sử dụng các ngoại vi trong vùng FPGA một cách tối thiểu vì phía HPS đã cung cấp cho ta rất nhiều lựa chọn các ngoại vi như Ethernet, UART, SDRAM, SD/MMC. Các giao tiếp HPS-FPGA và FPGA-HPS được cấu hình với độ rộng dữ liệu là 64 bit. Thiết kế GHRD này bao gồm các thành phần như sau:

- ARM Cortex™-A9 MPCore HPS
- 02 user push-button inputs

- 04 user DIP switch inputs
- 04 user I/O for LED outputs
- 64KB of on-chip memory
- JTAG to Avalon master bridges
- Interrupt capturer for use with System Console
- System ID

Thiết kế GHRD sử dụng cầu nối HPS-FPGA, FPGA-HPS và Lightweight HPS-FPGA, đồng thời, cấu trúc này cho thấy những ngoại vi được hỗ trợ trong mỗi phần FPGA và phần HPS như JTAG, Interrupt, ROM, RAM, DDR, ...

Các thành phần IP trong project GHRD:



Hình 2.14 Cấu trúc GHRD cung cấp từ Terrasic.

2.4.3.1 Hard Processing System

HPS là hệ thống xử lý phần cứng của SoC Altera. Hệ thống xử lý này bao gồm các vi xử lý ARM và các ngoại vi đi kèm. Vi xử lý ARM trên SoC của Cyclone 5 là một cặp ARM Cortex-9. Ngoài MPU Subsystem của ARM Cortex-9, HPS còn bao gồm các ngoại vi khác như PLLs, bộ điều khiển Flash Controllers, SDRAM Controllers, bộ nhớ on-chip, v.v.

Chức năng các Bridge trong HPS:

- Lightweight HPS-to-FPGA Bridge: Cho phép master từ HPS truy cập đến slave ánh xạ bộ nhớ các thành phần ở FPGA. Thông thường MPU ở HPS truy cập cầu nối này thực hiện các truy cập vào thanh ghi trạng thái và điều khiển tới các ngoại vi trong FPGA.

Lưu ý : Không sử dụng cầu này để truy cập vào memory FPGA.

- HPS-to-FPGA Bridge: Sử dụng để kết nối trực tiếp bộ nhớ của FPGA đến HPS.

Lưu ý: Truy cập thanh ghi trạng thái và điều khiển của FPGA không được sử dụng ở cầu này và sẽ được chuyển đến sử dụng ở Lightweight HPS-to-FPGA Bridge.

- FPGA-to-HPS Bridge: Cho phép master từ FPGA truy cập vào bộ nhớ và ngoại vi bên trong HPS.

2.4.3.2 System ID Peripheral Intel FPGA IP

Lỗi ID hệ thống với giao diện Avalon® là một thiết bị chỉ đọc đơn giản cung cấp cho các hệ thống Trình thiết kế nền tảng một mã định danh duy nhất. Các hệ thống bộ xử lý sử dụng lỗi ID hệ thống để xác minh rằng chương trình thực thi được biên dịch nhằm mục tiêu vào phần cứng thực được định cấu hình trong FPGA đích. Nếu ID dự kiến trong tệp thực thi không khớp với lỗi ID hệ thống trong FPGA, có thể phần mềm sẽ không thực thi chính xác.

Có hai cách cơ bản để sử dụng lỗi ID hệ thống:

- Xác minh ID hệ thống trước khi tải phần mềm mới xuống hệ thống. Phương pháp này được sử dụng bởi các công cụ phát triển phần mềm, chẳng hạn như môi trường phát triển tích hợp Nios® II

(IDE), Nios® II IDE kiểm tra xem lỗi ID hệ thống trong phần cứng có khớp với ID hệ thống dự kiến của phần mềm hay không trước khi tải xuống chương trình để chạy hoặc gỡ lỗi.

- Kiểm tra ID hệ thống sau khi đặt lại. Nếu một chương trình đang chạy trên phần cứng không phải là hệ thống Trình thiết kế nền tảng dự kiến, chương trình có thể không hoạt động hoàn toàn. Nếu chương trình không gặp sự cố, nó có thể hoạt động sai theo những cách tinh vi khó gỡ lỗi. Để tránh trường hợp này, một chương trình có thể so sánh ID hệ thống dự kiến với lỗi ID hệ thống và báo lỗi nếu chúng không khớp.

Trong project GHDR: Đóng vai trò Kiểm tra, xác minh ID cho hệ thống tránh thực hiện sai khó gỡ lỗi khi sai ID hệ thống.

2.4.3.3 JTAG to Avalon Master Bridge

Cung cấp kết nối, quyền truy cập giữa hệ thống máy chủ để truy cập các IP được ánh xạ bộ nhớ và các thanh ghi trạng thái và điều khiển IP ngoại vi thông qua giao diện JTAG. Trong Project GHDR đóng vai trò các khối:

- HPS only master
- Fpga only master
- F2sdram only master

2.4.3.4 Avalon MM Pipeline Bridge

Thực hiện chèn một đoạn thanh ghi giữa các đoạn phản hồi và các lệnh ánh xạ đến bộ nhớ Avalon. IP này thực hiện chức năng pipeline, phân phối dữ liệu các tín hiệu câu lệnh từ bộ xử lý và tín hiệu phản hồi từ các component. Ở đây tham số chờ xử lý tối đa là số lần đọc và phản hồi đang chờ xử lý được đưa vào hàng đợi, hàng đợi tối đa là 64.

Trong project GHDR: Dùng IP này với tên đặt mm – bridge với chức năng làm cầu nối để HPS truy cập vào gửi tín hiệu điều khiển ánh xạ đến bộ nhớ các thành phần FPGA.

2.4.3.5 JTAG UART Intel FPGA IP

Một UART JTAG hoạt động giống như một UART thông thường: Một khối có một số thanh ghi trạng thái và điều khiển để ghi byte vào FIFO truyền, đọc byte từ FIFO nhận và kiểm tra xem những FIFO đầy hoặc trống.

Ưu điểm so với UART thông thường:

- Không cần thêm cáp giữa bảng FPGA và PC.
- Không yêu cầu các chân I/O chức năng bổ sung.
- Có thể có nhiều UART JTAG trên mỗi FPGA, ví dụ: Một UART trên mỗi lõi CPU, tất cả trên cùng một kết nối JTAG.
- Tốc độ truyền dữ liệu có thể cao hơn, tùy thuộc vào đồng hồ JTAG.

Nhược điểm:

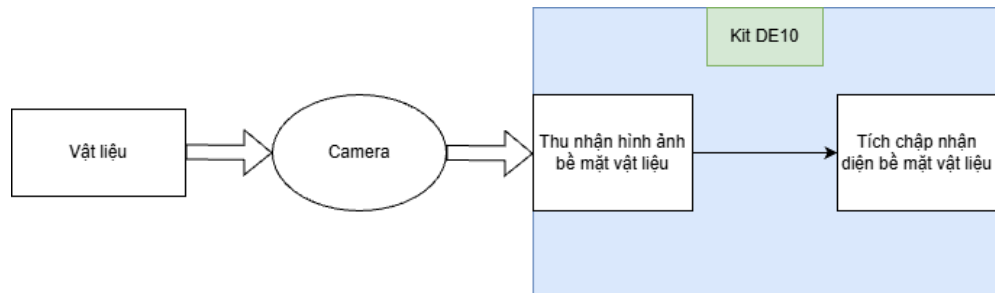
- Không có chuẩn JTAG UART chung mà mỗi nhà sản xuất tự đưa ra một thiết kế riêng.
- Xilinx và Intel có JTAG UART, nhưng cách triển khai của họ rất khác nhau và không tương thích với nhau.

2.4.3.6 Altera Interrupt Latency Counter Intel FPGA IP

Nhằm đo thời gian cần thiết tính theo chu kỳ xung nhịp để hoàn thành quy trình phục vụ ngắt. Dữ liệu thu được từ ILC được các IP với độ trễ khác nhau sử dụng để ILC duy trì hoạt động bình thường. Dữ liệu từ ILC cũng có thể được sử dụng để giúp tiến hành gỡ lỗi.

3. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG

3.1. Tổng quan hệ thống



Hình 3.1 Sơ đồ khối hoạt động của hệ thống.

Hệ thống phát hiện khiếm khuyết bề mặt vật liệu được thực hiện với minh họa ứng dụng kiểm tra vật liệu có bị lỗi qua hình ảnh từ camera. Một số ứng dụng của hệ thống này có thể kể đến như: Kiểm tra bề mặt thép và kết cấu công trình, kiểm tra chất lượng gạch men và vật liệu xây dựng, kiểm định linh kiện điện tử,...

Chức năng của hệ thống khi đưa vào ứng dụng nhận diện khiếm khuyết gồm:

- Thứ nhất, hệ thống lấy dữ liệu hình ảnh từ camera ở dạng video
- Thứ hai, hệ thống thực hiện xử lý trên FPGA và gửi kết quả nhận dạng lỗi khiếm khuyết bề mặt vật liệu lên màn hình máy tính.

Tóm lại, quá trình 1 chu kỳ thực hiện kiểm tra diễn ra như sau:

1. Bắt đầu lần kiểm tra xem vật liệu có bị lỗi không khi có vật liệu được đưa vào (vị trí mà camera bắt được khung hình chứa vật liệu đó).
2. Hệ thống bắt đầu nhận diện khiếm khuyết và trả kết quả tương ứng với lỗi mà nó gặp phải lên màn hình để người xem có thể quan sát trực quan.

3.2. Yêu cầu thiết kế

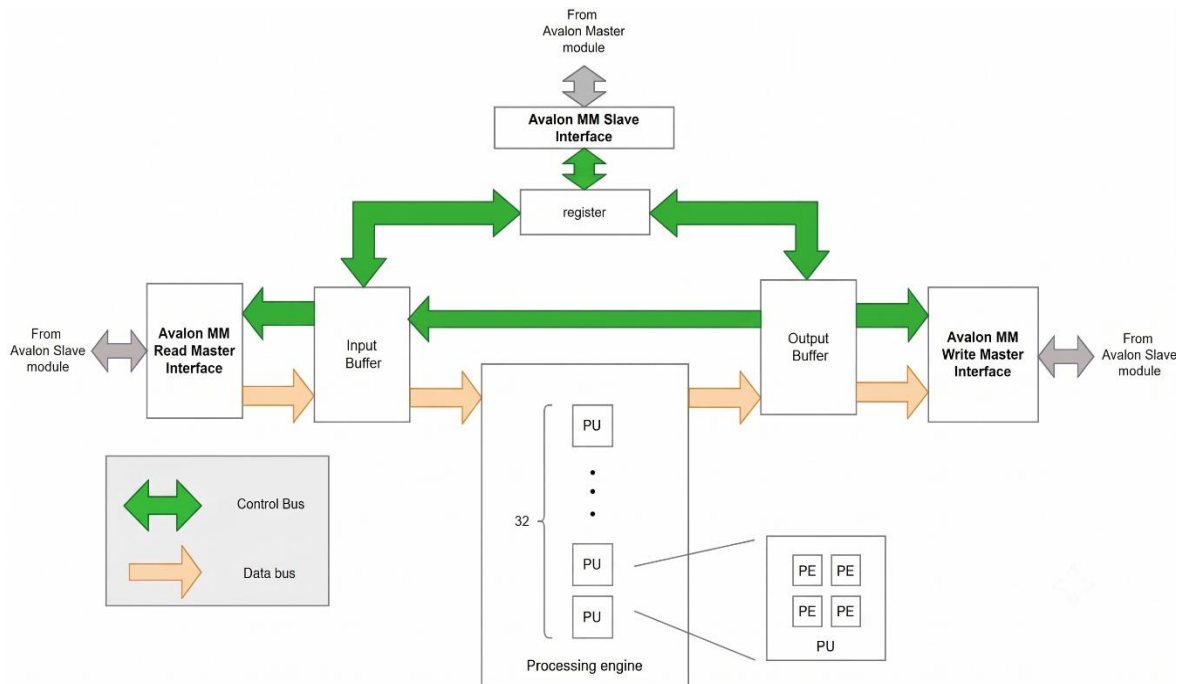
Dựa vào tổng quan hệ thống đã đề cập ở phần 3.1, hệ thống phần cứng của đề tài thiết kế sẽ bao gồm:

Phát hiện khiếm khuyết bề mặt vật liệu dựa trên FPGA, chúng ta sẽ sử dụng board DE10 Standard. Cụ thể yêu cầu một phần cứng có chức năng thực hiện tích chập từ feature map đầu vào với các trọng số được lưu ở bộ nhớ trung gian. Để thực hiện được các chức năng đọc, ghi và tính tích chập thì cần có các module như sau:

- Một bộ giao tiếp slave để nhận tín hiệu điều khiển từ HPS
- Một thanh ghi dùng để lưu giữ các tín hiệu điều khiển đó
- Các bộ giao tiếp master để đọc/ghi dữ liệu đến vùng nhớ
- Một bộ đếm đầu vào, một bộ đếm đầu ra
- Một bộ thực hiện tích chập

3.3. Thiết kế phần cứng

Phần cứng thực hiện tích chập trong đề tài bao gồm module thực hiện chức năng tính tích chập được mô tả bằng ngôn ngữ phần cứng Verilog. Phần cứng tích chập này sinh viên tham khảo trong dự án “Optimizing Convolutional Neural Network Accelerator on Low-Cost FPGA” của tác giả Trương Quang Vinh và Đinh Việt Hai [3] đã được kiểm chứng về chức năng và hoạt động; phần cứng tích chập này tên là “cnn_top”; Phần cứng được mô tả như Hình 3.2. Nhờ vào phần cứng này, đề tài xây dựng hệ thống SoC dựa trên các lõi IP sẵn có, cấu hình để hệ thống thực hiện tích chập cho giải thuật phát hiện khiếm khuyết bề mặt vật liệu. Đồng thời, đề án còn bao gồm thiết kế chương trình phần mềm để điều khiển hoạt động của phần cứng thông qua vi xử lý ARM nhằm hiện thực hóa một mạng thần kinh tích chập trên board SoC FPGA để có thể phát hiện khiếm khuyết bề mặt vật liệu.



Hình 3.2 Thiết kế phần cứng. [3]

Bên trong thiết kế phần cứng là các khối riêng biệt được liên kết bởi các bus dữ liệu hoặc bus điều khiển. Mỗi khối đảm nhận chức năng khác nhau trong quá trình xử lý các lớp tích chập. Các chức năng của từng khối này sẽ được mô tả ngắn gọn như sau:

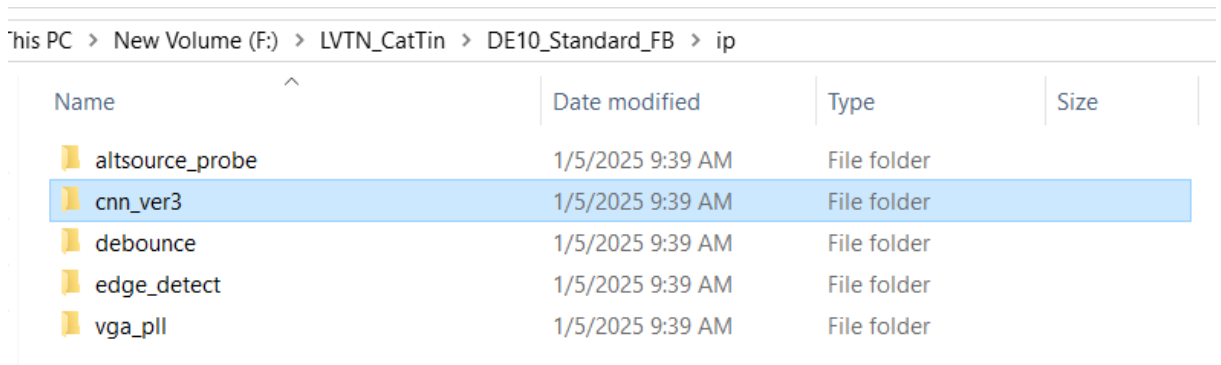
- Giao tiếp Avalon master/slave: Điều khiển các tín hiệu giao tiếp Avalon.
- Register: Các thanh ghi dùng để chứa thông tin điều khiển của HPS (thông qua Avalon MM Slave Interface) về kích hoạt thực hiện tích chập, kích thước dữ liệu, báo đã thực hiện xong tích chập, địa chỉ đọc, địa chỉ ghi và báo tràn dữ liệu.
- Input buffer: Bộ đệm chứa dữ liệu của input feature map, weight và bias nhận từ vùng nhớ trung gian thông qua Avalon MM Read Master Interface; đồng thời truyền dữ liệu vào bộ Processing Engine.
- Processing Engine: Bộ tính tích chập gồm 32 bộ PU, mỗi bộ PU gồm 4 bộ PE, mỗi bộ PE là một bộ accumulator có chức năng nhân và cộng dồn.
- Output buffer: Bộ đệm dữ liệu ngõ ra từ Processing Engine và truyền dữ liệu đến vùng nhớ trung gian thông qua Avalon MM Write Master Interface.

3.4. Thực hiện phần cứng

Để thực hiện phần cứng, ta cần kết nối IP “cnn_ver3” tích chập vào mô hình GHRD của Altera, ta có thể thực hiện thông qua công cụ Quartus của Intel để đổ xuống kit DE10- Standard. Ta sử dụng mô hình GHRD với các IP sẵn có do Intel hỗ trợ để kết nối khối “cnn_top”, HPS, nguồn clock, IP VGA (xuất kết quả ra màn hình)... với nhau để phát hiện và nhận diện khiếm khuyết bề mặt vật liệu. Ngoài ra, cần phải có các bước để thực hiện một phần cứng có khả năng đổ lên board và giao tiếp được với vi xử lý ARM là cấu hình hệ thống FPGA thông qua các kết nối trên Qsys và biên dịch hệ thống FPGA thông qua phần mềm Quartus phiên bản 17.0.

3.4.1 Cấu hình hệ thống FPGA

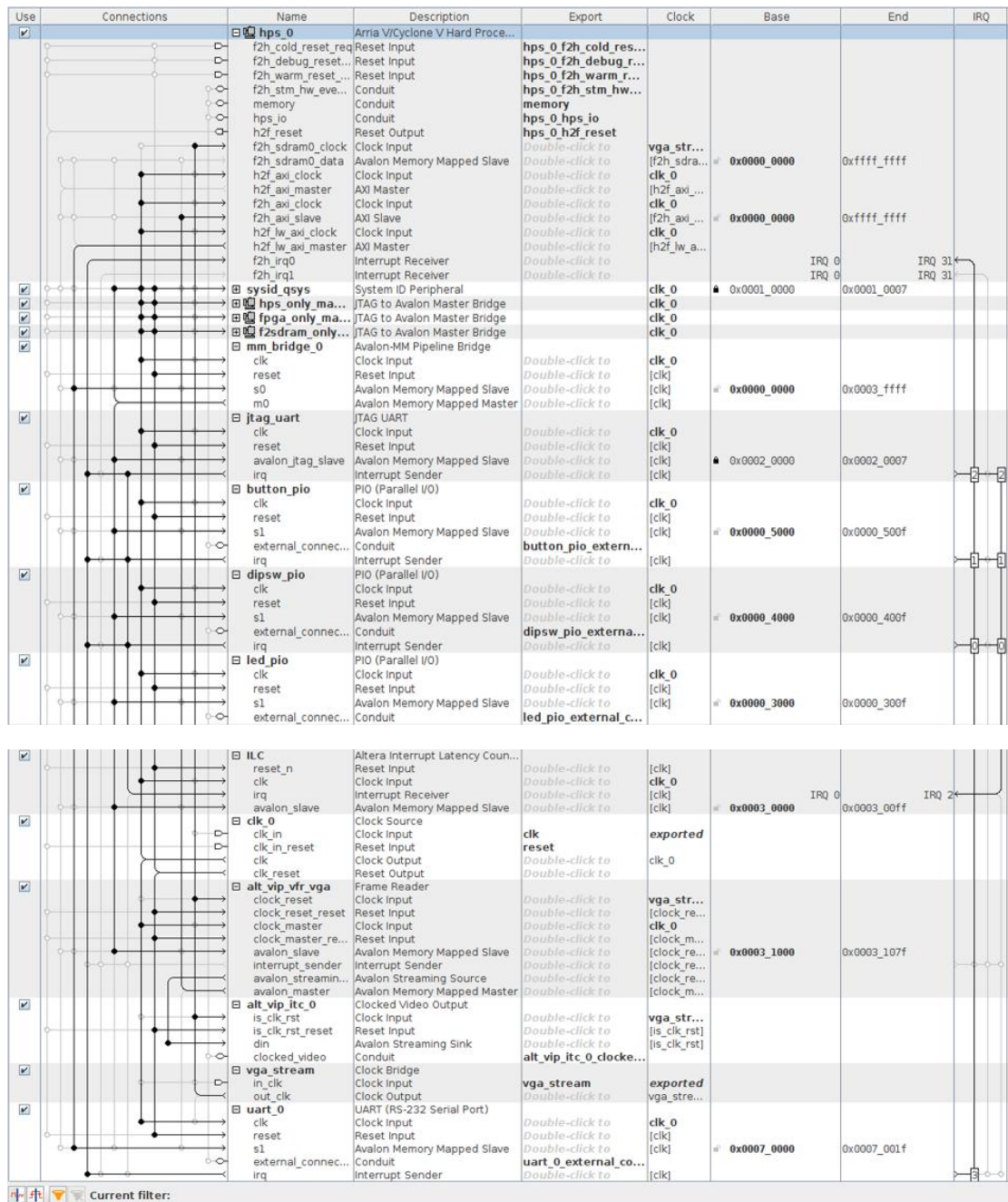
Trước tiên, sao chép IP tên là “cnn_ver3” vào trong mục ip của folder DE10_Standard_FB.



this PC > New Volume (F:) > LVTN_CatTin > DE10_Standard_FB > ip			
Name	Date modified	Type	Size
altsource_probe	1/5/2025 9:39 AM	File folder	
cnn_ver3	1/5/2025 9:39 AM	File folder	
debounce	1/5/2025 9:39 AM	File folder	
edge_detect	1/5/2025 9:39 AM	File folder	
vga_pll	1/5/2025 9:39 AM	File folder	

Hình 3.3 File cnn_ver3 trong thư mục ip

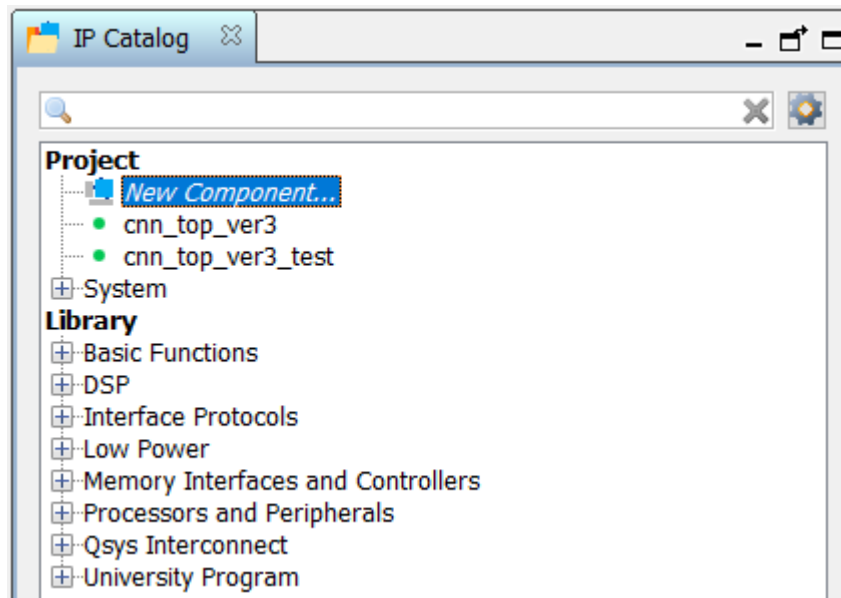
Mở công cụ Qsys cung cấp bởi Quartus và mở hệ thống GHRD như hình minh họa 4.3.2



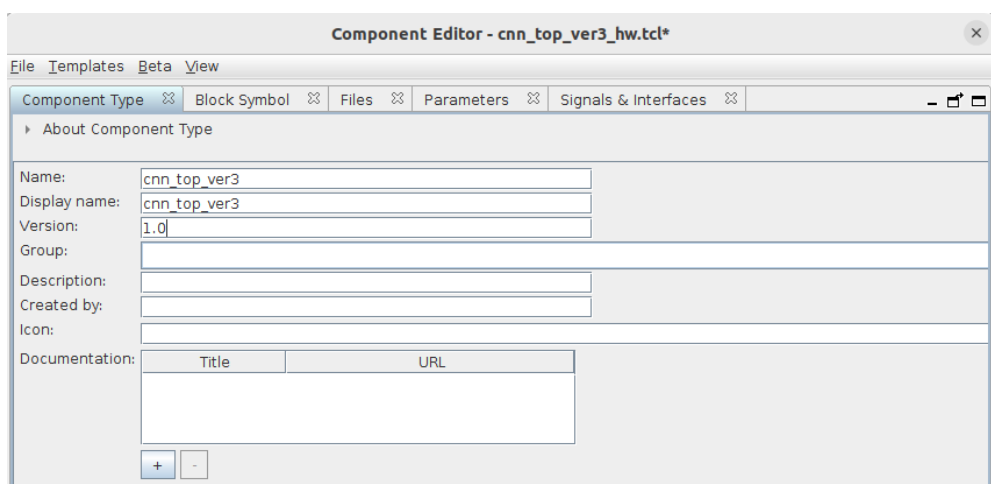
Hình 3.4 Các kết nối của GHRD trong Qsys.

Kế tiếp, nhấn đúp vào *New Component* và nhập tên thiết kế là “cnn_top_ver3” và sau đó chuyển sang mục *Files* để thêm vào các file code RTL đã có sẵn. Trong mục *Synthesis Files*, chọn Add File và chọn tất cả các file RTL và nhấn Enter. Tìm file “cnn_top” trong bảng liệt kê các file ở Synthesis Files, nhấn vào mục *Attributes* và tick chọn Top Level files để đặt file cnn_top

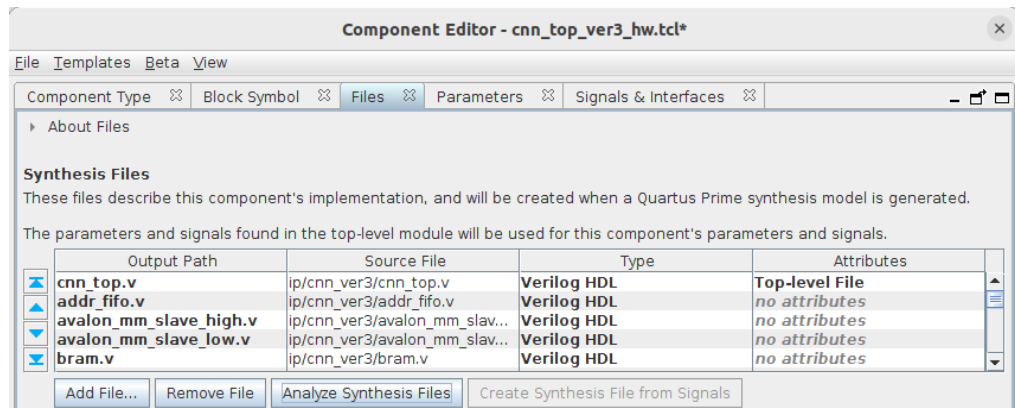
này là Top Design. Sau đó nhấn vào Analyze Synthesis Files, nếu thành công thì qua bước tiếp theo.



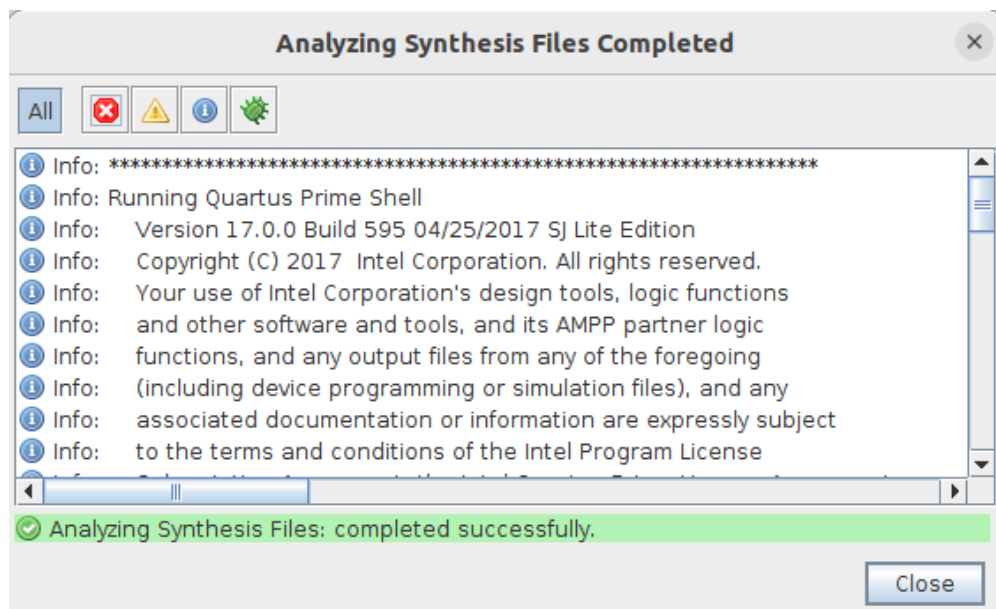
Hình 3.5 Mục New Component khi mở Qsys



Hình 3.6 Nhập tên thiết kế là “cnn_top_ver3”



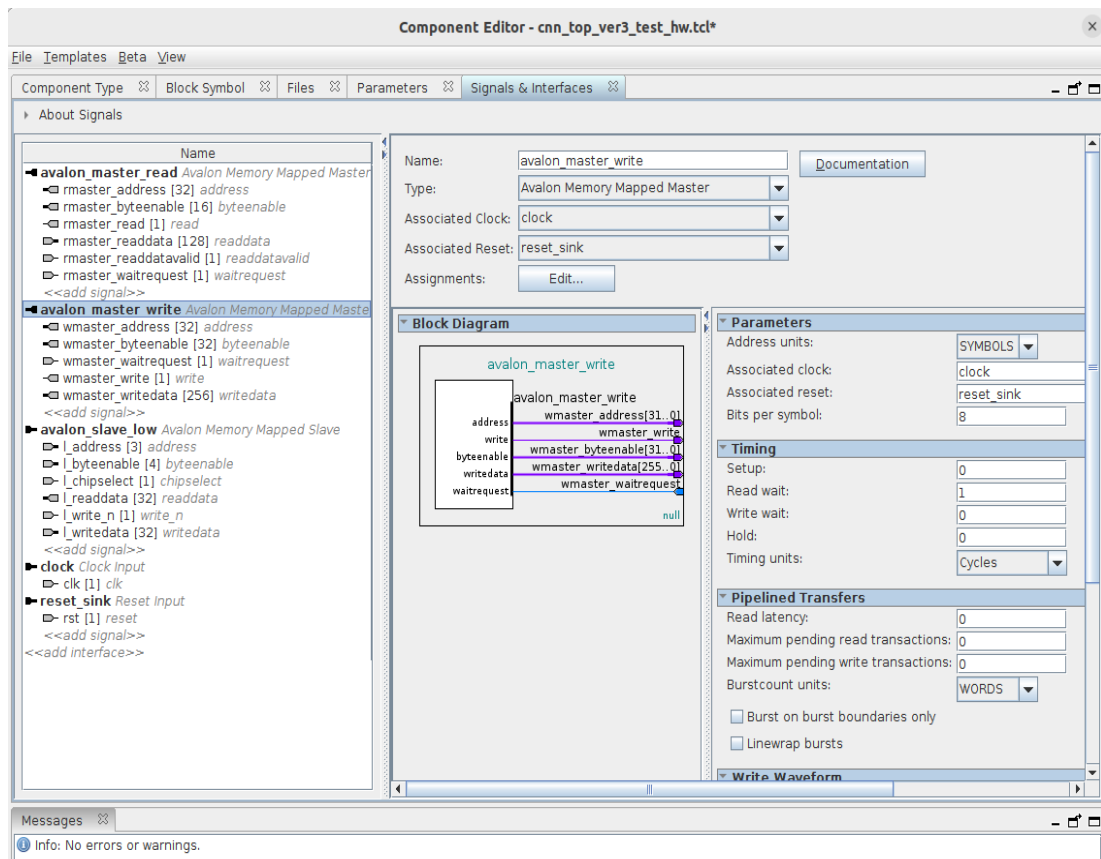
Hình 3.7 Thêm code RTL và đặt file cnn_top.v là Top Design



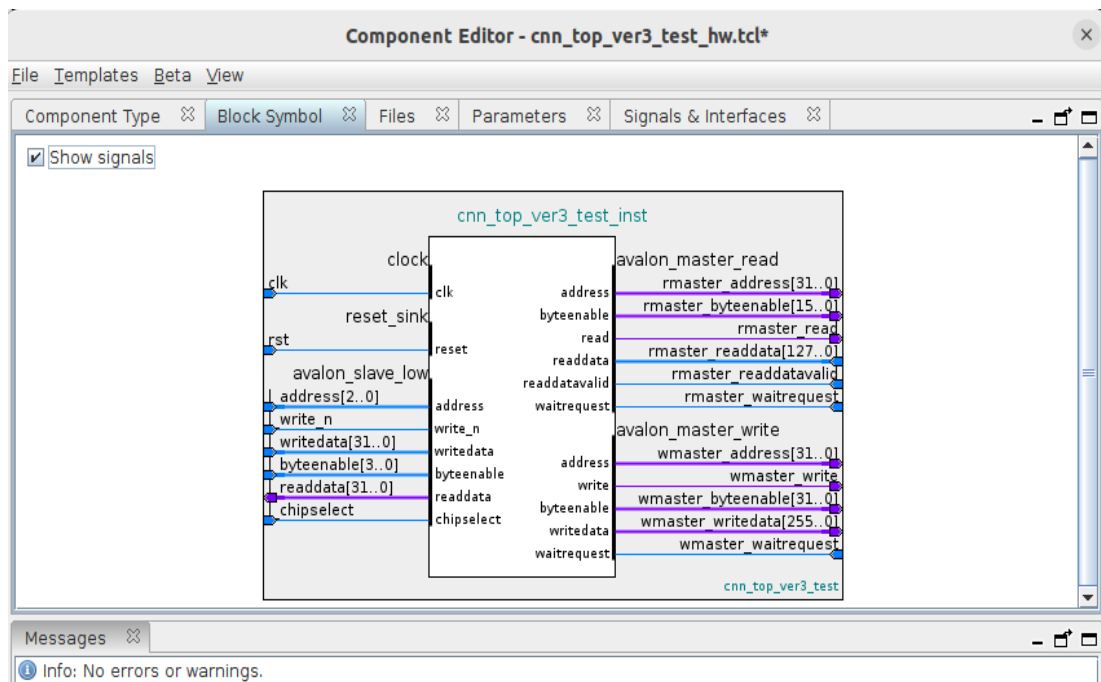
Hình 3.8 Analyze Synthesis Files thành công

Kế đến chuyển qua thẻ *Signals & Interfaces* để đặt các cổng giao tiếp tín hiệu. Ở mục này, Qsys cho phép nhóm lại các tín hiệu in và out tùy biến theo ý người sử dụng. Các tín hiệu này ban đầu được sắp xếp mặc định và người thiết kế cần thay đổi lại các cổng này theo ý tưởng thiết kế hệ thống của mình. Trên sơ đồ thiết kế sẵn có, IP “cnn_ver3” này gồm có 2 cổng giao tiếp Memory Mapped Master và 1 cổng giao tiếp Memory Mapped Slave, cùng với 1 clock và 1 reset. Để tạo interface mới, nhấn chọn <<add interface>> và chọn loại giao tiếp cần sử dụng. Sau khi tạo interface mới, ta có thể đổi tên interface này trong mục Name. Đồng thời, cần chọn Signal Type khớp với miêu tả tên của

các cổng giao tiếp này (ví dụ, rmaster_waitrequest có Signal Type là waitrequest nhưng mặc định để là beginbursttransfer, vì vậy cần chọn lại cho khớp). Lưu ý mục Messages, nếu trong quá trình chỉnh sửa *Signals & Interfaces* có interfaces nào chưa được chọn clock và reset hoặc sai kiểu input và output thì Qsys sẽ báo lỗi ở mục này.

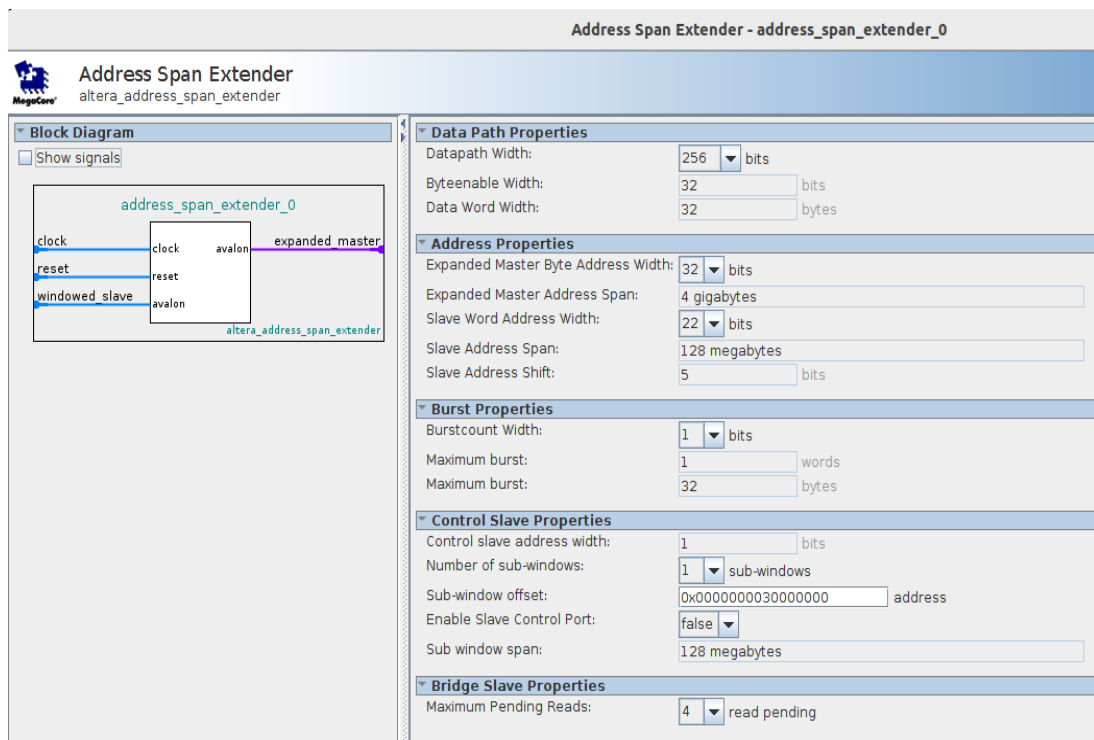


Hình 3.9 Signals & Interfaces của IP “cnn_top_ver3”

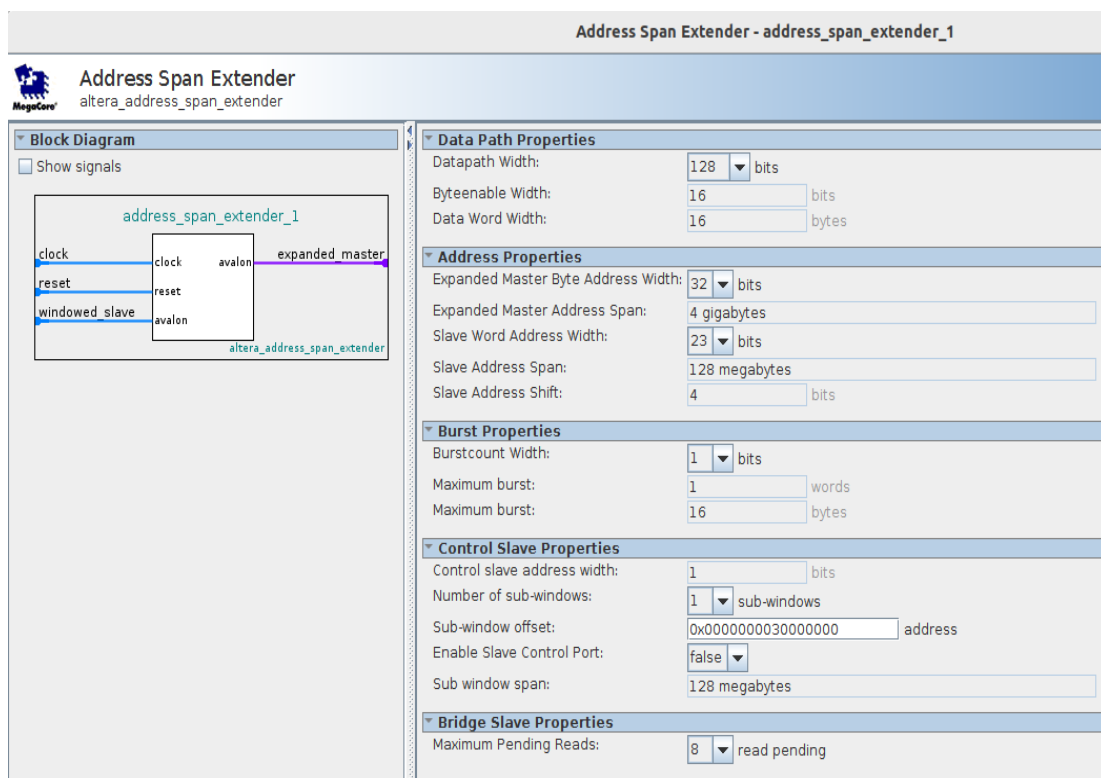


Hình 3.10 Block Symbol của IP “cnn_top_ver3”

Sau khi hoàn tất bước này, nhấn Finish → Chọn Save. Sau đó IP này đã xuất hiện ở trong mục component, có thể thêm IP này vào *System Contents* bằng cách nhấn đúp vào tên IP này. IP mới sẽ xuất hiện trong mục *System Contents* và sẵn sàng để kết nối vào hệ thống. Cuối cùng, chúng ta sẽ kết nối IP vào hệ thống Qsys, đồng thời ta sẽ cần thêm các modules bổ sung cho hệ thống như hình bên dưới.

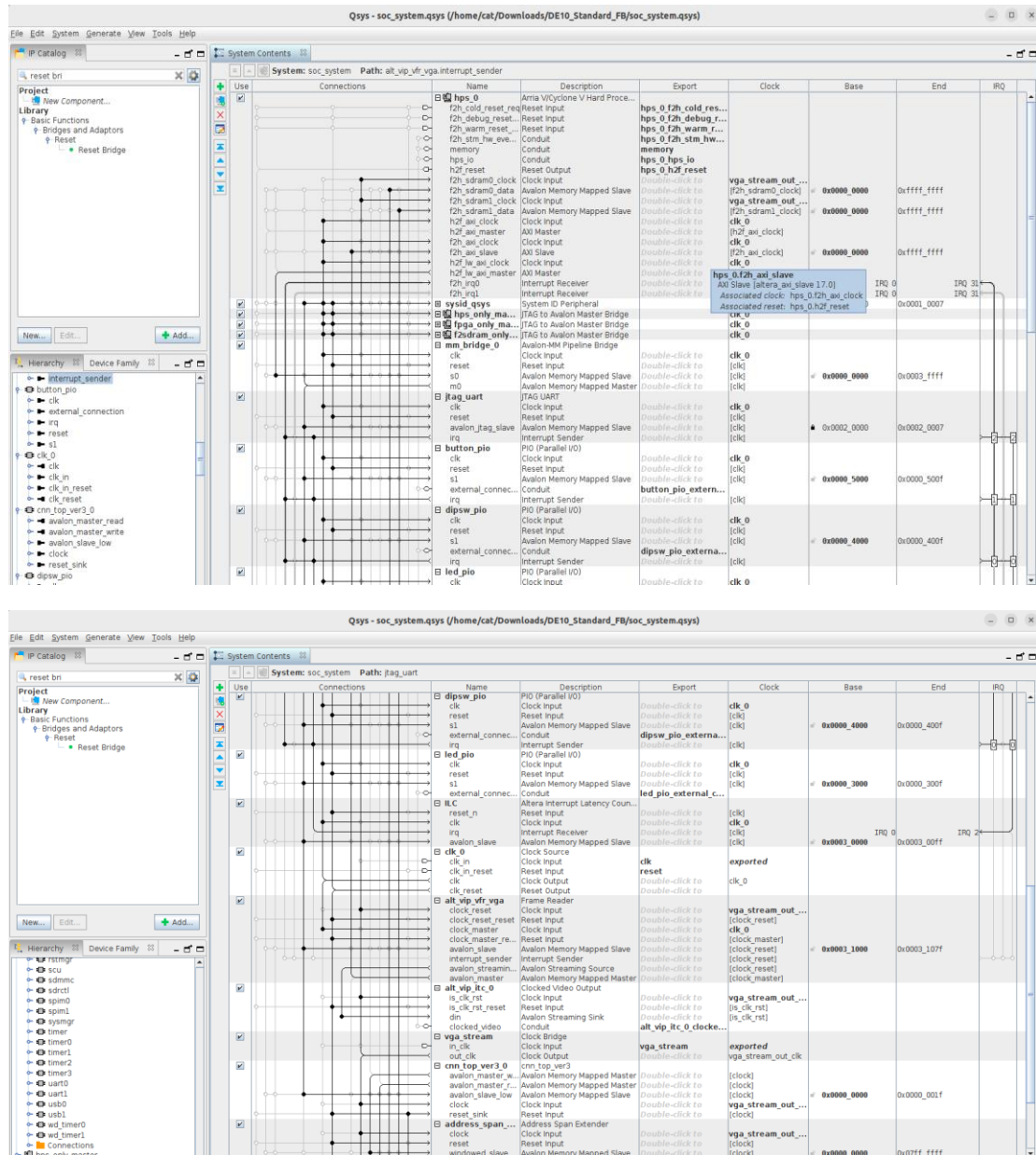


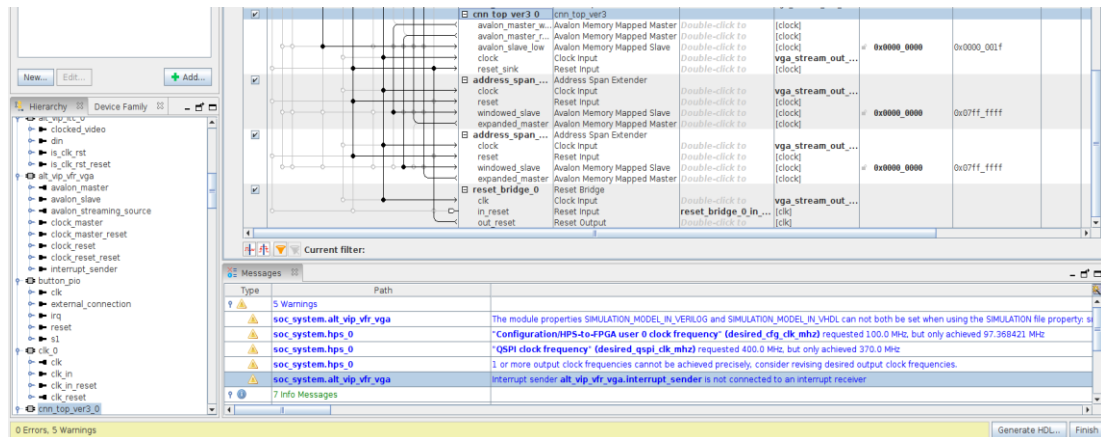
Hình 3.11 IP “address_span_extender_0”



Hình 3.12 IP “address_span_extender_1”

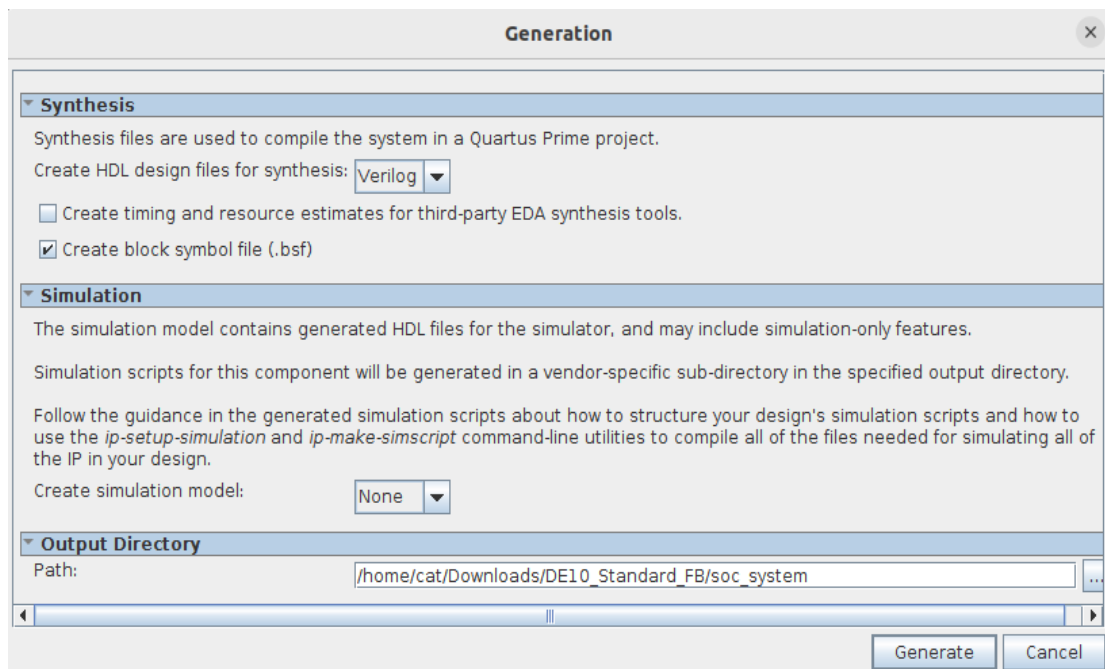
Cuối cùng là thực hiện kết nối IP vào hệ thống hoàn chỉnh như hình như



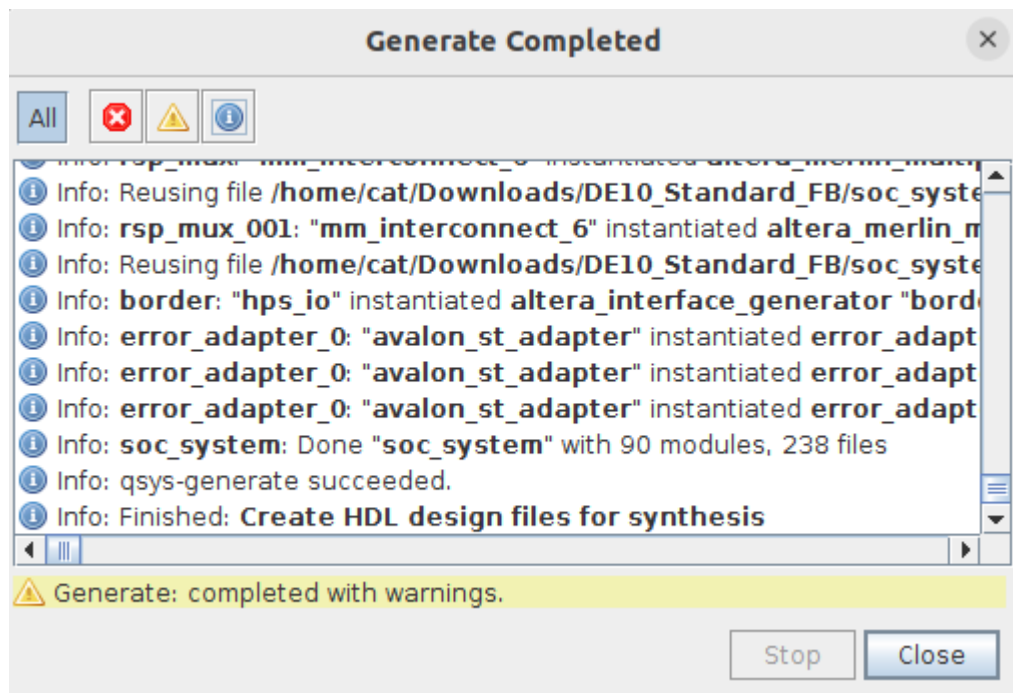


Hình 3.13 Kết nối hoàn chỉnh hệ thống SoC trên Qsys

Khi đã cấu hình được các kết nối trong phần cứng, ta có thể tạo ra file .sopcinfo theo trình tự *Generate > Generate HDL*. File .sopcinfo được board sử dụng để định địa chỉ, các kết nối và độ rộng bit của dữ liệu các khối bên trong phần cứng.



Hình 3.14 Giao diện Generate HDL



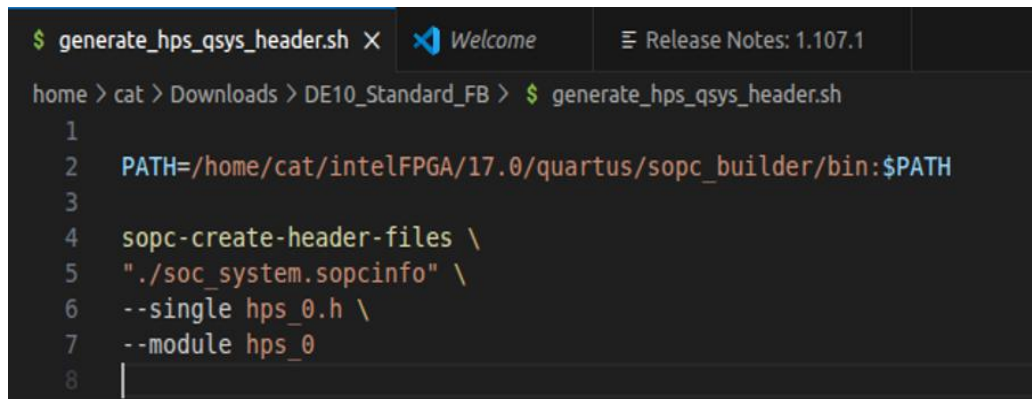
Hình 3.15 Tạo file .sopcinfo

PC > New Volume (F:) > LVTN_CatTin > DE10_Standard_FB

Name	Date modified	Type	Size
DE10_Standard_FB.qws	7/20/2026 10:09 PM	QWS File	1 KB
hps_sdram_p0_summary	7/19/2026 7:46 PM	Microsoft Excel Co...	2 KB
c5_pin_model_dump	7/19/2026 7:44 PM	Text Document	5 KB
DE10_Standard_FB.qsf	7/19/2026 7:20 PM	QSF File	73 KB
soc_system.sopcinfo	7/19/2026 5:30 PM	SOPCINFO File	2,918 KB
soc_system	7/19/2026 5:29 PM	QSYS File	61 KB
cnn_top_ver3_hw	7/19/2026 5:29 PM	Altium Script Doc	14 KB

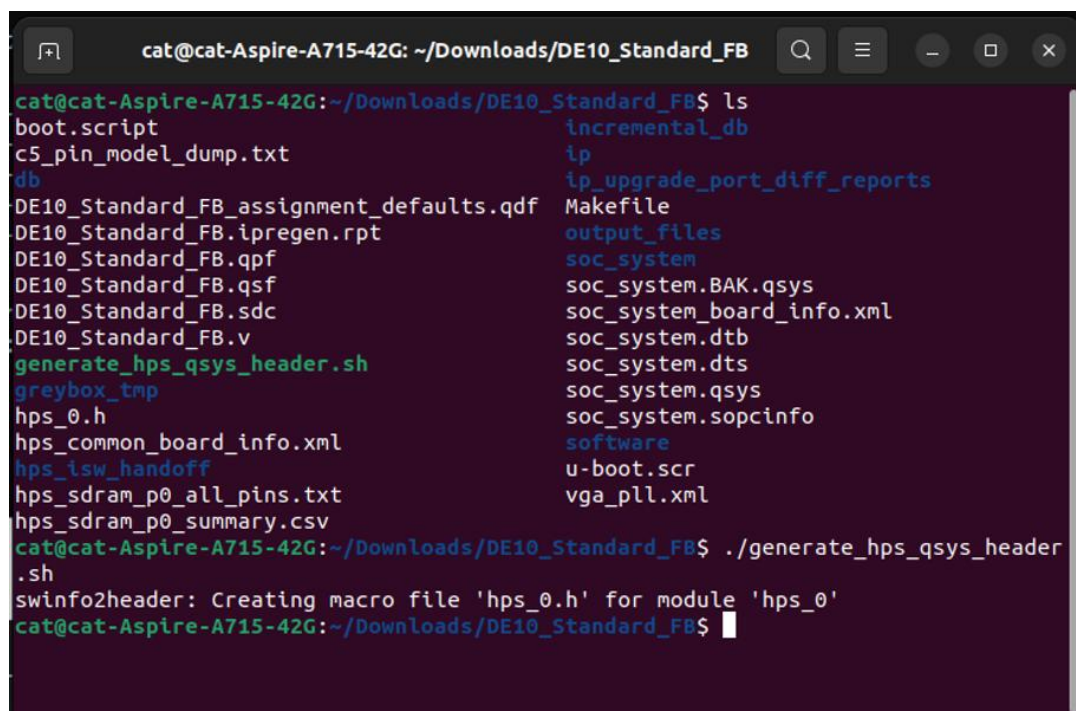
Hình 3.16 Sau khi tạo file .sopcinfo

Ngoài ra, để có file header của HPS được sử dụng trong thiết kế phần mềm ở mục 4.4, ta cần thực hiện một file script tại project DE10_Standard_FB có nội dung như Hình 4.3.15, với PATH là đường dẫn tới mục bin của Quartus. Sau khi hoàn thành, cần bật tool SoC-EDS, trở đến đường dẫn chứa script và thực hiện lệnh `./generate_hps_qsys_header.sh`.



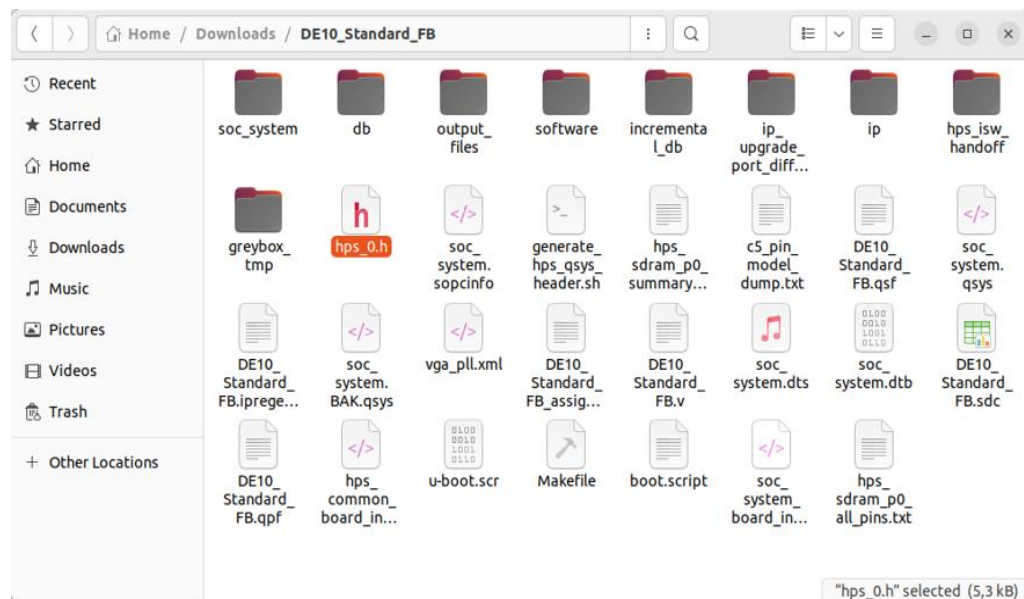
```
$ generate_hps_qsys_header.sh X Welcome Release Notes: 1.107.1
home > cat > Downloads > DE10_Standard_FB > $ generate_hps_qsys_header.sh
1
2 PATH=/home/cat/intelFPGA/17.0/quartus/sopc_builder/bin:$PATH
3
4 socp-create-header-files \
5   "./soc_system.sopcinfo" \
6   --single hps_0.h \
7   --module hps_0
8 |
```

Hình 3.17 Nội dung script tạo file header .h

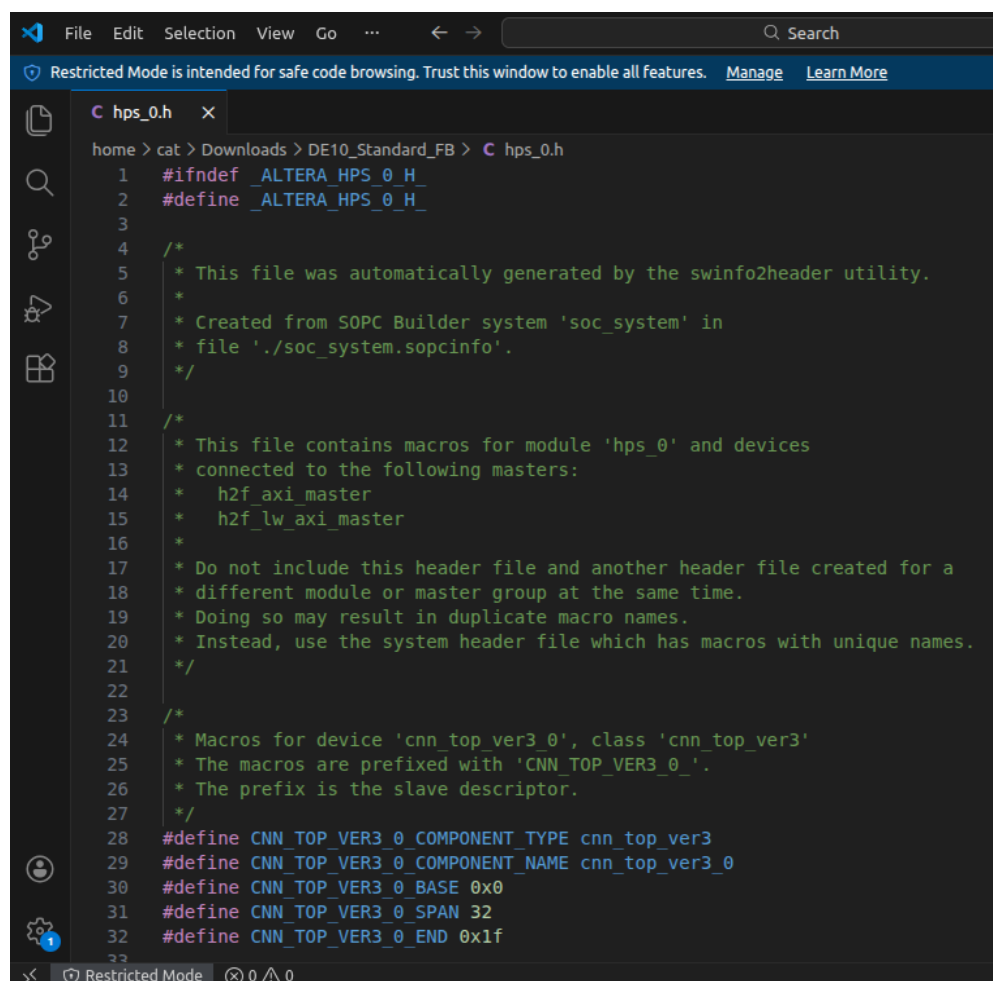


```
cat@cat-Aspire-A715-42G: ~/Downloads/DE10_Standard_FB
cat@cat-Aspire-A715-42G:~/Downloads/DE10_Standard_FB$ ls
boot.script                                incremental_db
c5_pin_model_dump.txt                     ip
db                                          ip_upgrade_port_diff_reports
DE10_Standard_FB_assignment_defaults.qdf  Makefile
DE10_Standard_FB.ipregen.rpt              output_files
DE10_Standard_FB.qpf                      soc_system
DE10_Standard_FB.qsf                      soc_system.BAK.qsys
DE10_Standard_FB.sdc                      soc_system_board_info.xml
DE10_Standard_FB.v                        soc_system.dtb
generate_hps_qsys_header.sh               soc_system.dts
greybox_tmp                              soc_system.qsys
hps_0.h                                  soc_system.sopcinfo
hps_common_board_info.xml                 software
hps_isw_handoff                           u-boot.scr
hps_sdram_p0_all_pins.txt                  vga_pll.xml
hps_sdram_p0_summary.csv
cat@cat-Aspire-A715-42G:~/Downloads/DE10_Standard_FB$ ./generate_hps_qsys_header
.sh
swinfo2header: Creating macro file 'hps_0.h' for module 'hps_0'
cat@cat-Aspire-A715-42G:~/Downloads/DE10_Standard_FB$
```

Hình 3.18 Quá trình chạy script



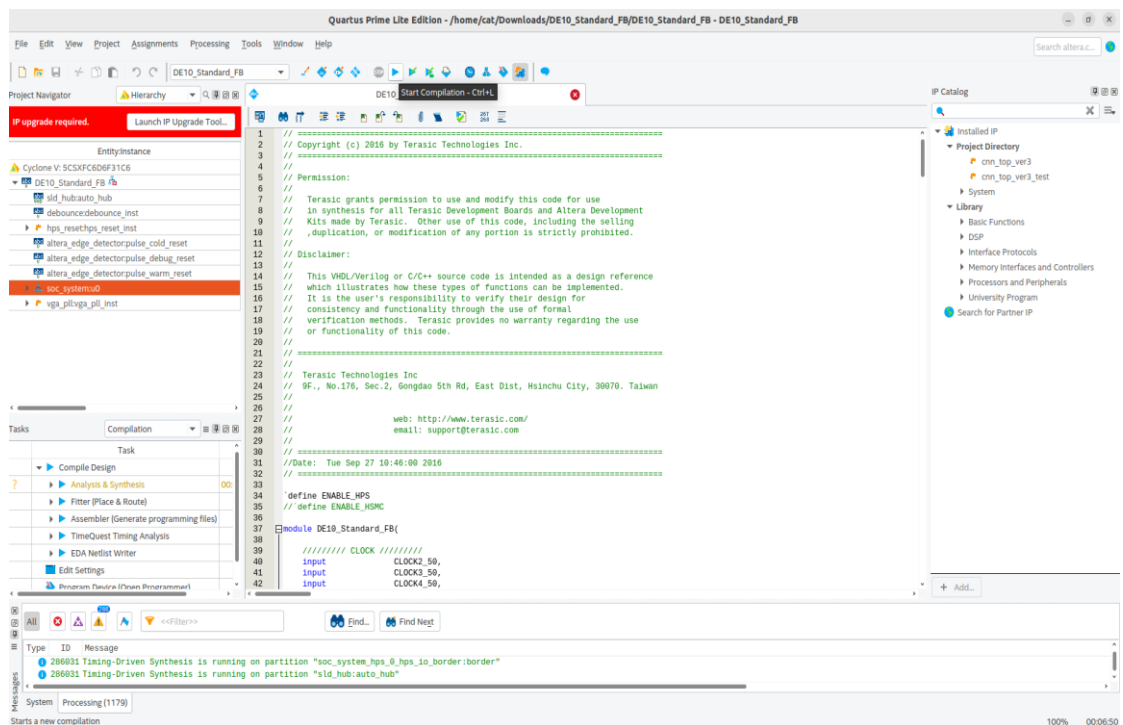
Hình 3.19 Sau khi tạo file .h



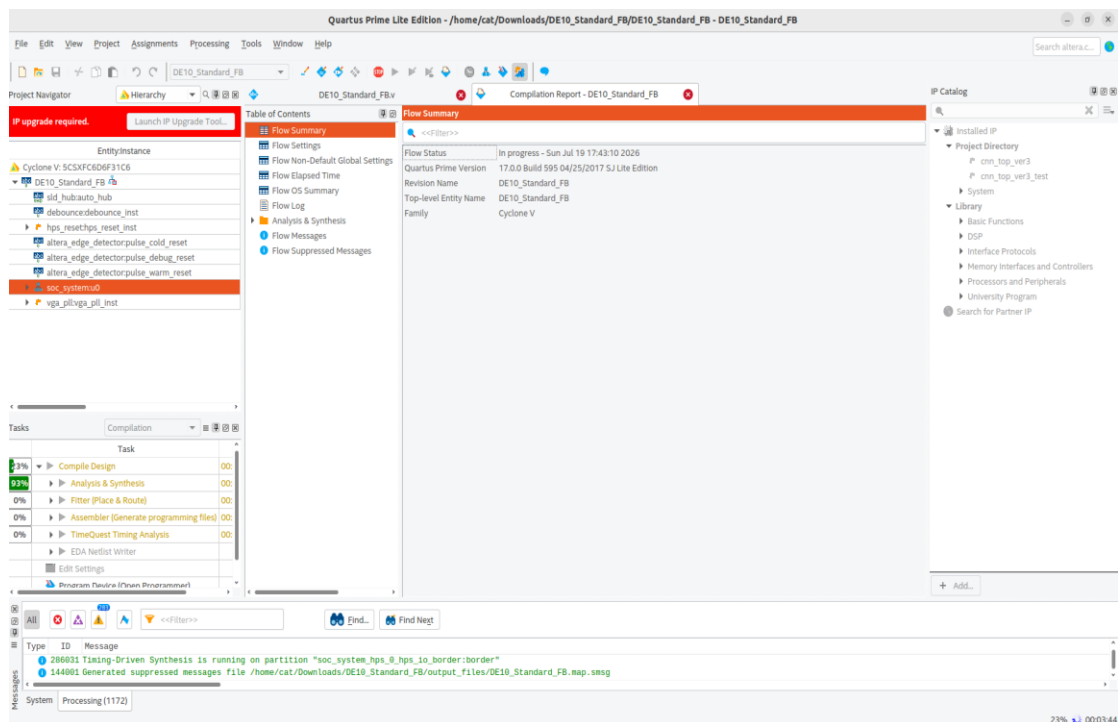
Hình 3.20 Nội dung file hps_0.h vừa tạo

3.4.2 Biên dịch hệ thống FPGA

Để hệ thống SoC có thể được nạp lên board DE10-Standard nhằm hiện thực hóa hệ thống, cần Compile hệ thống bằng cách nhấn vào nút *Start Compilation*.



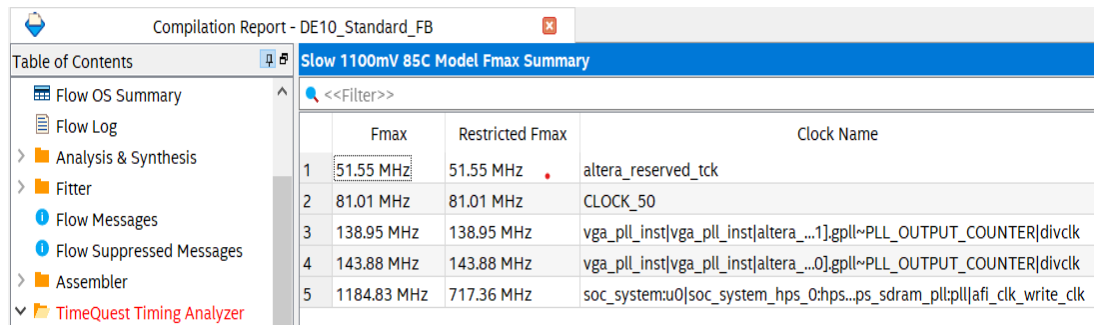
Hình 3.21 Giao diện khi load file DE10_Standard_FB.qpf đã hoàn thành phần Qsys



Hình 3.22 Quartus bắt đầu compile hệ thống

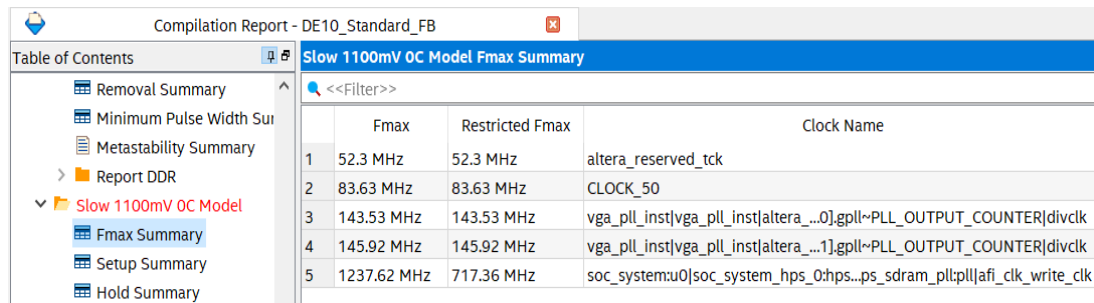
Compilation Report - DE10_Standard_FB	
Table of Contents	Flow Summary
Flow Summary	<<Filter>>
Flow Settings	Flow Status: Successful - Sun Jul 19 19:44:26 2026
Flow Non-Default Global Settings	Quartus Prime Version: 17.0.0 Build 595 04/25/2017 SJ Lite Edition
Flow Elapsed Time	Revision Name: DE10_Standard_FB
Flow OS Summary	Top-level Entity Name: DE10_Standard_FB
Flow Log	Family: Cyclone V
Analysis & Synthesis	Device: 5CSXFC6D6F31C6
Fitter	Timing Model: Device
Flow Messages	Logic utilization (in ALMs): 22,494 / 41,910 (54 %)
Flow Suppressed Messages	Total registers: 45604
Assembler	Total pins: 338 / 499 (68 %)
TimeQuest Timing Analyzer	Total virtual pins: 0
Summary	Total block memory bits: 4,313,198 / 5,662,720 (76 %)
Parallel Compilation	Total DSP Blocks: 112 / 112 (100 %)
SDC File List	Total HSSI RX PCSs: 0 / 9 (0 %)
Clocks	Total HSSI PMA RX Deserializers: 0 / 9 (0 %)
Slow 1100mV 85C Model	Total HSSI TX PCSs: 0 / 9 (0 %)
Fmax Summary	Total HSSI PMA TX Serializers: 0 / 9 (0 %)
Timing Closure Recommendation	Total PLLs: 1 / 15 (7 %)
Setup Summary	Total DLLs: 1 / 4 (25 %)
Hold Summary	
Recovery Summary	
Power Summary	

Hình 3.23 Báo cáo quá trình biên dịch hệ thống FPGA



	Fmax	Restricted Fmax	Clock Name
1	51.55 MHz	51.55 MHz	altera_reserved_tck
2	81.01 MHz	81.01 MHz	CLOCK_50
3	138.95 MHz	138.95 MHz	vga_pll_inst[vga_pll_inst[altera_...1].gpll~PLL_OUTPUT_COUNTER]divclk
4	143.88 MHz	143.88 MHz	vga_pll_inst[vga_pll_inst[altera_...0].gpll~PLL_OUTPUT_COUNTER]divclk
5	1184.83 MHz	717.36 MHz	soc_system:u0[soc_system_hps_0:hps...ps_sdram_pll:pll]afi_clk_write_clk

Hình 3.24 Báo cáo phân tích timing Slow 1100mV 85C của hệ thống



	Fmax	Restricted Fmax	Clock Name
1	52.3 MHz	52.3 MHz	altera_reserved_tck
2	83.63 MHz	83.63 MHz	CLOCK_50
3	143.53 MHz	143.53 MHz	vga_pll_inst[vga_pll_inst[altera_...0].gpll~PLL_OUTPUT_COUNTER]divclk
4	145.92 MHz	145.92 MHz	vga_pll_inst[vga_pll_inst[altera_...1].gpll~PLL_OUTPUT_COUNTER]divclk
5	1237.62 MHz	717.36 MHz	soc_system:u0[soc_system_hps_0:hps...ps_sdram_pll:pll]afi_clk_write_clk

Hình 3.25 Báo cáo phân tích timing Slow 1100mV 0C của hệ thống

Ở đề tài này, sau khi biên dịch thì hệ thống FPGA đã tiêu tốn các loại tài nguyên phần cứng bao gồm các module logic có thể lập trình được (ALM), bộ xử lý tín hiệu số, các khối RAM, bộ nhớ on-chip, vòng khóa pha PLL và vòng khóa độ trễ DLL theo Bảng 4.2.

Thiết kế phần cứng đã tiêu thụ hoàn toàn các khối DSP và RAM đi cùng với một nửa ALM của Cyclone V. Ngoài các tài nguyên tiêu tốn, với việc sử dụng clock 50 MHz thì hệ thống đạt được tần số cao nhất khoảng 83.63 MHz. Đối với hệ thống SoC, RAM DDR3 yêu cầu tần số 130MHz nhưng RAM DDR3 hoạt động được tần số 700 MHz, việc này cho thấy hệ thống SoC hoạt động ở tần số cao theo Hình 4.3.22 và 4.3.23.

Bảng 3.1 Tài nguyên hệ thống FPGA tiêu tốn

Loại tài nguyên	Giải thuật đề nghị	Tài nguyên của Cyclone V
ALMs	22,494 (54%)	41,910

DSPs	112 (100%)	112
RAMs	553 (100%)	553
On-chip memory (Kb)	4,313,198 (76%)	5,662,720
PLLs	1 (7%)	15
DLLs	1 (25%)	4

Ngoài ra, sau khi Compile thì hệ thống sẽ xuất file .sof. File này được dùng để nạp hệ thống phần cứng lên board ở mục 4.4.

is PC > New Volume (F:) > LVTN_Cat > DE10_Standard_FB > output_files

Name	Date modified	Type	Size
.qsys_edit	1/5/2025 9:39 AM	File folder	
sof_to_rbf	1/18/2017 6:44 AM	Windows Batch File	1 KB
soc_system.rbf	6/6/2017 7:18 AM	RBF File	2,150 KB
DE10_Standard_FB.map.smsg	7/19/2026 7:24 PM	SMSG File	14 KB
DE10_Standard_FB.map.summary	7/19/2026 7:28 PM	SUMMARY File	1 KB
DE10_Standard_FB.map	7/19/2026 7:28 PM	Report File	34,668 KB
DE10_Standard_FB.pin	7/19/2026 7:43 PM	PIN File	103 KB
DE10_Standard_FB.fit.smsg	7/19/2026 7:43 PM	SMSG File	1 KB
DE10_Standard_FB.fit.summary	7/19/2026 7:43 PM	SUMMARY File	1 KB
DE10_Standard_FB.fit	7/19/2026 7:43 PM	Report File	17,409 KB
DE10_Standard_FB.jdi	7/19/2026 7:44 PM	JDI File	40 KB
DE10_Standard_FB.sld	7/19/2026 7:44 PM	SLD File	1 KB
DE10_Standard_FB.sof	7/19/2026 7:44 PM	SOF File	7,280 KB
DE10_Standard_FB.som	7/19/2026 7:44 PM	Report File	5 KB

Hình 3.26 File .sof được tạo sau quá trình biên dịch

4. THỰC HIỆN PHẦN MỀM

4.1 Yêu cầu thiết kế

Dựa vào tổng quan thiết kế hệ thống đã nêu thì phần mềm sẽ gồm: Phần mềm cho hệ thống SOC trên board DE10.

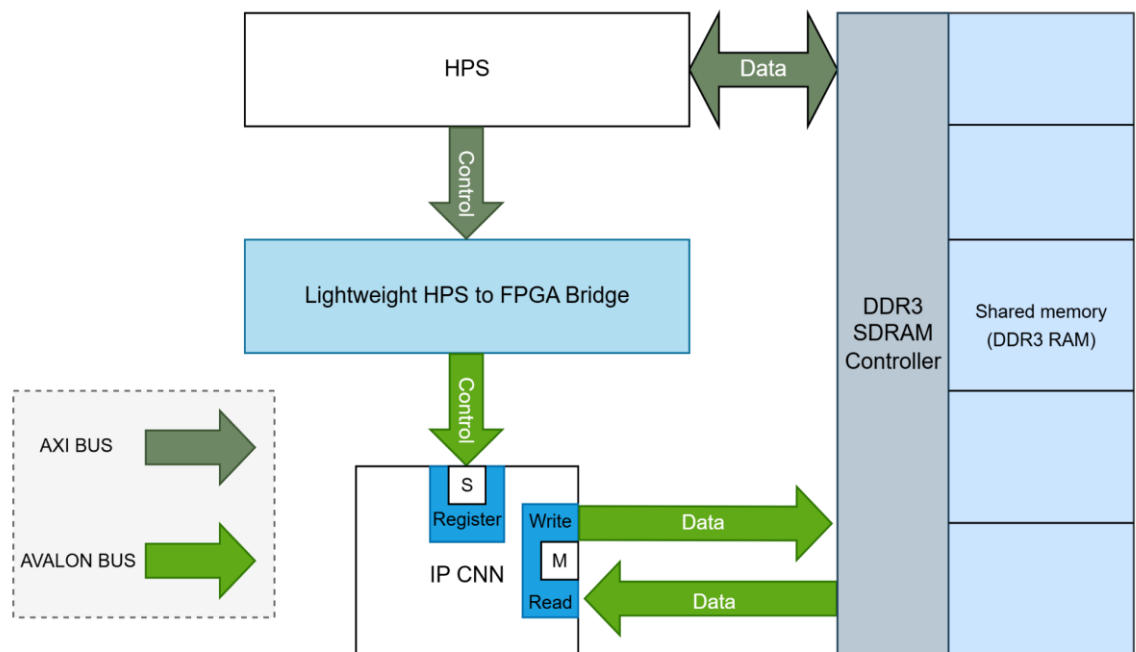
Phần mềm cho hệ thống SoC được hiện thực để chạy trên vi xử lý ARM Cortex-A9 tích hợp trong board SoC-FPGA bao gồm các chức năng chính sau:

- Thu nhận dữ liệu từ camera
- Điều khiển phần cứng thực hiện tích chập
- Nhận diện khiếm khuyết bề mặt vật liệu
- Gửi dữ liệu kết quả thu thập được qua màn hình hiển thị

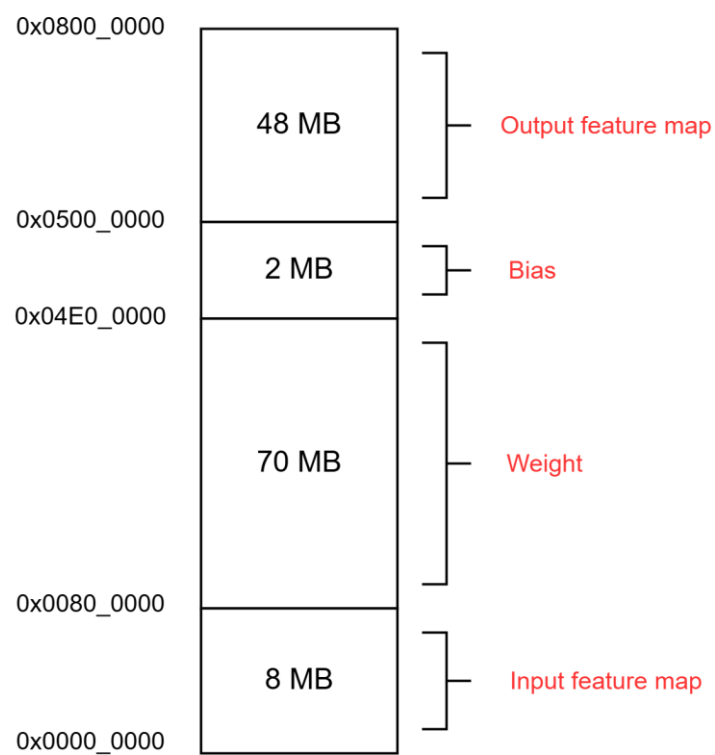
4.2 Thiết kế và thực hiện phần mềm cho hệ thống trên kit DE10

4.2.1 Thiết kế phần mềm

Trong quá trình thiết kế phần mềm cho hệ thống SOC, khối IP tích chập sử dụng bus Avalon MM để trao đổi thông tin. Vì thế, vi xử lý ARM sẽ sử dụng bus AXI thông qua cầu Lightweight HPS-to-FPGA để kết nối. Ngoài ra, cần có vùng nhớ trung gian (shared memory) RAM DDR3 để trao đổi dữ liệu giữa các thành phần trong board DE10 theo Hình 4.4.1. Phân vùng vùng nhớ trung gian RAM DDR3 được quy định như trong Hình 4.4.2. Với việc phân chia vùng nhớ như vậy thì hệ thống có thể luân phiên truy input và output feature map bởi ngõ ra của lớp tích chập trước sẽ là ngõ vào của lớp tích chập sau.



Hình 4.1 Các luồng dữ liệu giữa HPS và khối IP CNN

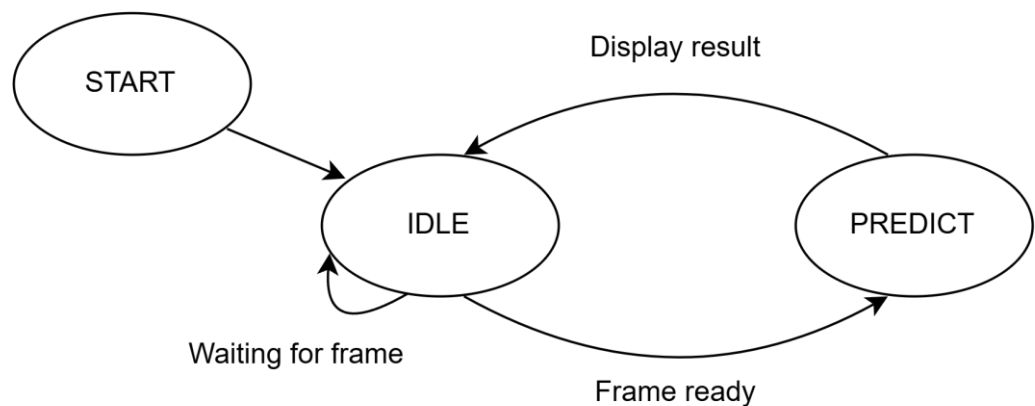


Hình 4.2 Phân vùng nhớ RAM DDR3

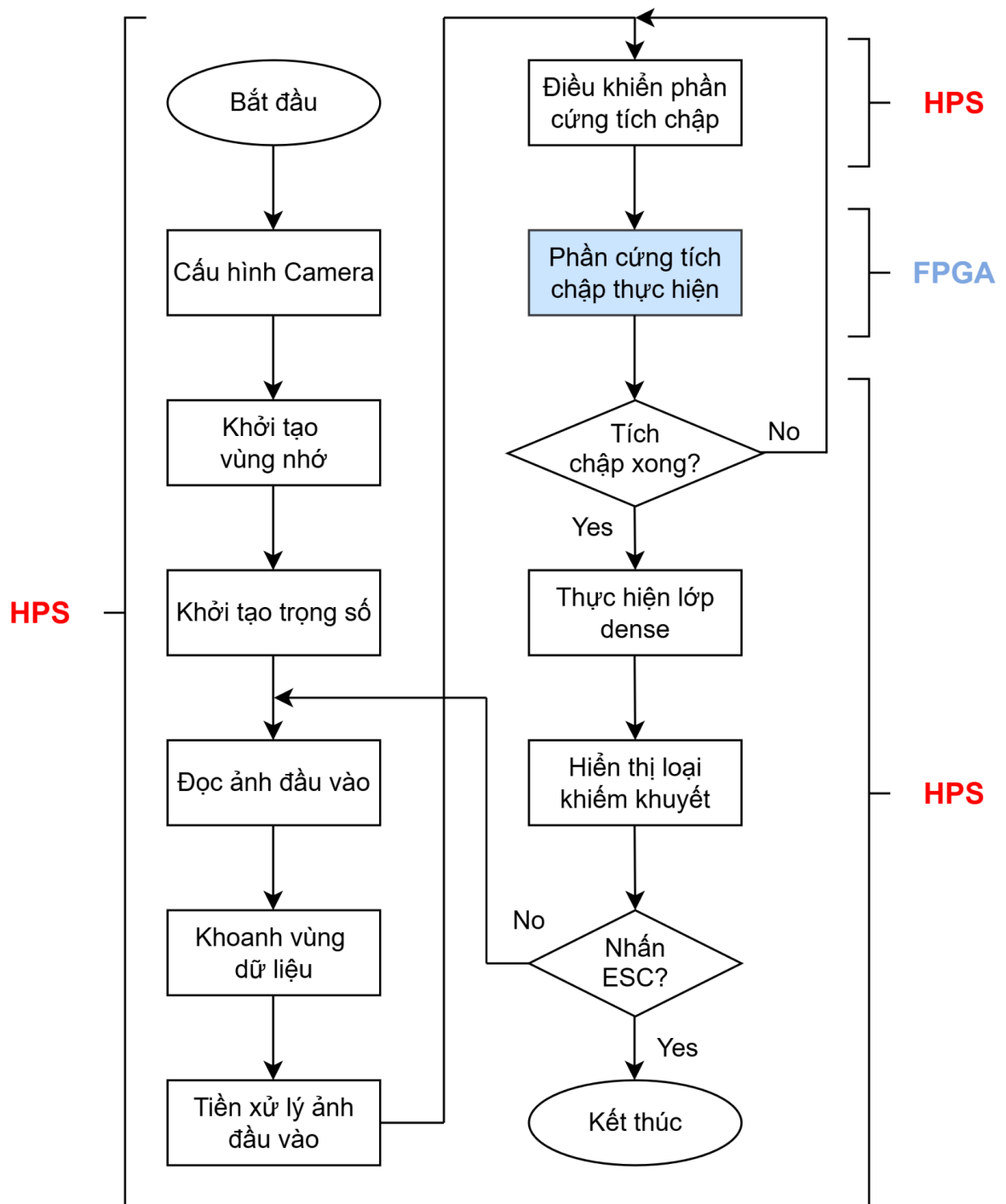
Thông qua việc thực hiện hệ thống FPGA, định hình được vùng nhớ trung gian cũng như định hình được dữ liệu trọng số đã huấn luyện, tiếp theo sẽ là code C/C++ để điều khiển vi xử lý ARM. Ngoài bốn chức năng chính đã nêu ra ở phần yêu cầu thiết kế thì chương trình phần mềm cần phải được thực hiện theo quy trình như sau:

- Khởi tạo các vùng nhớ ảo
- Khởi tạo các trọng số và lưu vào vùng nhớ
- Đọc ảnh đầu vào từ camera và khoanh vùng dữ liệu
- Tiền xử lý hình ảnh đầu vào
- Điều khiển phần cứng thực hiện tích chập
- Thực hiện lớp dense và trả kết quả nhận diện

Với các yêu cầu đã đề ra, máy trạng thái của hệ thống phần mềm được xác định như Hình 4.3. Hệ thống sau khi được khởi động sẽ rơi vào trạng thái IDLE để luôn nhận dữ liệu từ camera; nếu phát hiện khung hình thì hệ thống sẽ chuyển qua trạng thái PREDICT nhằm tiến hành nhận diện khiếm khuyết trên đó, trả kết quả lên màn hình, đồng thời hệ thống cũng sẽ trở lại trạng thái IDLE. Và cụ thể hơn thì lưu đồ chương trình phần mềm chính được chú thích như Hình 4.4 đại diện cho hoạt động của hệ thống SOC trong suốt quá trình từ khởi tạo camera để phát hiện khiếm khuyết bề mặt vật liệu đến việc trả kết quả nhận diện và gửi dữ liệu ra màn hình rồi.

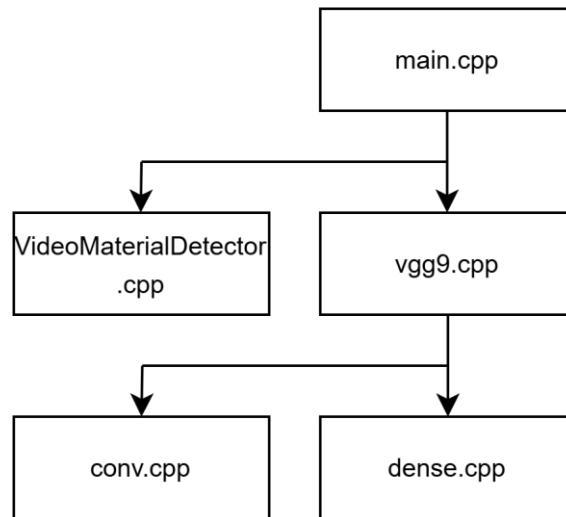


Hình 4.3 Máy trạng thái phần mềm



Hình 4.4 Lưu đồ phần mềm điều khiển phần cứng HPS trên vi xử lý ARM

Ngoài ra, cấu trúc chương trình phần mềm theo mục đích sử dụng để nhận diện khiếm khuyết được mô tả theo Hình 4.5, gồm 5 files code C.



Hình 4.5 Cấu trúc chương trình phần mềm

Theo hình 4.5, module main, là module chính của chương trình, có chức năng khởi tạo vùng nhớ cho hệ thống, đồng thời thu nhập dữ liệu đầu vào từ camera và chuyển cho module VideoMaterialDetector khoanh vùng khung hình. Sau khi phát hiện khung hình sẽ được chuyển cho module vgg9 thực hiện tiền xử lý rồi sau đó phân loại khiếm khuyết thông qua module conv và module dense. Với module conv có nhiệm vụ điều khiển phần cứng FPGA thực hiện tích chập và trả output feature map cuối thành mảng một chiều, còn module dense có nhiệm vụ thực hiện các lớp dense sau khi lớp tích chập đã được tính xong rồi trả kết quả nhận diện. Cuối cùng kết quả sau khi được nhận diện sẽ được gửi ra màn hình rồi tại hàm main. Các hàm chính trong các module trên được mô tả chức năng như sau:

Bảng 4.1 Các hàm chính trong main.cpp

Hàm	Chức năng
Void mmap (void *address, size_t length, int protect, int flags, int fildes, off_t offset)	Tạo không gian bộ nhớ ảo chia sẻ dữ liệu trực tiếp giữa CPU ARM và FPGA, lấy nội dung có dung lượng là <i>length</i> trong <i>fildes</i> bỏ đi offset gán vào vùng nhớ ảo từ địa chỉ <i>address</i> .
Void init_mem_conv()	Khởi tạo vùng nhớ trên RAM DDR3 và nạp trọng số cho các lớp tích chập.
Void init_memory()	Khởi tạo vùng nhớ trên RAM HPS cho các lớp Dense.
Void read_weights()	Đọc trọng số và lưu vào vùng nhớ của lớp Dense.
Int vgg9_predict()	Thực hiện dự đoán phân loại 5 loại khiếm khuyết trên bề mặt vật liệu.
Void free_memory()	Giải phóng các bộ nhớ đã cấp phát để tránh rò rỉ tài nguyên.

Bảng 4.2 Các hàm chính trong VideoMaterialDetector.cpp

Hàm	Chức năng
cv::VideoCapture camera(0)	Khởi tạo ngõ vào video để lấy dữ liệu tĩnh từ camera.
VideoMaterialDetector()	Cấu hình thiết bị truyền khung hình.
isMaterialFound()	Kiểm tra và xác nhận camera có trả về khung hình bề mặt vật liệu hợp lệ hay không.

<code>getROI()</code>	Thuật toán cắt vùng quan tâm (ROI) tính ở trung tâm ảnh nhằm loại bỏ viền thừa trước khi đưa vào IP CNN.
-----------------------	--

Bảng 4.3 Các hàm chính trong vgg9.cpp

Hàm	Chức năng
<code>Int vgg9_predict()</code>	Tiền xử lý, đóng gói điểm ảnh vào DDR3, đồng thời gọi hàm <code>vgg9_convolution</code> (tích chập trên FPGA) và hàm <code>get_VGG9_predict</code> (phân loại khiếm khuyết trên ARM).

Bảng 4.4 Các hàm chính trong conv.cpp

Hàm	Chức năng
<code>Void init_mem_conv()</code>	Khởi tạo vùng nhớ và gán trọng số vào vùng nhớ đó.
<code>Void write_weight() / write_bias()</code>	Đọc file trọng số/bias từ thẻ nhớ và ghi xuống vùng nhớ DDR3.
<code>Void vgg9_convolution()</code>	Điều phối vòng lặp thực hiện quét lần lượt 7 lớp tích chập của mạng VGG9 trên FPGA.
<code>Void implement_a_layer()</code>	Tính toán và truyền thông số cấu hình của từng lớp xuống thanh ghi của bộ tích chập trên FPGA.
<code>Void write_layer_status()</code>	Ghi dữ liệu cấu hình vào thanh ghi điều khiển.
<code>Void write_start()</code>	Gửi xung kích hoạt để khối IP CNN phản ứng bắt đầu hoạt động.

Void flatten()	Đọc Feature Map cuối cùng và trải thành mảng 1 chiều (2048 phần tử).
----------------	--

Bảng 4.5 Các hàm chính trong dense.cpp

Hàm	Chức năng
Int get_VGG9_predict()	Quản lý luồng tính toán phần mềm từ mảng một chiều sau quá trình trải flatten, trả về nhãn khiếm khuyết có xác suất cao nhất.
Void reset_mem_block_dense()	Khởi tạo lại bộ đệm RAM tạm thời cho lớp Dense.
Void dense()	Thực hiện nhân ma trận đặc trưng với trọng số.
Void add_bias_and_relu_flatten()	Cộng bias vào kết quả phép nhân và thực hiện hàm kích hoạt phi tuyến ReLU.
Void softmax()	Tính toán hàm Softmax để xuất ra xác suất phân phối cho 5 lớp khiếm khuyết.

Ngoài chương trình mà người dùng thực hiện, Quartus còn hỗ trợ các header giúp định địa chỉ các khối bên trong như thư mục socal/ và file hwlib.h có trong đường dẫn của tool SoC EDS như trong hình 4.6, 4.7 và 4.8

Windows-SSD (C:) > intelFPGA_pro > 18.1 > embedded > ip > altera > hps > altera_hps > hwlib > include

Name	Date modified	Type	Size
alt_can_private.h	9/22/2018 10:12 PM	C Header Source File	33 KB
alt_dma.h	9/22/2018 10:12 PM	C Header Source File	41 KB
alt_dma_common.h	9/22/2018 10:12 PM	C Header Source File	4 KB
alt_dma_program.h	9/22/2018 10:12 PM	C Header Source File	38 KB
alt_eth_phy_ksz9031.h	9/22/2018 10:12 PM	C Header Source File	12 KB
alt_ethernet.h	9/22/2018 10:12 PM	C Header Source File	49 KB
alt_generalpurpose_io.h	9/22/2018 10:12 PM	C Header Source File	46 KB
alt_globaltmr.h	9/22/2018 10:12 PM	C Header Source File	19 KB
alt_i2c.h	9/22/2018 10:12 PM	C Header Source File	91 KB
alt_interrupt.h	9/22/2018 10:12 PM	C Header Source File	40 KB
alt_interrupt_common.h	9/22/2018 10:12 PM	C Header Source File	6 KB
alt_i2_p310.h	9/22/2018 10:12 PM	C Header Source File	28 KB
alt_mmu.h	9/22/2018 10:12 PM	C Header Source File	79 KB
alt_mpu_registers.h	9/22/2018 10:12 PM	C Header Source File	8 KB
alt_nand.h	9/22/2018 10:12 PM	C Header Source File	50 KB
alt_printf.h	9/22/2018 10:12 PM	C Header Source File	4 KB
alt_qspi.h	9/22/2018 10:12 PM	C Header Source File	71 KB
alt_sdmmc.h	9/22/2018 10:12 PM	C Header Source File	72 KB
alt_spi.h	9/22/2018 10:12 PM	C Header Source File	67 KB
alt_timers.h	9/22/2018 10:12 PM	C Header Source File	27 KB
alt_watchdog.h	9/22/2018 10:12 PM	C Header Source File	31 KB
hwlib.h	9/22/2018 10:12 PM	C Header Source File	7 KB

Hình 4.6 Đường dẫn của file hwlib.h

Windows-SSD (C:) > intelFPGA_pro > 18.1 > embedded > ip > altera > hps > altera_hps > hwlib > include > soc_a10

Name	Date modified	Type	Size
socal	11/25/2022 11:03 AM	File folder	
alt_bridge_manager.h	9/22/2018 10:12 PM	C Header Source File	11 KB
alt_clock_manager.h	9/22/2018 10:12 PM	C Header Source File	12 KB
alt_config.h	9/22/2018 10:12 PM	C Header Source File	3 KB
alt_dma_periph.h	9/22/2018 10:12 PM	C Header Source File	6 KB
alt_ecc.h	9/22/2018 10:12 PM	C Header Source File	24 KB
alt_fpga_manager.h	9/22/2018 10:12 PM	C Header Source File	35 KB
alt_int_device.h	9/22/2018 10:12 PM	C Header Source File	15 KB
alt_reset_manager.h	9/22/2018 10:12 PM	C Header Source File	10 KB
alt_sdram.h	9/22/2018 10:12 PM	C Header Source File	11 KB
alt_system_manager.h	9/22/2018 10:12 PM	C Header Source File	11 KB
altx_ethernet.h	9/22/2018 10:12 PM	C Header Source File	14 KB

Hình 4.7 Đường dẫn của folder socal

Windows-SSD (C:) > intelFPGA_pro > 18.1 > embedded > ip > altera > hps > altera_hps > hwlib > include > soc_a10 > socal

Name	Date modified	Type	Size
alt_noc_mpu_iq.n	9/22/2018 10:12 PM	C Header Source File	23 KB
alt_noc_mpu_m0_main_qos.h	9/22/2018 10:12 PM	C Header Source File	38 KB
alt_noc_mpu_m0_rate_adresp_main_rate.h	9/22/2018 10:12 PM	C Header Source File	29 KB
alt_noc_mpu_m1_main_qos.h	9/22/2018 10:12 PM	C Header Source File	37 KB
alt_noc_mpu_m1toddrresp_main_rate.h	9/22/2018 10:12 PM	C Header Source File	27 KB
alt_noc_mpu_nand_m_main_qos.h	9/22/2018 10:12 PM	C Header Source File	39 KB
alt_noc_mpu_prb.h	9/22/2018 10:12 PM	C Header Source File	694 KB
alt_noc_mpu_sdmmc_m_main_qos.h	9/22/2018 10:12 PM	C Header Source File	39 KB
alt_noc_mpu_usb0_m_main_qos.h	9/22/2018 10:12 PM	C Header Source File	39 KB
alt_noc_mpu_usb1_m_main_qos.h	9/22/2018 10:12 PM	C Header Source File	39 KB
alt_pinmux.h	9/22/2018 10:12 PM	C Header Source File	763 KB
alt_qspi.h	9/22/2018 10:12 PM	C Header Source File	310 KB
alt_rstmgr.h	9/22/2018 10:12 PM	C Header Source File	356 KB
alt_sdmmc.h	9/22/2018 10:12 PM	C Header Source File	443 KB
alt_spim.h	9/22/2018 10:12 PM	C Header Source File	167 KB
alt_spis.h	9/22/2018 10:12 PM	C Header Source File	139 KB
alt_sysmgr.h	9/22/2018 10:12 PM	C Header Source File	611 KB
alt_tmr.h	9/22/2018 10:12 PM	C Header Source File	41 KB
alt_uart.h	9/22/2018 10:12 PM	C Header Source File	341 KB
alt_usb.h	9/22/2018 10:12 PM	C Header Source File	5,707 KB
hps.h	9/22/2018 10:12 PM	C Header Source File	972 KB
socal.h	9/22/2018 10:12 PM	C Header Source File	12 KB

Hình 4.8 Các file header có trong folder socal

Hơn thế nữa, mô hình mạng thần kinh tích chập đề xuất được triển khai với 9 lớp gồm 7 lớp tích chập và 2 lớp fully-connected.

Bảng 4.6 Thông số của mạng noron tích chập đề nghị.

Conv layer	Input size	Output size	Nimap	Nomap	Max pooling
CONV1	64x64	64x64	3	32	0
CONV2	64x64	32x32	32	32	1
CONV3	32x32	32x32	32	32	0
CONV4	32x32	16x16	32	32	1

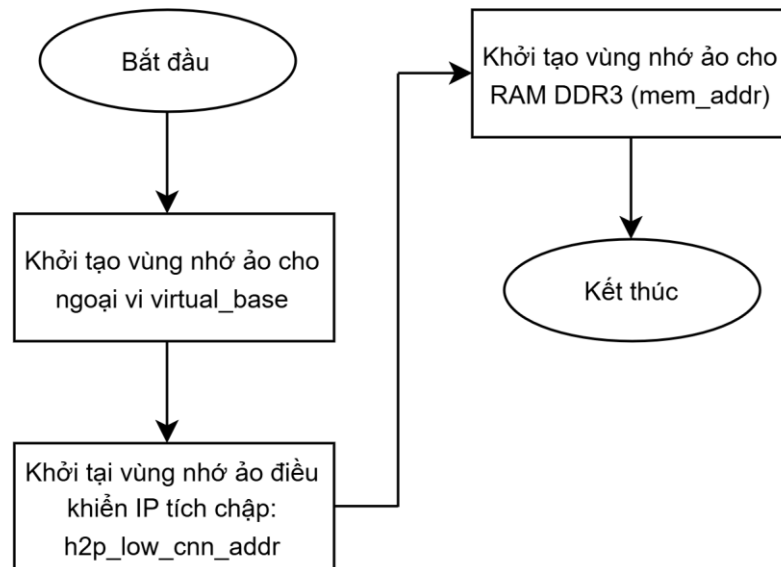
CONV5	16x16	16x16	32	64	0
CONV6	16x16	8x8	64	64	1
CONV7	8x8	4x4	64	128	1
Dense layer		Input size		Output size	
DENSE1		2048		128	
DENSE2		128		5	

4.2.2 Thực hiện phần mềm

Khi đã có được các file header chứa địa chỉ phần cứng và định hình được các module, tiếp theo sẽ là phần thực hiện phần mềm trên môi trường Linux. Ở mục này sẽ trình bày các giải thuật thực hiện các chức năng đã nêu ở mục 4.2.1

4.2.2.1 Khởi tạo vùng nhớ ảo

Việc khởi tạo vùng nhớ ảo sẽ được thông qua hàm mmap. Trong đề tài này, ta sẽ chia ra hai vùng nhớ chính đó là vùng nhớ dùng để chứa các dữ liệu điều khiển các khối phần cứng (kích hoạt tích chập, kích thước dữ liệu,...) và vùng nhớ chứa dữ liệu trong quá trình tích chập (trọng số weight và bias, hình đầu vào,...). Giải thuật trình khởi tạo các vùng nhớ này được thể hiện qua Hình 4.9.



Hình 4.9 Lưu đồ chương trình khởi tạo vùng nhớ ảo

- Khởi tạo vùng nhớ chứa dữ liệu điều khiển phản cứng:

```

virtual_base = mmap( NULL, HW_REGS_SPAN, ( PROT_READ | PROT_WRITE
), MAP_SHARED, fd, HW_REGS_BASE );

h2p_low_cnn_addr = virtual_base + ((unsigned long)(ALT_LWFPGASLVS_OFST
+ CNN_TOP_VER3_0_BASE) & (unsigned long)( HW_REGS_MASK ));
    
```

Trong đó:

Bảng 4.7 Các địa chỉ ngoại vi sử dụng trong phần mềm.

Parameter	Base
HW_REGS_SPAN	0x0400_0000
HW_REGS_BASE	0xFC00_0000
ALT_LWFPGASLVS_OFST	0xFF20_0000
CNN_TOP_VER3_0_BASE	0x00
HW_REGS_MASK	0x03FF_FFFF

Hàm `mmap` dùng để ánh xạ vùng nhớ vật lý của ngoại vi (`HW_REGS_BASE`) đến địa chỉ ảo `virtual_base`. Để điều khiển được các ngoại vi, người dùng có thể tính toán các địa chỉ ảo bằng cách cộng thêm các offset liên quan trở đến ngoại vi vào địa chỉ ảo `virtual_base`. Offset đầu tiên là `0xFF20_0000` của `Lightweight HPS-FPGA (ALT_LWFPGASLVS_OFST)` được định nghĩa trong file header `hps.h` có trong thư mục `socal/`. Còn offset thứ hai là `0x00` được định nghĩa trong file header `hps_0.h` của `CNN_TOP_VER3_0_BASE` đã được tạo ở mục 3.4.1.

- Khởi tạo vùng nhớ cho quá trình tích chập ở phần cứng:

```
mem_addr = ( unsigned long *)mmap( NULL, 0x8000000, ( PROT_READ
| PROT_WRITE ), MAP_SHARED, fd, 0x30000000 );
```

Trong đó: Hàm `mmap` như đã đề cập sẽ ánh xạ vùng nhớ của RAM DDR3 với địa chỉ `0x3000_0000` đến địa chỉ ảo `mem_addr`.

Vùng `mem_addr` được vi xử lý ARM dùng để thực hiện gửi các dữ liệu (hình ảnh, weight và bias) đến RAM DDR3; sau đó, khối IP sẽ đọc các dữ liệu đó cùng với các tín hiệu điều khiển thông qua `h2p_low_cnn_addr` để thực hiện quá trình tích chập theo lệnh điều khiển của vi xử lý ARM.

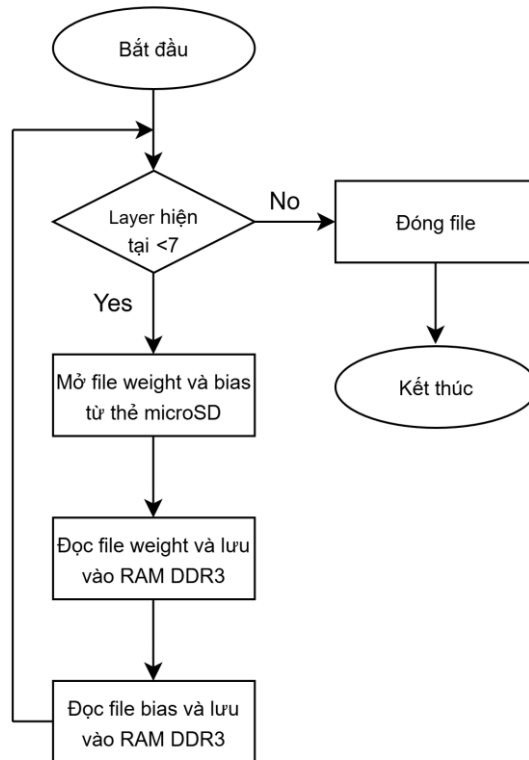
4.2.2.2 Khởi tạo trọng số

Trước khi thực hiện quá trình nhận diện, hệ thống phần cứng cần được lưu các trọng số (được khởi tạo ở mục 5.1.1) và chỉ có bộ nhớ RAM DDR3 mới có thể đáp ứng nhu cầu dữ liệu cầu lưu lớn và truy xuất nhanh. Đồng thời, các file dữ liệu đã qua xử lý sẽ được lưu dưới dạng `.txt` ở thẻ nhớ; hệ thống sẽ đọc từ đó và lưu vào RAM DDR3 cho quá trình thực hiện mạng thần kinh tích chập. Việc khởi tạo của hai quá trình trên được mô tả qua hai mục con như bên dưới.

- Khởi tạo trọng số cho quá trình tích chập

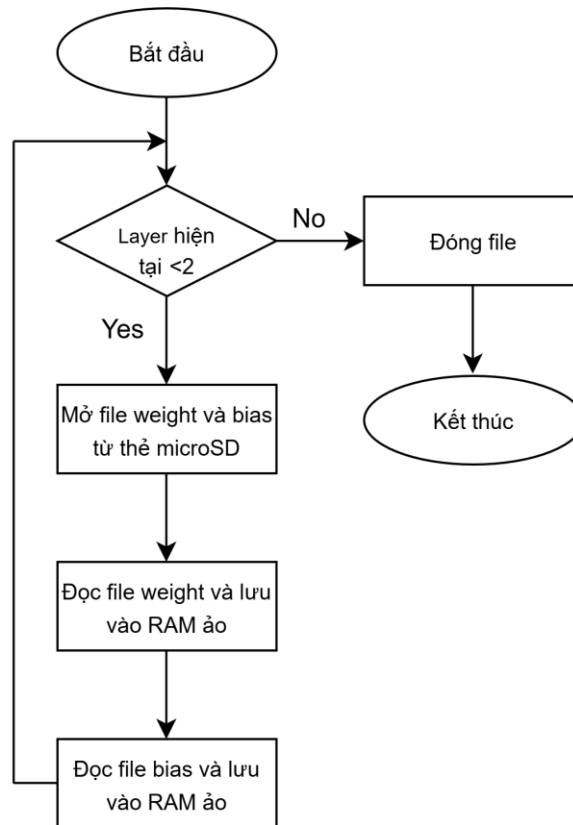
Hệ thống thực hiện mở các file trọng số trong thẻ nhớ microSD để đọc và ghi chúng đến vùng nhớ trung gian (RAM DDR3). Với việc phân chia vùng nhớ như hình 4.2 có thể giúp khối IP truy cập trực tiếp RAM DDR3 để đọc dữ

liệu trọng số. Vì bước khởi tạo trọng số (mục 5.1.1) đã chuyển đổi các file weight và bias của lớp tích chập theo số nguyên 16 bit và xếp thành 2 cột nên việc đọc ở đây cũng phải đọc dựa trên 2 cột; sau đó cột sau sẽ được đẩy lên 16 bit cao, gộp với 16 bit thấp ở cột trước để tạo thành số 32 bit và gửi đến vùng nhớ. Cơ chế này được áp dụng tương tự với các file bias.



Hình 4.10 Lưu đồ chương trình khởi tạo trọng số cho quá trình tích chập

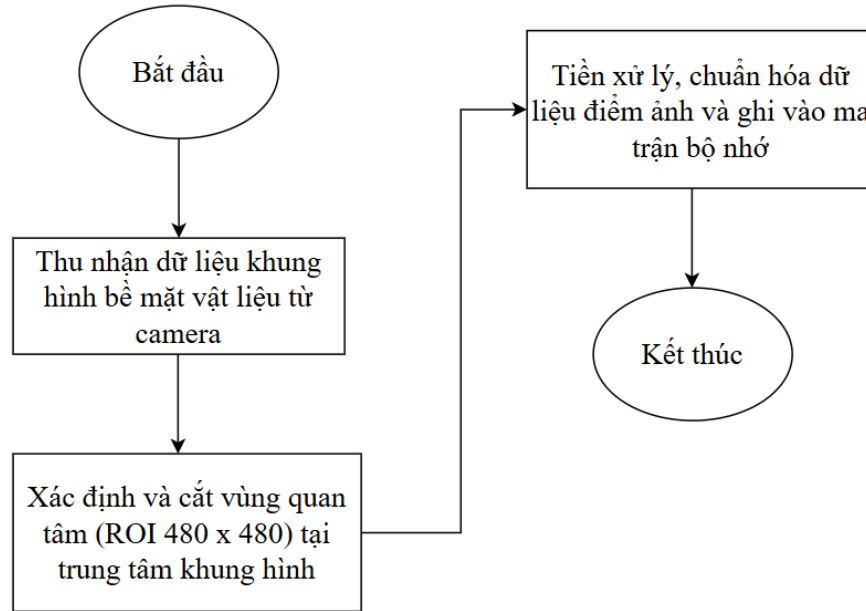
- Khởi tạo trọng số cho quá trình fully-connected



Hình 4.11 Lưu đồ chương trình khởi tạo trọng số cho quá trình fully-connected

Cũng như quá trình tích chập, ở quá trình này tiếp tục sử dụng RAM DDR3 để lưu các trọng số trong lúc thực hiện fully-connected. Điểm khác ở đây là việc lưu dữ liệu trọng số không cần phải tuân theo vùng chỉ định như hình 4.2. Bởi hệ thống thực hiện các lớp dense trực tiếp trên vi xử lý ARM nên việc lưu trọng số ở một vùng nhớ cố định là không cần thiết. Ngoài ra, quá trình fully-connected sẽ sử dụng số chấm động 16 bit để thực hiện.

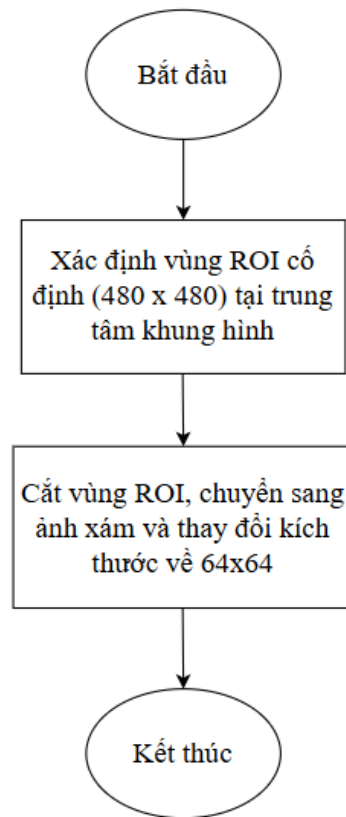
4.2.2.3 Đọc ảnh đầu vào bằng camera, xác định vùng ROI và thực hiện tiền xử lý ảnh



Hình 4.12 Lưu đồ chương trình đọc dữ liệu từ camera

Quy trình đọc và nạp dữ liệu hình ảnh được thực hiện tuần tự theo Hình 4.12 với sự hỗ trợ từ thư viện OpenCV. Đầu tiên, vi xử lý ARM cấu hình và thu nhận luồng video liên tục từ camera để quan sát bề mặt vật liệu. Khung hình sau khi thu nhận sẽ được định vị và trích xuất ngay một vùng quan tâm (ROI) có kích thước cố định (480 x 480 pixel) đặt tại trung tâm khung hình. Cuối cùng, dữ liệu điểm ảnh trong vùng này sẽ được tiến hành tiền xử lý, chuẩn hóa các dải giá trị và ghi vào ma trận bộ nhớ để hệ thống chuẩn bị cho các bước tính toán tiếp theo.

Chi tiết các thao tác xử lý trong bước khoanh vùng dữ liệu được mô tả cụ thể tại Hình 4.13. Hệ thống trực tiếp xác định tọa độ vùng ROI (480 x 480 pixel) mà không cần sử dụng các thuật toán dò tìm phức tạp. Tiếp đó, chương trình tiến hành cắt lấy vùng ROI này, chuyển đổi từ ảnh màu sang ảnh mức xám và resize chuẩn về 64 x 64 pixel trước khi đưa vào khối IP CNN phân cứng.



Hình 4.13 Lưu đồ chương trình khoanh vùng dữ liệu.

Sự kết hợp đồng bộ giữa quy trình thu nhận luồng hình ảnh từ camera theo Hình 4.12 và giải thuật khoanh vùng dữ liệu bề mặt theo Hình 4.13 mang lại hiệu quả vượt trội về mặt tốc độ và tài nguyên cho toàn hệ thống. Nhờ việc cắt thẳng vùng ROI cố định tại trung tâm và hạ kích thước chuẩn về ma trận 64 x 64 mức xám, hệ thống đã loại bỏ hoàn toàn các phép toán phức tạp không cần thiết, triệt tiêu vùng nền nhiễu và tinh giản lượng dữ liệu đầu vào cho phần cứng. Điều này giúp bộ vi xử lý ARM xử lý dữ liệu với độ trễ thấp, tăng đáng kể tốc độ khung hình (FPS) và đảm bảo cung cấp nguồn dữ liệu đầu vào chuẩn xác, kịp thời cho bộ tăng tốc phần cứng CNN hoạt động liên tục trong thời gian thực.

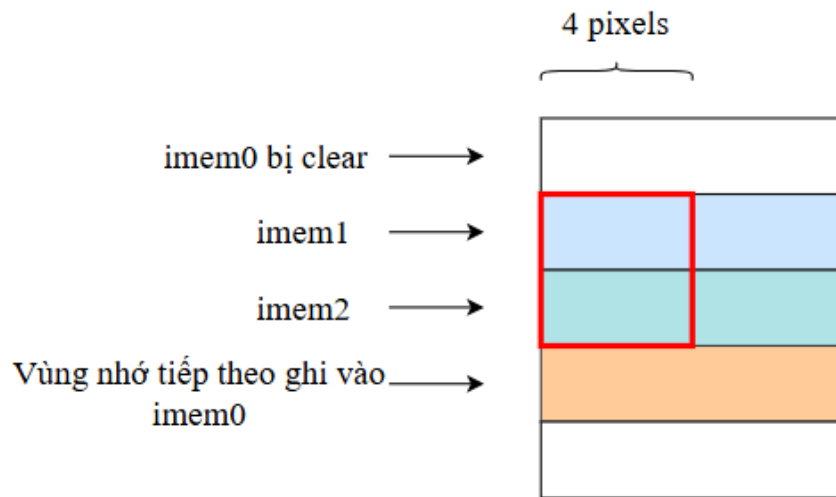
4.2.2.4 Thực hiện tiền xử lý ảnh đầu vào

Khâu tiền xử lý hình ảnh trên vi xử lý ARM có nhiệm vụ tái định dạng và sắp xếp lại dữ liệu ảnh xám 64 x 64 sau khi cắt ROI để tương thích hoàn toàn với kiến trúc nạp và cơ chế đọc dữ liệu của khối Input Buffer (IB) trong lõi IP CNN phần cứng.

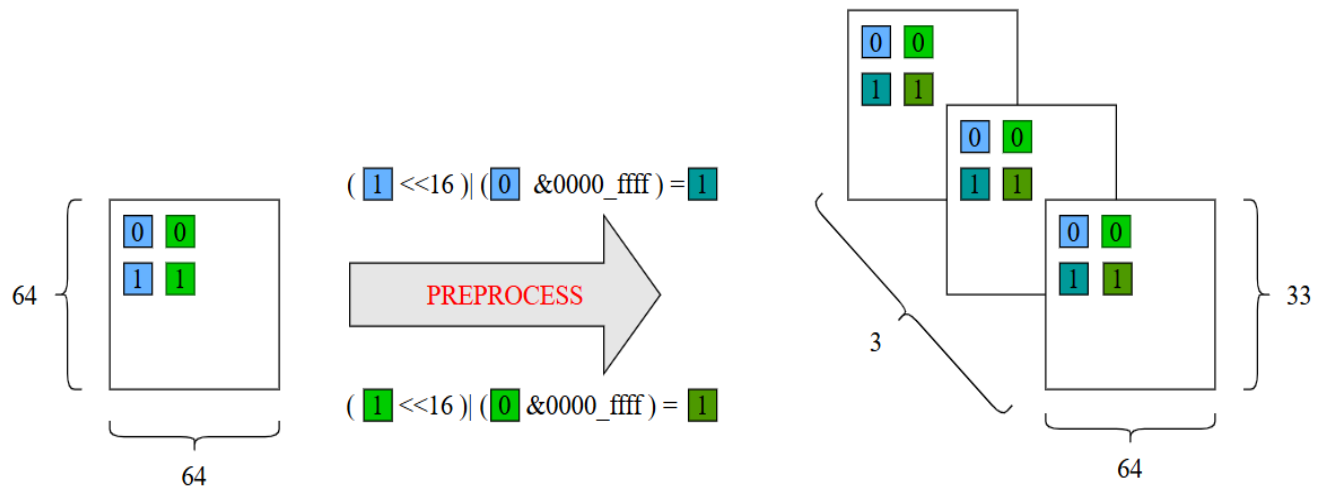
Input buffer được thiết kế để đọc 4 pixels/lần theo dạng ma trận 2x2, với mỗi pixel có độ rộng 16 bit. Do đó, hệ thống phần mềm sẽ quét 2 pixel hàng trên và 2 pixel hàng dưới của khung hình 64x64 trong cùng một khoảng thời gian, gom chúng lại thành một dãy dữ liệu có độ rộng 32 bit để ghi vào RAM DDR3 theo công thức:

$$Data\ (32 - bit) = (Pixel\ hàng\ dưới << 16) | (Pixel\ hàng\ trên \& 0x0000FFFF)$$

Bên cạnh đó, vì ảnh đầu vào là ảnh xám (chỉ có 1 kênh màu) trong khi phần cứng được thiết kế cho dữ liệu 3 kênh, hệ thống sẽ sao chép giá trị của mỗi pixel và ghi 3 lần vào vùng nhớ để giả lập thành 3 kênh tương tự như cấu trúc của ảnh RGB.



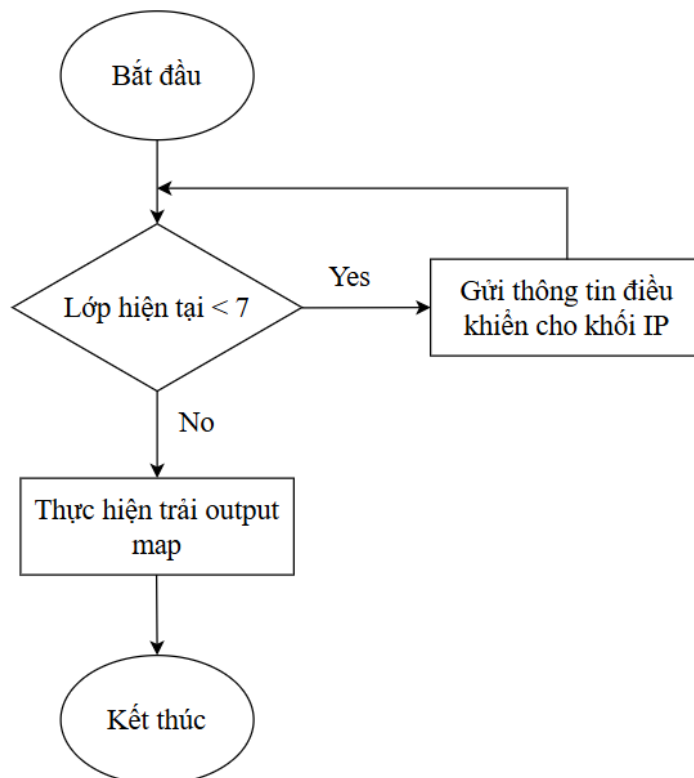
Hình 4.14 Cơ chế hoạt động đọc của Input buffer.



Hình 4.15 Giải thuật tiền xử lý ảnh đầu vào.

4.2.2.5 Thực hiện điều khiển phần cứng

Sau khi đã khởi tạo đầy đủ file trọng số và chuẩn bị dữ liệu ảnh đầu vào, nhóm chúng em sẽ điều khiển phần cứng thực hiện tính tích chập dựa theo quy trình được mô tả ở Hình 4.16.

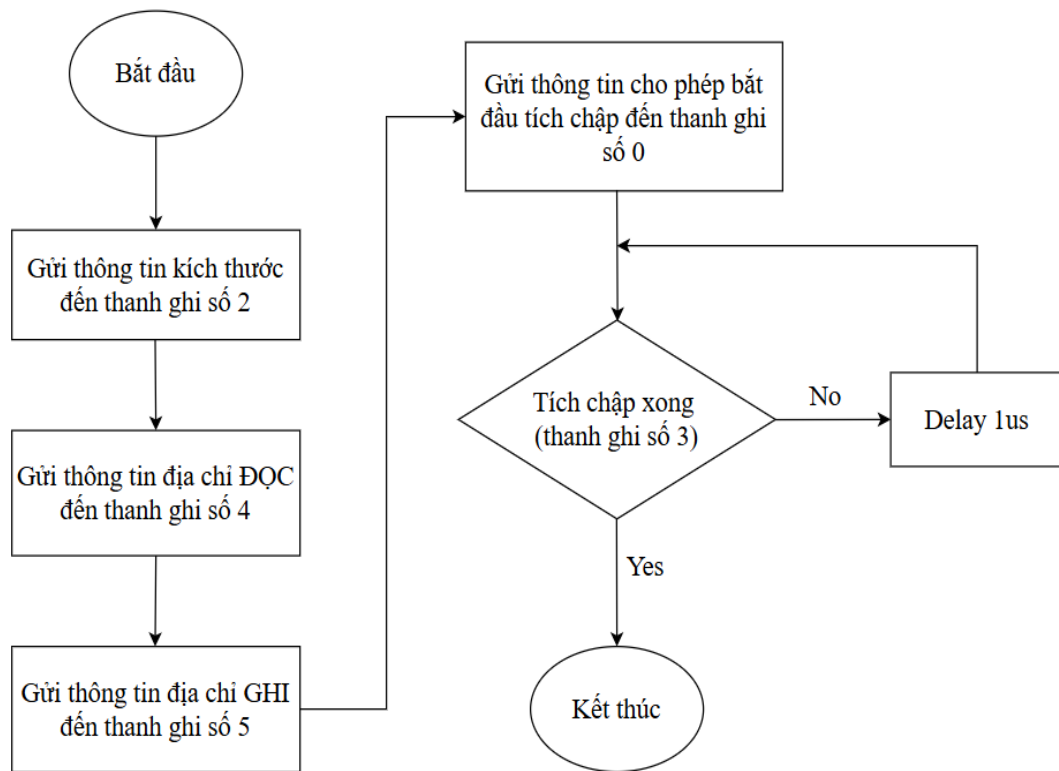


Hình 4.16 Lưu đồ chương trình điều khiển phần cứng.

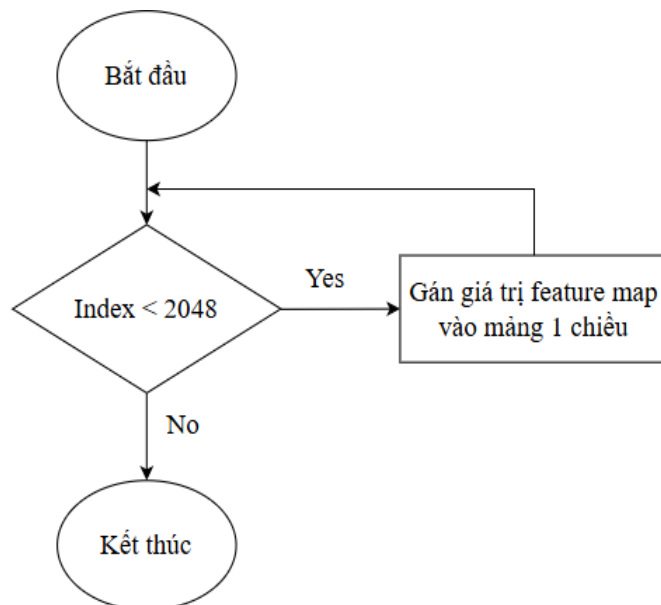
Quá trình điều khiển lõi IP CNN để trích xuất đặc trưng khiếm khuyết được thực hiện thông qua vùng nhớ ảo `h2p_low_cnn_addr` kết nối với bộ REGISTER ở phần cứng. Vi xử lý ARM sẽ nạp các thông tin cấu hình mạng vào các thanh ghi tương ứng:

- Thanh ghi số 0: Truyền tín hiệu kích hoạt quá trình tính toán tích chập trên khối IP.
- Thanh ghi số 2: Gửi thông tin về kích thước input feature map, kích thước kernel và số kênh.
- Thanh ghi số 3: là tín hiệu dùng để báo thực hiện xong một lớp tích chập để vi xử lý ARM tiếp tục thực hiện lớp sau
- Thanh ghi số 4 và 5: Gửi địa chỉ đọc (rbase) và địa chỉ ghi (wbase) dữ liệu từ RAM DDR3.

Hệ thống sẽ chờ tín hiệu phản hồi từ thanh ghi số 3 để xác nhận lõi IP đã hoàn thành việc tính toán. Sau khi lớp tích chập cuối cùng hoàn tất, vi xử lý ARM sẽ truy cập vào vùng nhớ RAM DDR3 chứa output feature map và thực hiện hàm trải phẳng (flatten) ma trận này thành một mảng một chiều (vector) theo mô tả Hình 4.17 và Hình 4.18 bên dưới.



Hình 4.17 Lưu đồ chương trình gửi thông tin điều khiển.

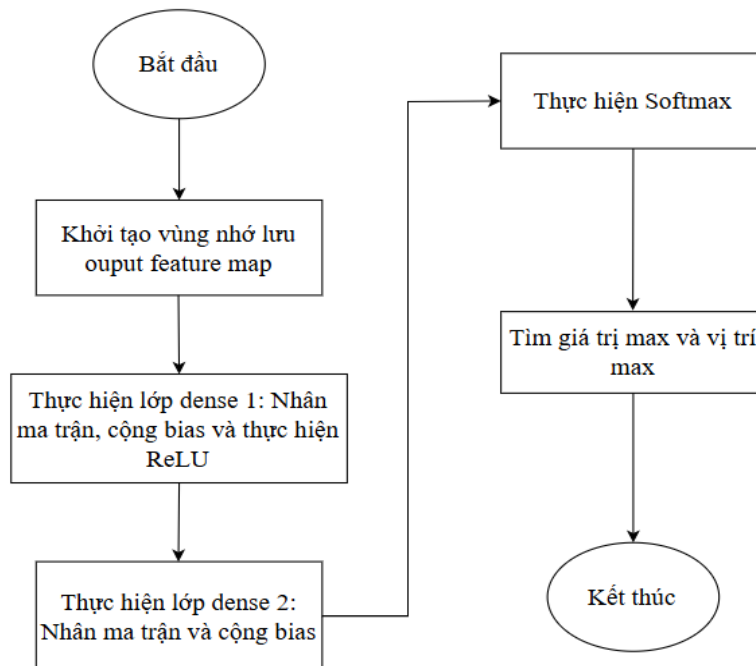


Hình 4.18 Lưu đồ chương trình trả output feature map.

Các đặc trưng của khiếm khuyết sẽ do hàm `vgg_convolution` chịu trách nhiệm điều phối toàn bộ quá trình trích xuất đặc trưng trên phần cứng thông qua địa chỉ ảo `h2p_low_cnn_addr` trở trực tiếp đến khối REGISTER của lõi IP CNN. Tại đây, vi xử lý ARM (HPS) lần lượt cấu hình các tham số vận hành cần thiết bao gồm: thông số kích thước input feature map, số lượng kênh, địa chỉ bộ nhớ đọc/ghi (`rbase`, `wbase`) trên SDRAM DDR3 và phát xung kích hoạt (start signal) để khối tăng tốc bắt đầu tính tích chập. Khi lõi IP xử lý xong lớp tích chập cuối cùng, phần mềm sẽ thực hiện hàm trải phẳng (flatten) ma trận đặc trưng ngõ ra thành một vector một chiều nhằm cung cấp dữ liệu đầu vào cho các lớp fully-connected tiếp theo đúng như mô tả ở trên.

4.2.2.6 Thực hiện lớp dense và trả kết quả phân loại khiếm khuyết

Sau khi lõi IP CNN hoàn thành 7 lớp tích chập và trích xuất thành công các đặc trưng không gian của bề mặt vật liệu, quá trình phân loại chi tiết sẽ được chuyển tiếp cho vi xử lý ARM (HPS) đảm nhiệm thông qua module `dense.cpp`. Toàn bộ quy trình tính toán được thực thi tuần tự theo lưu đồ tại Hình 4.19.



Hình 4.19 Lưu đồ thực hiện chương trình lớp dense và trả kết quả phân loại khiếm khuyết.

Dựa theo mô tả, trước khi tính toán, hệ thống tiến hành làm sạch và khởi tạo lại các mảng bộ nhớ tạm để sẵn sàng lưu trữ kết quả trung gian sau mỗi lớp dense.

Thực thi lớp dense đầu tiên: Dữ liệu đầu vào là mảng vector đặc trưng một chiều sau bước trải phẳng. Vi xử lý ARM thực hiện phép nhân ma trận giữa mảng đặc trưng này với tập trọng số của lớp dense đầu tiên đã được nạp sẵn từ thẻ nhớ ở mục 4.2.2.2. Kết quả của phép nhân sau đó được cộng với giá trị bias tương ứng và lọc qua hàm kích hoạt phi tuyến ReLU để loại bỏ các thành phần tín hiệu âm, tạo ra mảng đặc trưng ẩn trung gian.

Thực thi lớp dense thứ hai: Mảng đặc trưng từ lớp thứ nhất tiếp tục được nhân với ma trận trọng số và cộng bias của lớp dense thứ hai. Số lượng phần tử đầu ra ở lớp này tương ứng đúng với số lượng nhóm phân loại bề mặt được định nghĩa trước (bao gồm bề mặt chuẩn và các dạng khiếm khuyết cụ thể).

Chuẩn hóa xác suất và kết luận phân loại: Dữ liệu ngõ ra của lớp dense cuối cùng được đưa qua hàm Softmax để chuyển đổi thành phân bố xác suất cho từng lớp đối tượng. Chương trình sẽ quét toàn bộ mảng kết quả để tìm ra giá trị xác suất cao nhất cùng vị trí chỉ số (index) tương ứng. Vị trí này chính là nhãn phân loại của bề mặt vật liệu, đồng thời giá trị xác suất cao nhất sẽ đại diện cho độ tin cậy của dự đoán, sẵn sàng để xuất ra màn hình theo dõi và điều khiển cơ cấu phân loại.

5. THỰC HIỆN HỆ THỐNG

Trong phần này, chúng em sẽ thực hiện một hệ thống nhận dạng và phát hiện được khiếm khuyết của bề mặt vật liệu (Dựa trên tập dữ liệu Synthetic Industrial Metal Surface Defects) được triển khai trên thiết bị SoC FPGA. Mục tiêu của nhóm là tạo ra hệ thống có thể tự động phát hiện nơi có khiếm khuyết và thực hiện phân loại các loại chúng

Dựa trên lõi IP đã thiết kế trên FPGA được sử dụng để tính toán các lớp mạng thần kinh tích chập của tác giả Phạm Thế Vinh. Chúng em sẽ xây dựng một mạng CNN tối ưu với phần cứng được tùy chỉnh lại để cho phù hợp với mạng CNN này. Sau đó, thực hiện nhận dạng và phân loại với kết quả cho ra . Hệ thống phần cứng được thiết kế trên SoC FPGA nhóm thực hiện có thể xử lý XX khung hình mỗi giây với tần số hoạt động XX MHz.

Nhiệm vụ nhận dạng và phân loại khiếm khuyết thành hai giai đoạn. Giai đoạn đầu tiên là phát hiện nơi xảy ra khiếm khuyết, trong đó nhóm sẽ xây dựng mô-đun nhận diện bằng cách chạy trên CPU để cắt vùng 64x64 pixel rồi thực hiện quét tuần tự các vùng 64x64 pixel cho đến hết kích thước ảnh đầu vào ở thang xám. Giai đoạn thứ hai là đưa ảnh thang xám này vào 3 kênh (nimap = 3) rồi thực hiện tích chập và đưa ra kết quả dự đoán bằng mô hình CNN đã xây dựng.

Để có được trọng số cho quá trình phát hiện khiếm khuyết cần trải qua hai bước là huấn luyện mô hình Deep Learning và xử lý trọng số sau khi huấn luyện. Với IP tích chập đã cấu hình ở phần cứng, để hệ thống có thể phát hiện khiếm khuyết chính xác mà lại có thể đáp ứng thời gian thực thì mạng VGG đề nghị nên có 7 lớp tích chập (convolution) và 2 lớp fully connected; các thông số của bảng được mô tả như sau:

Bảng 5.1 Thông số của mạng nơron tích chập đề nghị.

Conv layer	Input size	Output size	Nimap	Nomap	Max pooling
CONV1	64x64	64x64	3	32	0
CONV2	64x64	32x32	32	32	1
CONV3	32x32	32x32	32	32	0
CONV4	32x32	16x16	32	32	1
CONV5	16x16	16x16	32	64	0
CONV6	16x16	8x8	64	64	1
CONV7	8x8	4x4	64	128	1
Dense layer		Input size		Output size	
DENSE1		2048		128	
DENSE2		128		5	

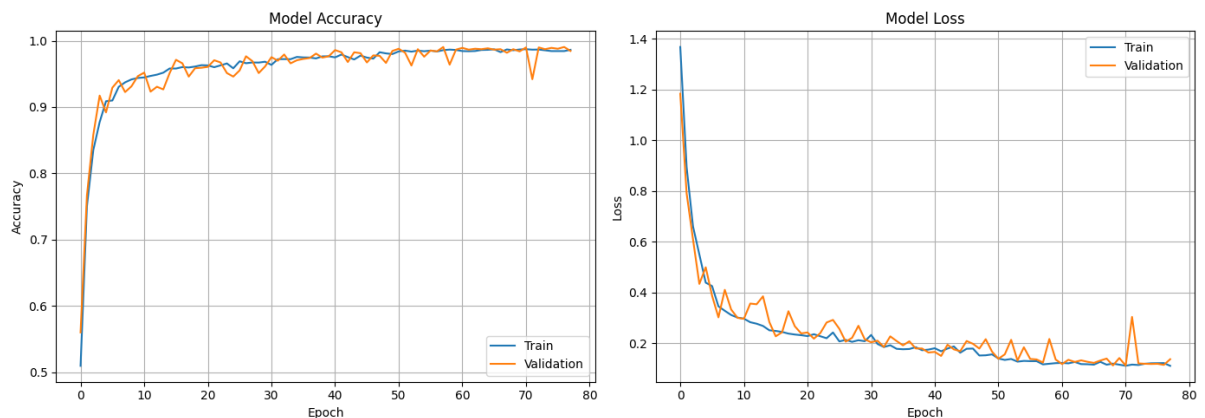
5.1 Huấn luyện mô hình Deep Learning trên Google Colab và chuyển đổi trọng số

5.1.1 Thực hiện huấn luyện mô hình

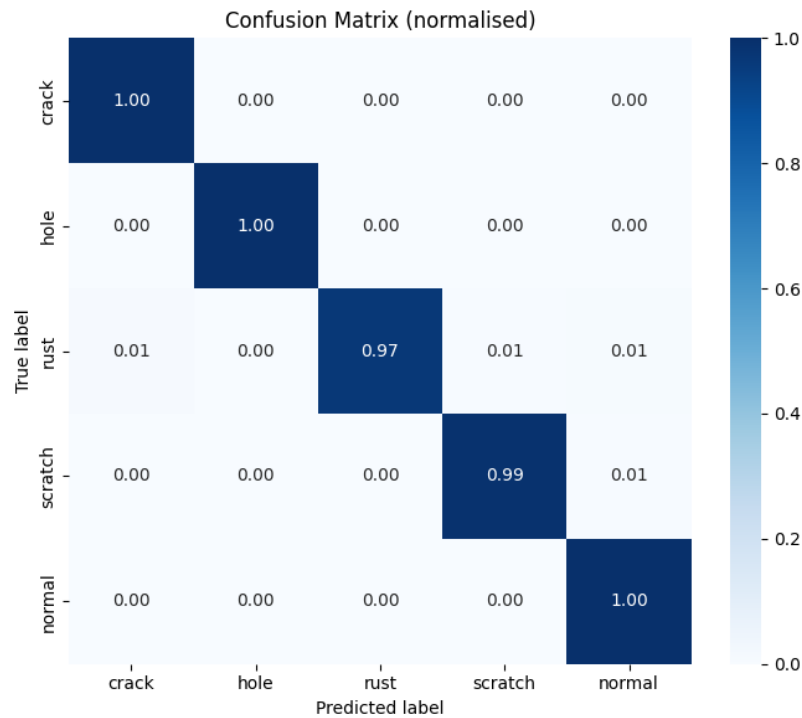
Dựa vào cấu trúc của mô hình mạng tích chập trên, nhóm sẽ xây dựng trọng số bằng cách huấn luyện mô hình dựa trên dữ liệu mà nhóm tự thu thập ở thực tế. Cụ thể, nhóm sẽ phân biệt 4 loại khiếm khuyết thường thấy là: vết nứt, rỉ sét, vết xước, lỗ hỏng. Và nếu không thuộc 4 loại trên, hệ thống sẽ đánh giá bức ảnh này là bình thường, nên tổng cộng sẽ có 5 class.

Chia tỷ lệ train/valid/test là 80/10/10 cho 15.000 ảnh RGB có kích thước tỷ lệ 1:1 được huấn luyện trên Google Colab sử dụng framework Keras. Rồi thực hiện chuyển toàn bộ RGB về ảnh xám và thực hiện training theo thông số của bảng 5.1

Mô hình được huấn luyện trong 78 epoch với độ chính xác cao nhất ở epoch 58 trên tập validation là 99,07% và trên tập train là 98,59%.



Hình 5.1 Đường cong học tập được thực hiện.



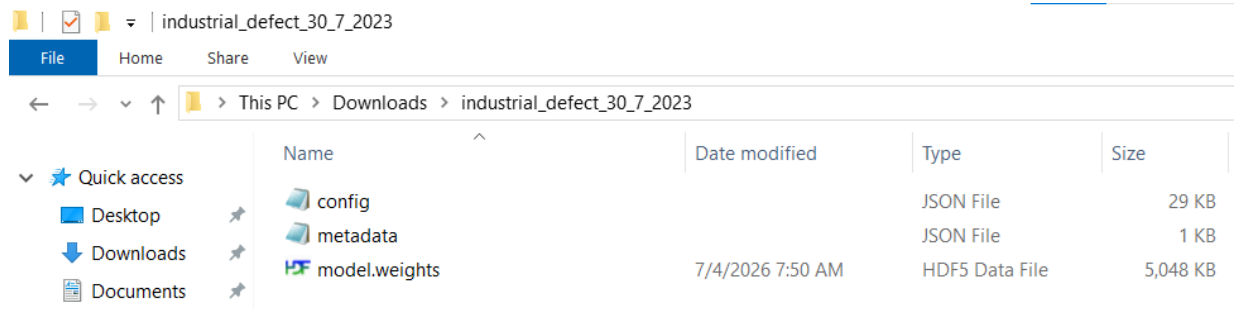
Hình 5.2 Ma trận nhầm lẫn.

Sau đó, chúng em thực hiện tính toán các thông số như precision, recall, f1-score với số lượng ảnh test là 300 cho mỗi lớp để quan sát độ chính xác của từng lớp:

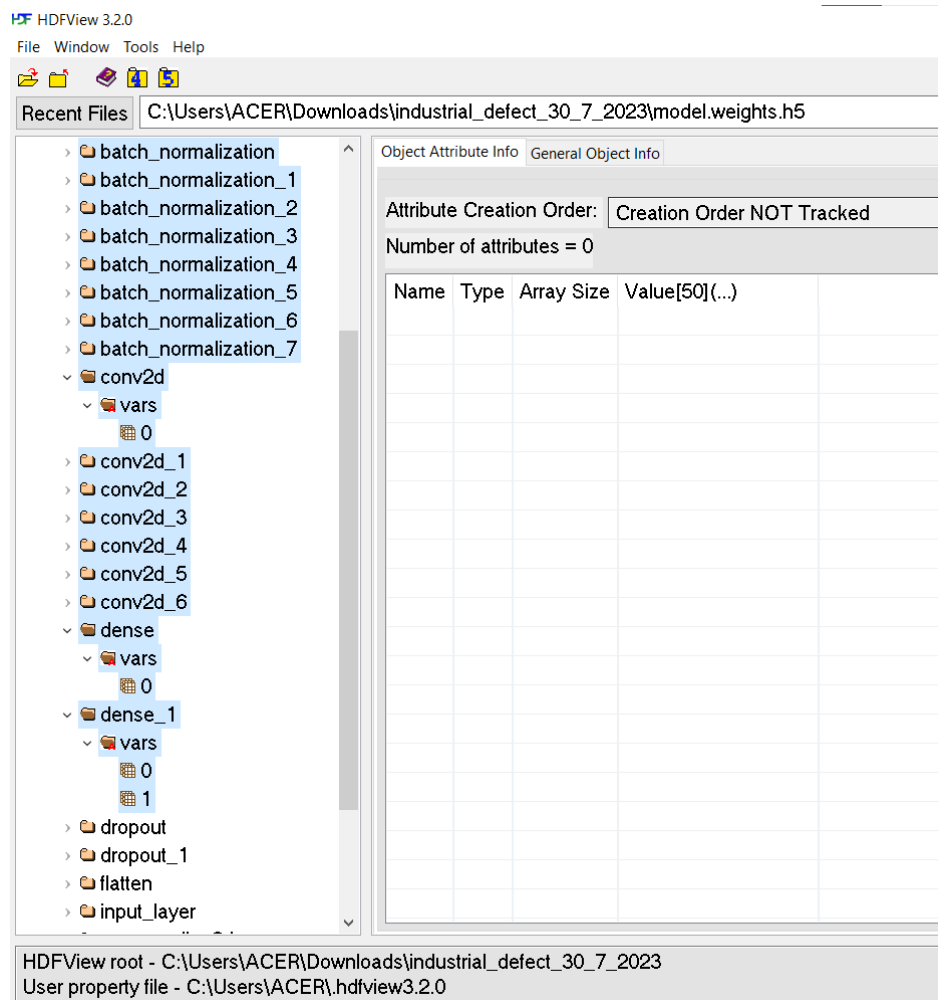
Bảng 5.2 Các chỉ số đánh giá mô hình

	Precision	Recall	F1-Score	Support
crack	0.9836	1.0000	0.9917	300
hole	0.9967	0.9967	0.9967	300
rust	1.0000	0.9667	0.9831	300
scratch	0.9933	0.9900	0.9917	300
normal	0.9804	1.0000	0.9901	300

5.1.2 Chuyển đổi trọng số thành file .txt

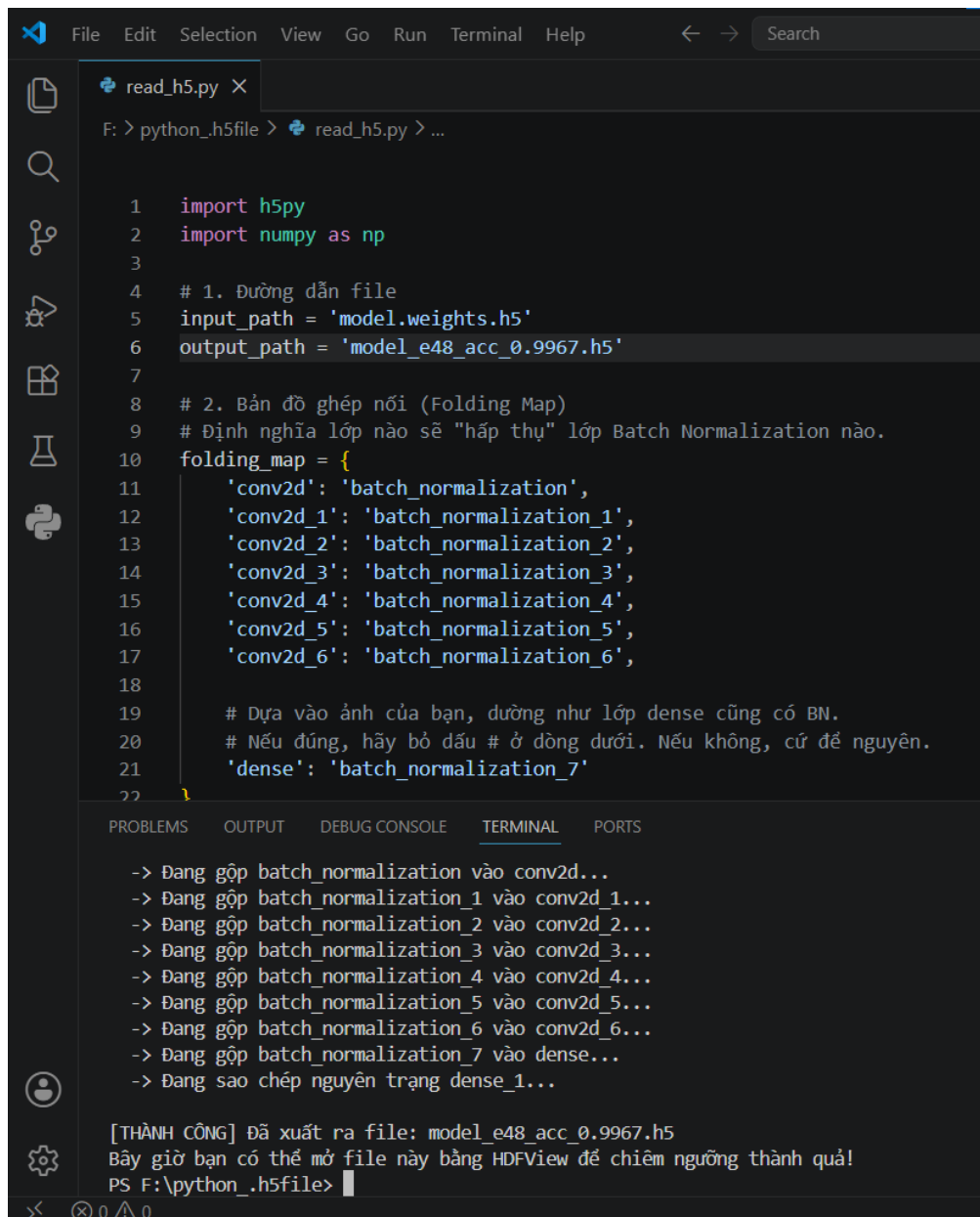


Hình 5.3 File .hdf5 sau khi train



Hình 5.4 Bên trong file model.weights.hdf5 khi chưa gộp batch

Vì code training sử dụng batch normalization nên bias ở các lớp conv và lớp dense đầu tiên bị loại bỏ, vì vậy chúng ta phải chuyển đổi tính gộp file trọng số để tạo ra bias và weights từ 8 lớp batch normalize. Chúng em thực hiện file `read_h5.py` như hình 4.4 để tạo ra file trọng số hoàn chỉnh `model_e48_acc_0.9967.h5` từ file trọng số gốc `model.weights.hdf5`



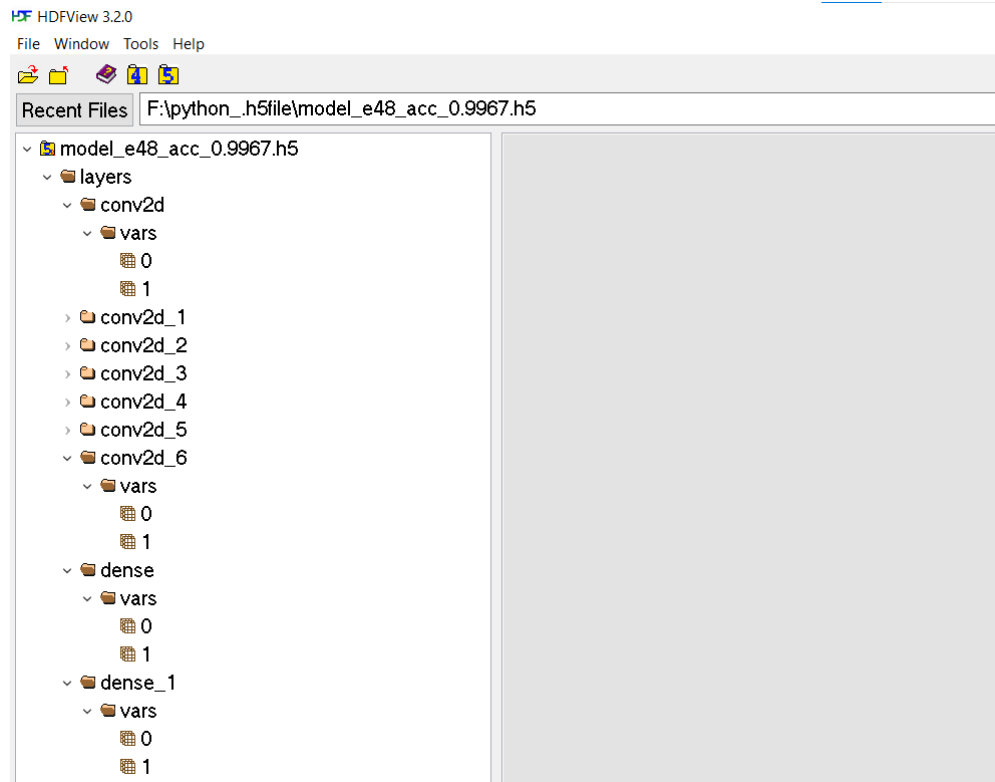
```
File Edit Selection View Go Run Terminal Help
read_h5.py X
F: > python_h5file > read_h5.py > ...

1 import h5py
2 import numpy as np
3
4 # 1. Đường dẫn file
5 input_path = 'model.weights.h5'
6 output_path = 'model_e48_acc_0.9967.h5'
7
8 # 2. Bản đồ ghép nối (Folding Map)
9 # Định nghĩa lớp nào sẽ "hấp thụ" lớp Batch Normalization nào.
10 folding_map = {
11     'conv2d': 'batch_normalization',
12     'conv2d_1': 'batch_normalization_1',
13     'conv2d_2': 'batch_normalization_2',
14     'conv2d_3': 'batch_normalization_3',
15     'conv2d_4': 'batch_normalization_4',
16     'conv2d_5': 'batch_normalization_5',
17     'conv2d_6': 'batch_normalization_6',
18
19     # Dựa vào ảnh của bạn, đường như lớp dense cũng có BN.
20     # Nếu đúng, hãy bỏ dấu # ở dòng dưới. Nếu không, cứ để nguyên.
21     'dense': 'batch_normalization_7'
22 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
-> Đang gộp batch_normalization vào conv2d...
-> Đang gộp batch_normalization_1 vào conv2d_1...
-> Đang gộp batch_normalization_2 vào conv2d_2...
-> Đang gộp batch_normalization_3 vào conv2d_3...
-> Đang gộp batch_normalization_4 vào conv2d_4...
-> Đang gộp batch_normalization_5 vào conv2d_5...
-> Đang gộp batch_normalization_6 vào conv2d_6...
-> Đang gộp batch_normalization_7 vào dense...
-> Đang sao chép nguyên trạng dense_1...

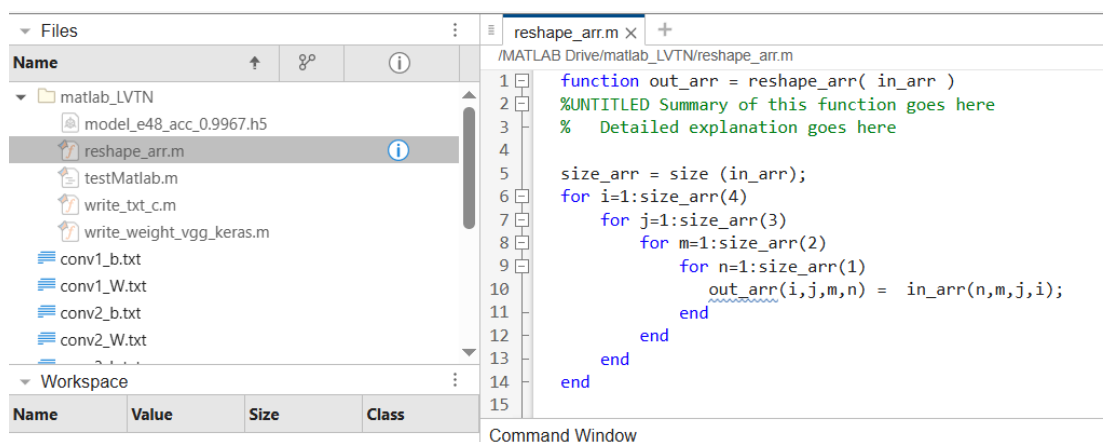
[THÀNH CÔNG] Đã xuất ra file: model_e48_acc_0.9967.h5
Bây giờ bạn có thể mở file này bằng HDFView để chiêm ngưỡng thành quả!
PS F:\python_h5file>
```

Hình 5.5 Chạy file `read_h5.py`



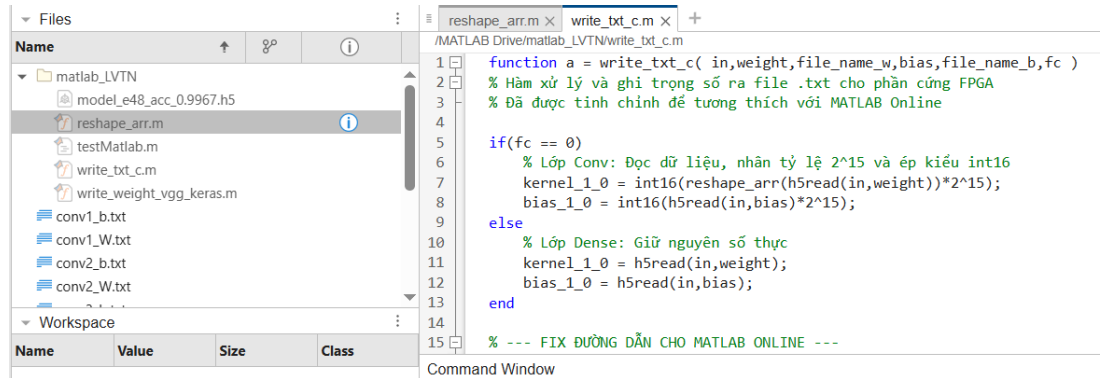
Hình 5.6 Bên trong file model_e48_acc_0.9967.h5 sau khi gộp batch

Chúng em sử dụng MATLAB online với các hàm được cấu hình để đọc và ghi các giá trị của tệp .hdf5, sử dụng các file reshape_arr, write_weight_vgg_keras, write_txt_c và testMatlab để chuyển đổi tệp trọng số thô (model.weights.hdf5) sau quá trình huấn luyện thành các tệp trọng số ở định dạng văn bản (.txt) để hệ thống dễ dàng đọc và lưu trữ hơn



Hình 5.7 File reshape_arr.m

File `reshape_arr.m` dùng để sắp xếp lại thứ tự của dữ liệu Hoán vị và định hình lại kích thước hình học của các khối ma trận trọng số đa chiều nhằm tương thích với logic quét bộ nhớ của phần cứng.



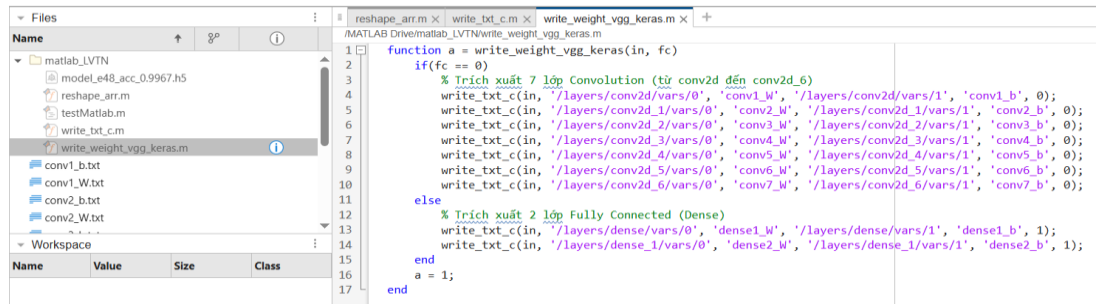
Hình 5.8 File `write_txt_c.m`

File `write_txt_c.m` là hàm xử lý toán học cốt lõi, thực hiện nhiệm vụ lượng tử hóa số thực float 32 bit sang số nguyên fixed-point 16 bit và đóng gói dữ liệu theo kiến trúc Bus phần cứng, hàm thực hiện 2 chức năng sau:

Xử lý và đóng gói cho Tầng Tích chập (Conv): Lượng tử hóa Fixed-Point để tối ưu tài nguyên logic và tốc độ xung nhịp trên FPGA, toàn bộ trọng số số thực (Float32) của các tầng tích chập được nhân với hệ số tỷ lệ 2^{15} trước khi ép kiểu về số nguyên có dấu 16-bit (int16). Điều này giúp chuyển đổi các phép tính toán số thực phức tạp thành các phép toán số nguyên tốc độ cao trên FPGA mà không làm suy giảm độ chính xác của mô hình nhận dạng lỗi. Đối với lớp tích chập, dữ liệu được gom cụm theo từng nhóm số (khi đọc được 2 số xếp theo hàng ngang sẽ xuống dòng qua lệnh `mod(t,2)`) mỗi dòng chứa 2 trọng số tức 32 bit, cấu trúc này được thiết kế để khớp chính xác với băng thông của Bus dữ liệu 256-bit kết nối với hệ thống RAM lưu trữ trọng số. Mỗi lần truyền trọng số lớp Conv trên bus 256 bit sẽ gửi đến cho 16/32 PU parallel.

Trích xuất tuyến tính cho Tầng Phân loại (Dense): Giữ nguyên định dạng Float 32-bit. Ngược lại với FPGA, dữ liệu của các lớp Dense và hàm Softmax cuối cùng được thực thi trên phần mềm của vi xử lý (CPU). Do đó, hàm không thực hiện lượng tử hóa mà giữ nguyên giá trị số thực nguyên bản. Đối với lớp này, hệ thống không thực hiện bất kỳ phép hoán vị hình học nào. Dữ liệu Trọng số (Weight) và Độ lệch (Bias) được quét qua vòng lặp tuần tự và ghi trực tiếp mỗi giá trị trên một dòng đơn lẻ. Cấu trúc dạng mảng cột cơ bản này cho phép

mã C/C++ trên CPU dễ dàng cấp phát bộ nhớ và nạp dữ liệu một cách trực tiếp vào các ma trận tiêu chuẩn.

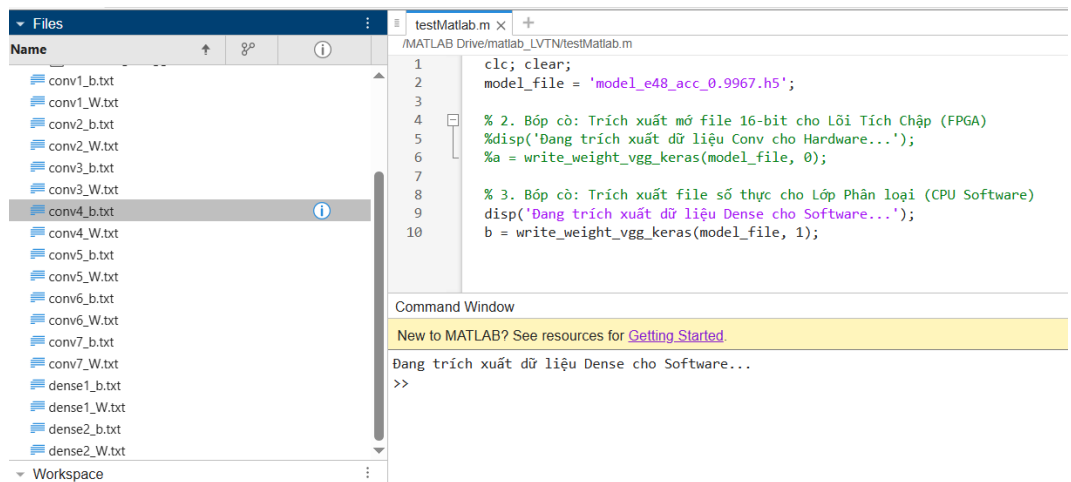


Hình 5.9 File write_weight_vgg_keras.m

File write_weight_vgg_keras chịu trách nhiệm ánh xạ chính xác cấu trúc lưu trữ của thư viện Keras đời mới sang sơ đồ quản lý tầng của cấu trúc phần cứng chuyên biệt. Thay vì xử lý thủ công, hàm tự động duyệt qua toàn bộ mô hình:

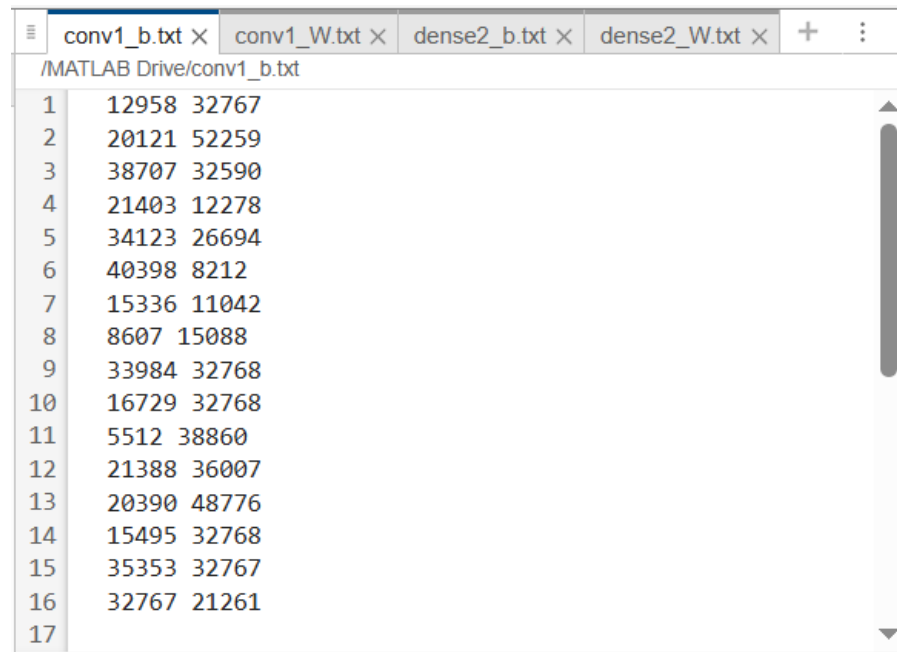
Khi $fc = 0$, hàm định vị chính xác tọa độ các tập dữ liệu trọng số và độ lệch từ lớp conv2d đến conv2d_6, sau đó gán các tên file đích chuẩn hóa từ conv1_W/conv1_b cho đến conv7_W/conv7_b.

Khi $fc = 1$, hàm chuyển dịch sang phân vùng các lớp phân loại cuối cùng (dense và dense_1), ánh xạ về tên file dense1_W và dense2_b. Sự chuẩn hóa này giúp đồng bộ hóa tên gọi tệp tin đầu ra khớp hoàn toàn với cấu trúc phần cứng.



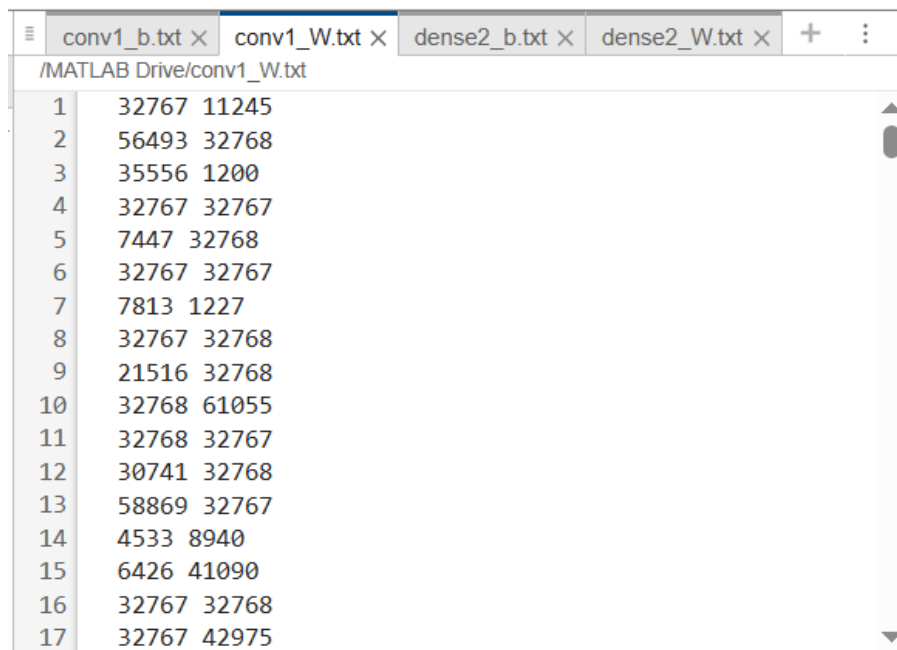
Hình 5.10 File testMatlab.m

File testMatlab.m là Script chính được kỹ sư sử dụng để kích hoạt toàn bộ quy trình trích xuất và chuyển đổi dữ liệu mô hình AI.



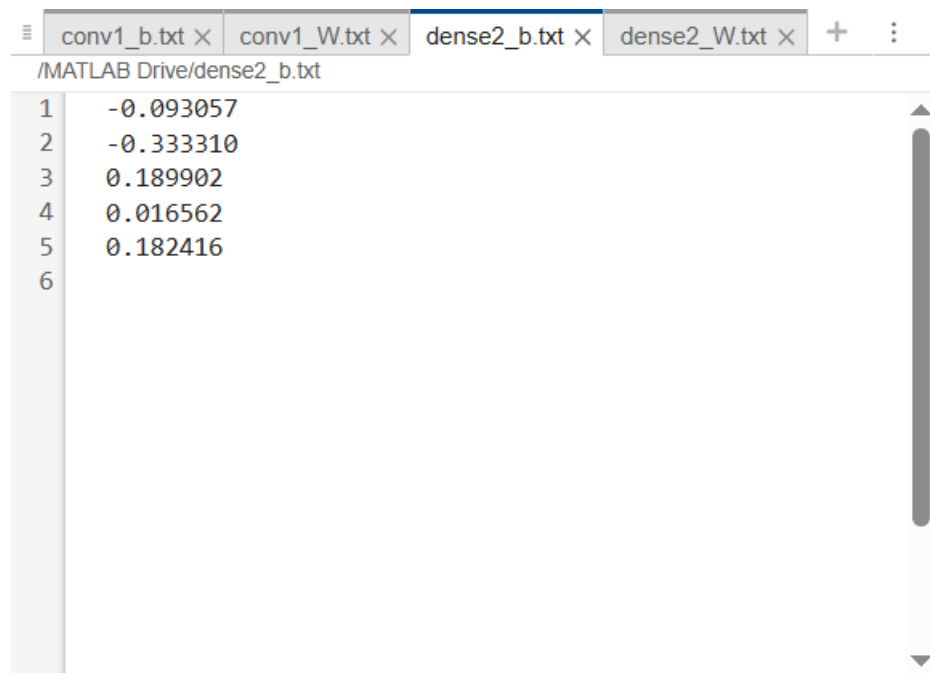
Line	Value 1	Value 2
1	12958	32767
2	20121	52259
3	38707	32590
4	21403	12278
5	34123	26694
6	40398	8212
7	15336	11042
8	8607	15088
9	33984	32768
10	16729	32768
11	5512	38860
12	21388	36007
13	20390	48776
14	15495	32768
15	35353	32767
16	32767	21261
17		

Hình 5.11 Bias của lớp convolution 1 trong định dạng .txt



Line	Value 1	Value 2
1	32767	11245
2	56493	32768
3	35556	1200
4	32767	32767
5	7447	32768
6	32767	32767
7	7813	1227
8	32767	32768
9	21516	32768
10	32768	61055
11	32768	32767
12	30741	32768
13	58869	32767
14	4533	8940
15	6426	41090
16	32767	32768
17	32767	42975

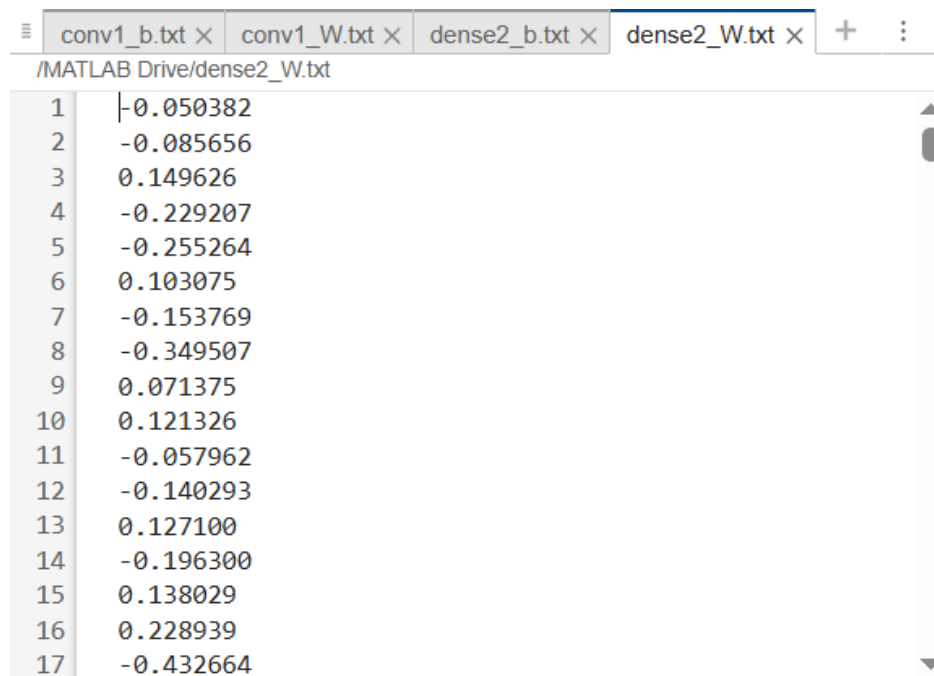
Hình 5.12 Trọng số của lớp convolution 1 trong định dạng .txt



The screenshot shows a text editor with four tabs: conv1_b.txt, conv1_W.txt, dense2_b.txt, and dense2_W.txt. The active tab is dense2_b.txt, which displays the following data:

```
/MATLAB Drive/dense2_b.txt
1 -0.093057
2 -0.333310
3 0.189902
4 0.016562
5 0.182416
6
```

Hình 5.13 Bias của lớp dense 2 trong định dạng .txt



The screenshot shows a text editor with four tabs: conv1_b.txt, conv1_W.txt, dense2_b.txt, and dense2_W.txt. The active tab is dense2_W.txt, which displays the following data:

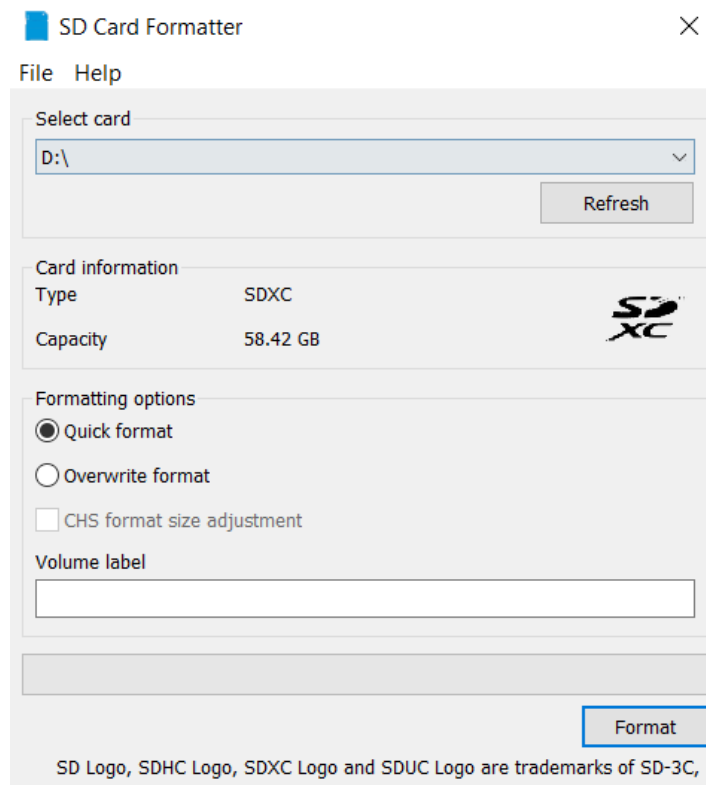
```
/MATLAB Drive/dense2_W.txt
1 -0.050382
2 -0.085656
3 0.149626
4 -0.229207
5 -0.255264
6 0.103075
7 -0.153769
8 -0.349507
9 0.071375
10 0.121326
11 -0.057962
12 -0.140293
13 0.127100
14 -0.196300
15 0.138029
16 0.228939
17 -0.432664
```

Hình 5.14 Trọng số của lớp dense 2 trong định dạng .txt

5.1.3 Khởi tạo môi trường Linux trên board DE10

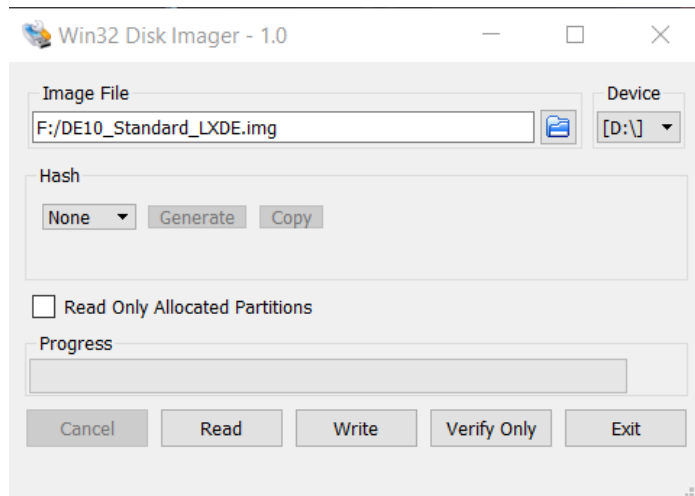
Việc tạo môi trường trên board là vô cùng cần thiết để hiện thực một dự án SoC lên board FPGA vì khi có môi trường thì việc giao tiếp giữa người dùng với board sẽ dễ dàng hơn. Một lưu ý nhỏ rằng bước này nên thực hiện trước khi nạp code phần cứng lên board, trước khi bật nguồn kết nối VGA với board và kết nối Putty.

Bước đầu tiên để tạo môi trường cho board, cần chuẩn bị một thẻ nhớ micro SD tối thiểu 4 GB và format bằng phần mềm *SD Card Formatter*.



Hình 5.15 Phần mềm SD Card Formatter

Tiếp theo là burn hệ điều hành vào thẻ micro SD trên bằng phần mềm *Win32 Disk Imager*. Ở đây, chọn file với tên là *DE10_Standard_LXDE.img* như Hình 5.16; Với *DE10_Standard_LXDE.img* được tải từ trang của nhà sản xuất Terasic [4]. Sau khi burn hệ điều hành, ta gắn thẻ nhớ micro SD vào board và bật nguồn.



Hình 5.16 Phần mềm Win32 Disk Imager

Sau khi nhấn Write, quá trình hoàn tất sẽ phân microSD ra làm 3 ổ đĩa:

+ Phân vùng 1 (FAT32 - Boot): Đây là phân vùng chứa file khởi động. Windows đọc được định dạng này.

+ Phân vùng 2 (Ext4 - RootFS): Đây là phân vùng chứa hệ điều hành Linux (nặng nhất, chiếm nhiều GB). Windows không đọc được định dạng Linux (Ext4).

+ Phân vùng 3 (Preloader): Phân vùng thô chứa mã nạp khởi động cấp thấp. Windows cũng không đọc được.

Lưu ý: Windows có thể sẽ "nhảy" lên vài cửa sổ thông báo lỗi dạng *"You need to format the disk..."*. Hãy nhớ bấm Cancel tất cả các bảng đó, tuyệt đối không Format.

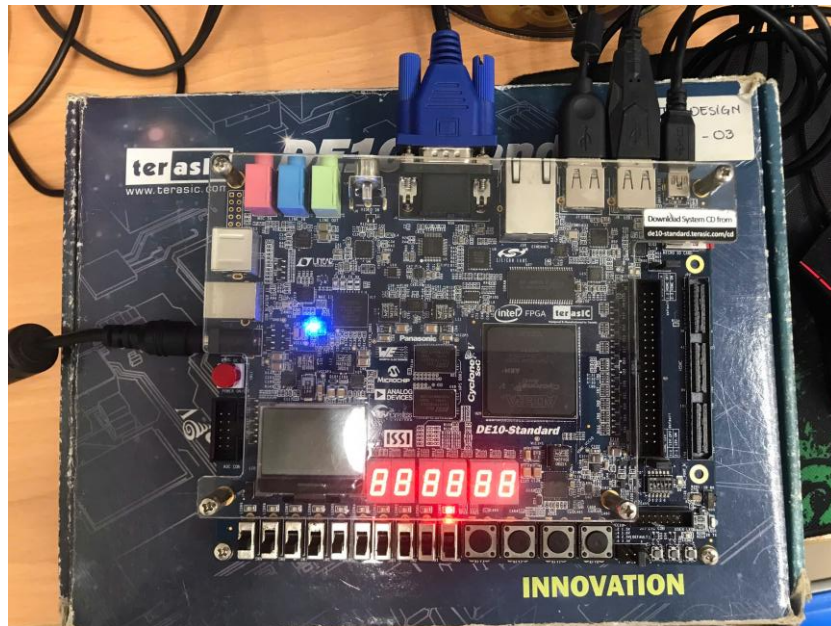
Trong phân vùng 1, ta thấy xuất hiện các file như sau:

Name	Date modified	Type	Size
soc_system.dtb	6/6/2017 7:22 AM	DTB File	26 KB
soc_system.rbf	6/6/2017 7:22 AM	RBF File	2,856 KB
zImage	4/27/2017 7:24 AM	File	5,051 KB
u-boot	1/17/2017 5:39 PM	Screen saver	1 KB

Hình 5.17 Kết quả sau khi burn hệ điều hành

5.1.4 Thiết lập kết nối từ laptop đến board DE10

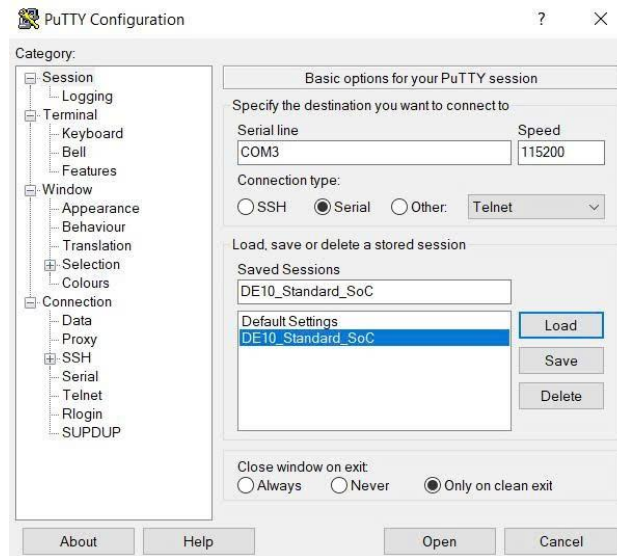
Khi đã có môi trường Linux trên board, ta cần kết nối board với laptop thông qua cổng *UART to USB* để thực hiện một số các phép điều khiển; Gắn con chuột và camera vào cổng *USB1* và *USB2*; Kết nối cổng VGA máy tính vào *VGA 24-bit DAC*.



Hình 5.18 Thiết lập kết nối giữa laptop, máy tính và board

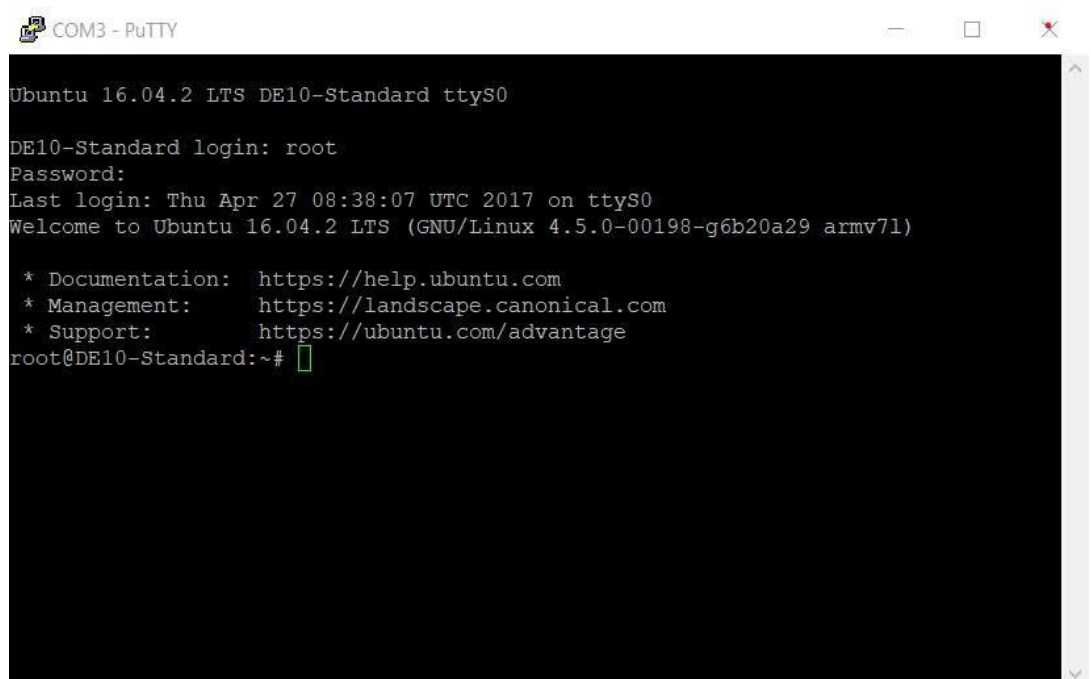
Với môi trường Linux, ta có thể tạo các folder và lưu trữ các file thực thi theo:

- + Kết nối kit với laptop thông qua cổng *UART to USB* như Hình 5.18
- + Bật phần mềm PuTTY và cấu hình cho serial 115200 baudrate, no parity, 1 stop bit.



Hình 5.19 Kết nối laptop với board thông qua PuTTY

Sau khi bấm Open, một cửa sổ màu đen sẽ hiện ra. Nhấn phím ENTER 1 lần để hiện dòng chữ “DE10-Standard login:”. Để vào được root của DE10_Standard board nhập: “**root**”. Password nhấn enter.



Hình 5.20 Đăng nhập vào root của board

Tiếp đến, ta tạo 1 folder làm việc mới, ta nhập lệnh: **mkdir /mnt/FAT**

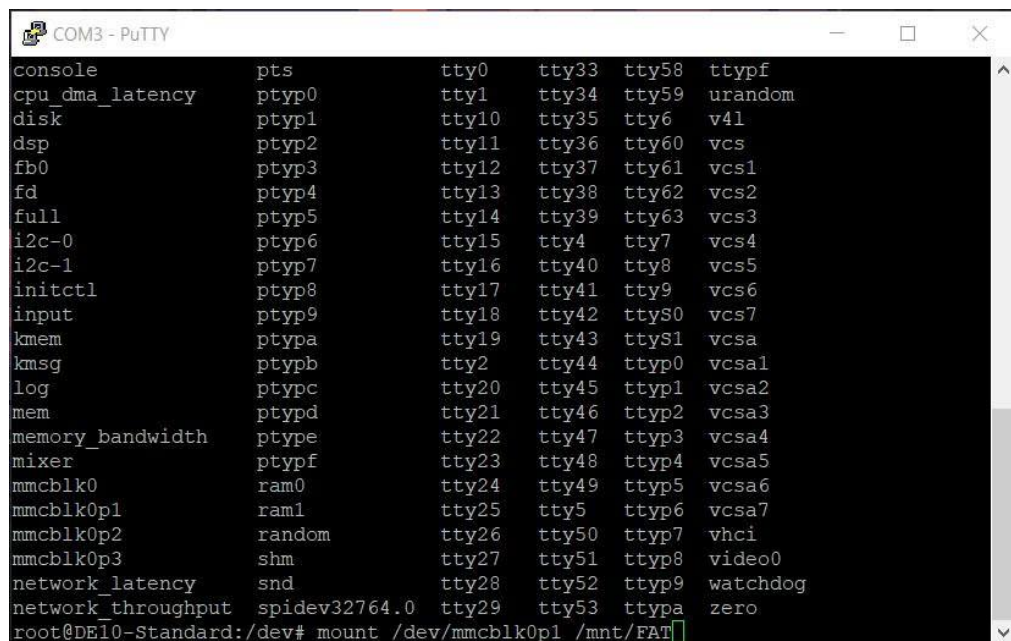
Để truy cập vào thư mục mới vừa tạo, ta nhập lệnh: **cd /mnt/FAT**



```
COM3 - PuTTY
root@DE10-Standard:~# mkdir /mnt/FAT
root@DE10-Standard:~# cd /mnt/FAT
root@DE10-Standard:/mnt/FAT# ls
root@DE10-Standard:/mnt/FAT#
```

Hình 5.21 Tạo folder làm việc mới

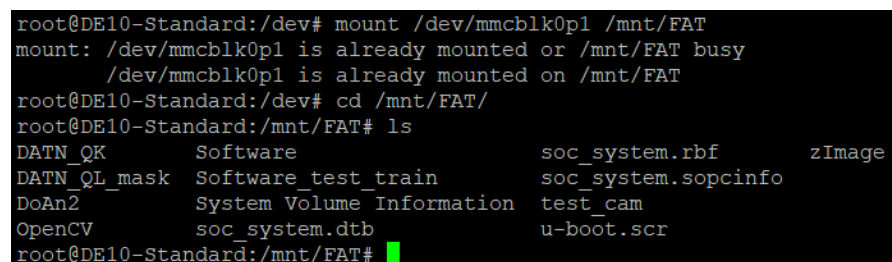
Truy cập vào vùng nhớ dev để xem danh sách và tìm vùng nhớ để sử dụng bằng lệnh: **cd dev**



```
COM3 - PuTTY
console pts tty0 tty33 tty58 ttyvf
cpu_dma_latency ptyp0 tty1 tty34 tty59 urandom
disk ptyp1 tty10 tty35 tty6 v4l
dsp ptyp2 tty11 tty36 tty60 vcs
fb0 ptyp3 tty12 tty37 tty61 vcs1
fd ptyp4 tty13 tty38 tty62 vcs2
full ptyp5 tty14 tty39 tty63 vcs3
i2c-0 ptyp6 tty15 tty4 tty7 vcs4
i2c-1 ptyp7 tty16 tty40 tty8 vcs5
initctl ptyp8 tty17 tty41 tty9 vcs6
input ptyp9 tty18 tty42 ttyS0 vcs7
kmem ptypa tty19 tty43 ttys1 vcsa
kmsg ptypb tty2 tty44 tty0 vcsa1
log ptypc tty20 tty45 tty1 vcsa2
mem ptypd tty21 tty46 tty2 vcsa3
memory_bandwidth ptype tty22 tty47 tty3 vcsa4
mixer ptypf tty23 tty48 tty4 vcsa5
mmcblk0 ram0 tty24 tty49 tty5 vcsa6
mmcblk0p1 ram1 tty25 tty5 tty6 vcsa7
mmcblk0p2 random tty26 tty50 tty7 vchi
mmcblk0p3 shm tty27 tty51 tty8 video0
network_latency snd tty28 tty52 tty9 watchdog
network_throughput spidev32764.0 tty29 tty53 ttya zero
root@DE10-Standard:/dev# mount /dev/mmcblk0p1 /mnt/FAT
```

Hình 5.22 Truy cập vùng nhớ dev

Gán vùng nhớ trên thẻ nhớ vào một vùng nhớ có sẵn trên board bằng lệnh: **mount /dev/mmcblk0p1 /mnt/FAT**. Sau đó truy cập lại vào /mnt/FAT để kiểm tra các file trên thẻ microSD đã được gán chưa.



```
root@DE10-Standard:/dev# mount /dev/mmcblk0p1 /mnt/FAT
mount: /dev/mmcblk0p1 is already mounted or /mnt/FAT busy
/dev/mmcblk0p1 is already mounted on /mnt/FAT
root@DE10-Standard:/dev# cd /mnt/FAT/
root@DE10-Standard:/mnt/FAT# ls
DATN_QK Software soc_system.rbf zImage
DATN_QL_mask Software_test_train soc_system.sopcinfo
DoAn2 System Volume Information test_cam
OpenCV soc_system.dtb u-boot.scr
root@DE10-Standard:/mnt/FAT#
```

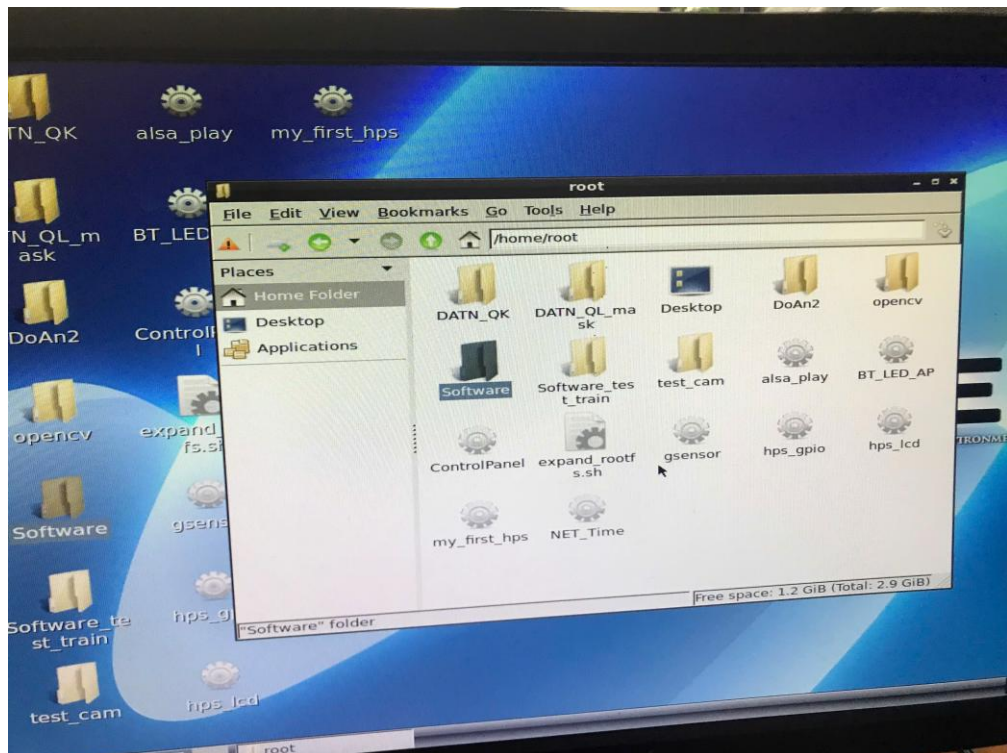
Hình 5.23 Gán vùng nhớ trên thẻ microSD vào vùng nhớ trên board

Thực hiện copy folder Software (chứa các file thực hiện phần mềm) vào mục /home/root sử dụng lệnh: **cp -r Software/home/root/**. Sau đó quay lại root để kiểm tra folder đã được copy chưa.

```
root@DE10-Standard:/mnt/FAT# cp -rf Software /home/root
root@DE10-Standard:/mnt/FAT# cd /home/root
root@DE10-Standard:~# ls
BT_LED_AP  Desktop  Software  test_train  hps_gpio  test_cam
ControlPanel  DoAn2    alsa_play  hps_lcd
DATN_QK    NET_Time  expand_rootfs.sh  my_first_hps
DATN_QL_mask  Software  gsensor    opencv
root@DE10-Standard:~#
```

Hình 5.24 Copy folder Software vào phân vùng root

Màn hình hiển thị đã copy folder Software ra /**home/root**/ thành công.



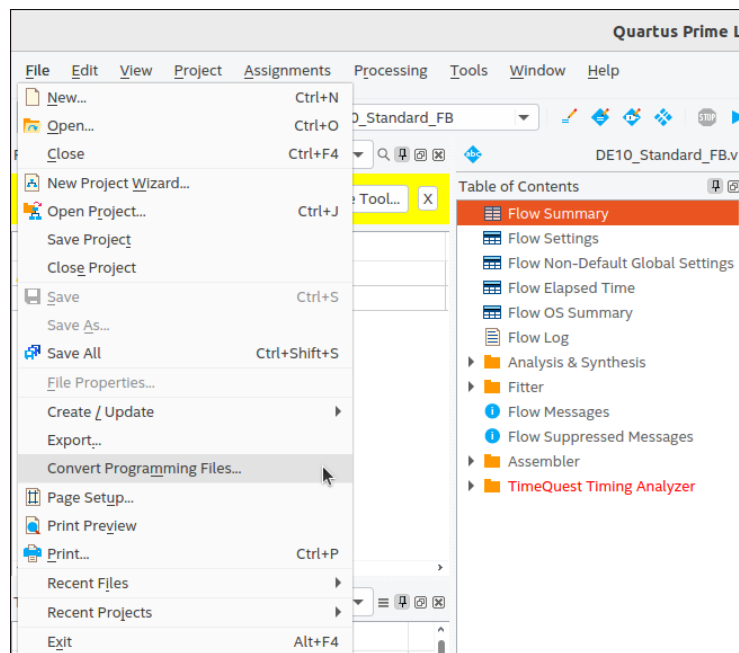
Hình 5.25 Folder Software hiển thị trên màn hình máy tính

5.2 Vận hành hệ thống SoC

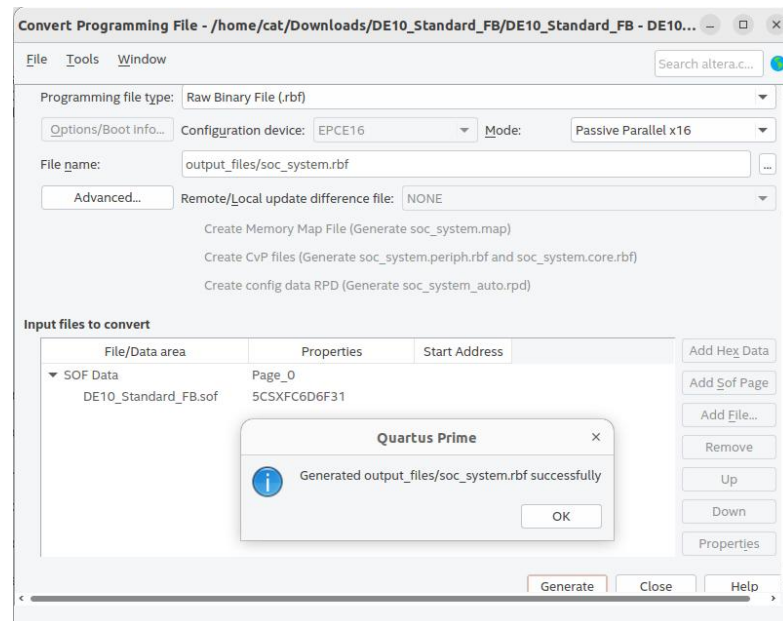
5.2.1 Vận hành phần cứng trên board DE10

Để vận hành phần cứng trên board, việc nạp các file **.sopcinfo** (optional), **.rbf** và **.dtb** vào phân vùng boot của board và thực hiện đồ file **.sof** thông qua USB BLASTER là cần thiết. Thường thì các project trên FPGA sau khi biên dịch sẽ tạo ra file **.sof** để nạp vào kit qua JTAG-UART port thông qua

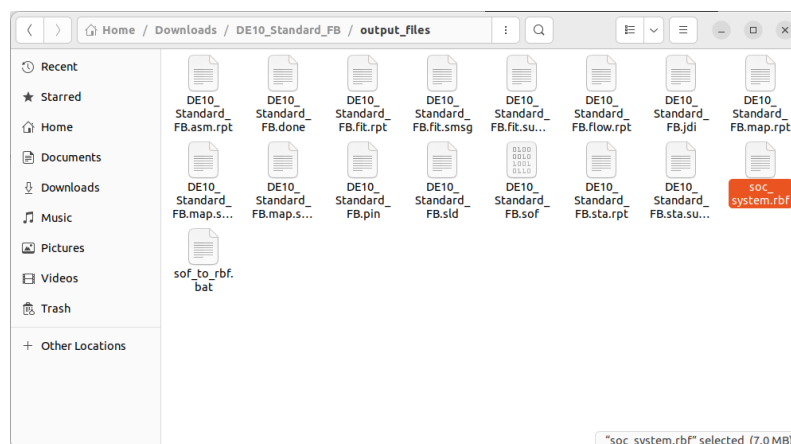
Programmer Device. Nhưng ở project này, vi xử lý ARM (HPS) lại cần file **.rbf** để boot hệ thống trên kit thông qua microSD card. Ta có thể convert từ file **.sof** sang **.rbf** (Raw Binary File) để nhúng vào hệ thống boot của board. Để thực hiện tạo file **.rbf**, ta sử dụng tool Quartus để convert file thông qua File → Convert Programming Files... Sau khi Generated, file **soc_system.rbf** nằm trong mục *output_files* của project.



Hình 5.26 Quá trình tạo file .rbf



Hình 5.27 Quá trình tạo file .rbf

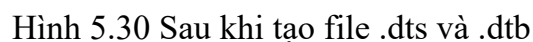
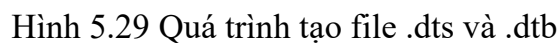


Hình 5.28 Sau khi tạo file .rbf

Còn về file **.dtb**, ta sử dụng file **soc_system_board_info.xml** từ nhà sản xuất để tạo ra lần lượt file **.dts** và **.dtb** với 2 hàm sau trên tool SoC EDS:

```
sopc2dts --input soc_system.sopcinfo --output soc_system.dts --type dts
--board soc_system_board_info.xml --board hps_common_board_info.xml --
bridge-removal all --clocks
```

```
dtc -I dts -O dtb -o soc_system.dtb soc_system.dts.
```

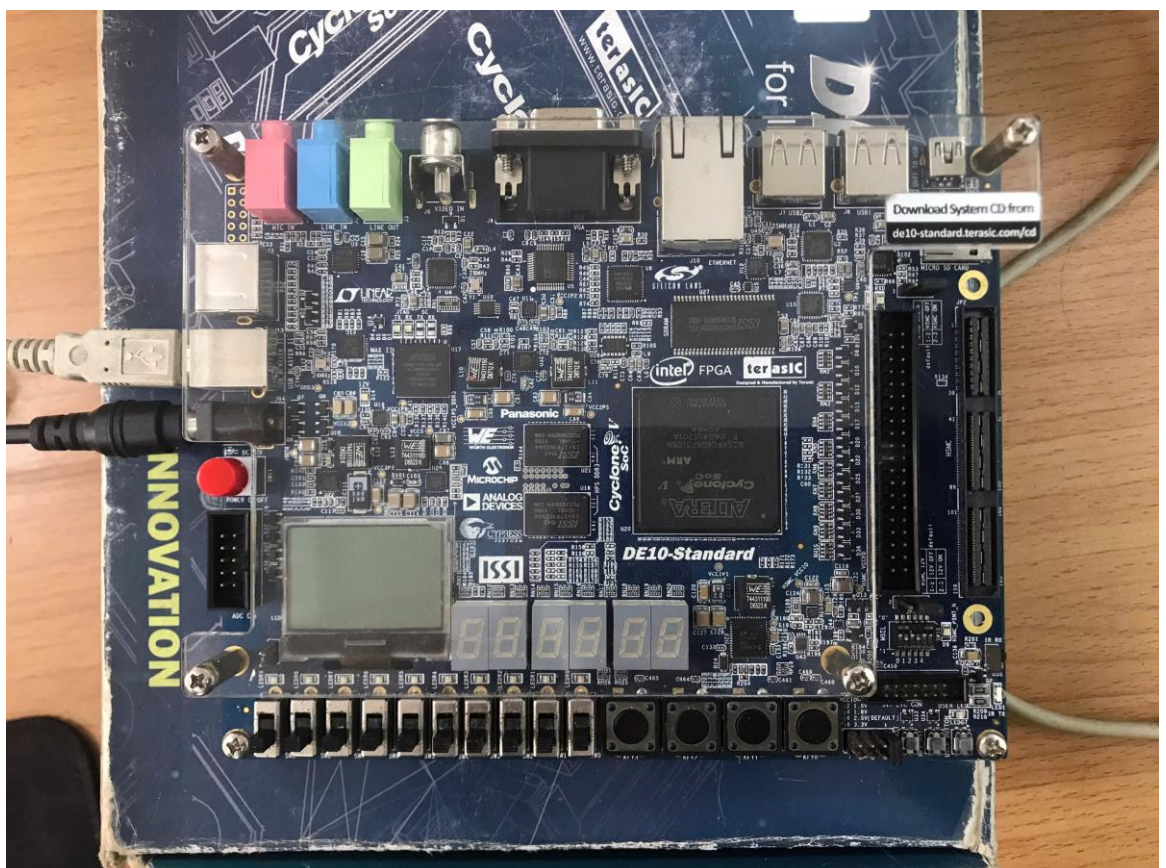


93

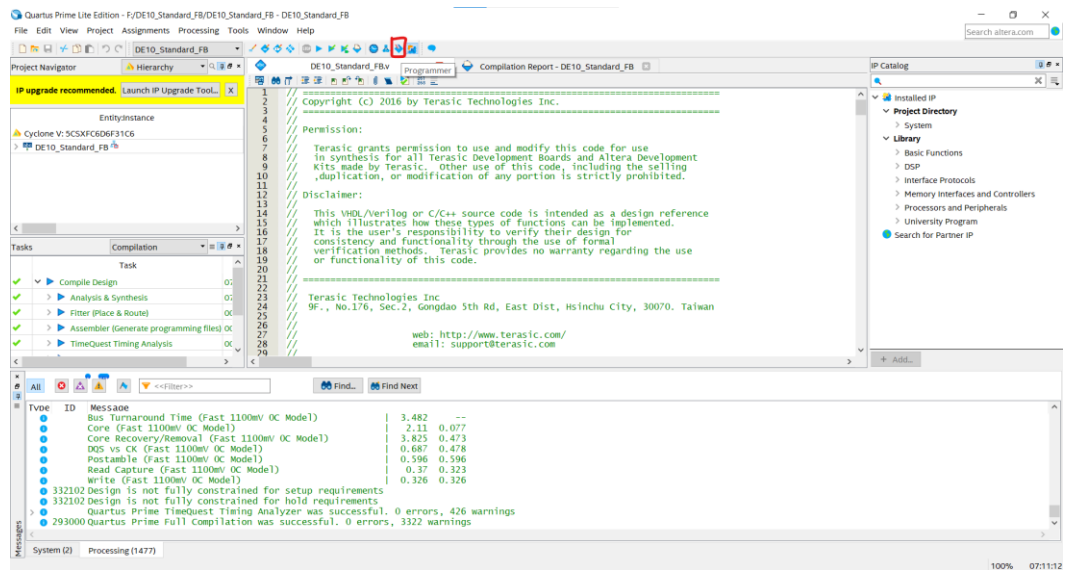
Name	Date modified	Type	Size
 soc_system.dtb	6/6/2017 7:22 AM	DTB File	26 KB
 soc_system.rbf	6/6/2017 7:22 AM	RBF File	2,856 KB
 zImage	4/27/2017 7:24 AM	File	5,051 KB
 u-boot	1/17/2017 5:39 PM	Screen saver	1 KB
 DoAn2	12/27/2025 11:46 AM	File folder	

Hình 5.31 Các file trên thẻ nhớ dùng để boot

Ngoài việc copy file vào phân vùng boot, để nạp được lên board hệ thống FPGA sau khi biên dịch project **DE10_Standard_FB**, ta thực hiện kết nối cổng BLASTER của kit với laptop và nạp hệ thống đó (.sof) thông qua *Program Device* trên *Quartus* [4].

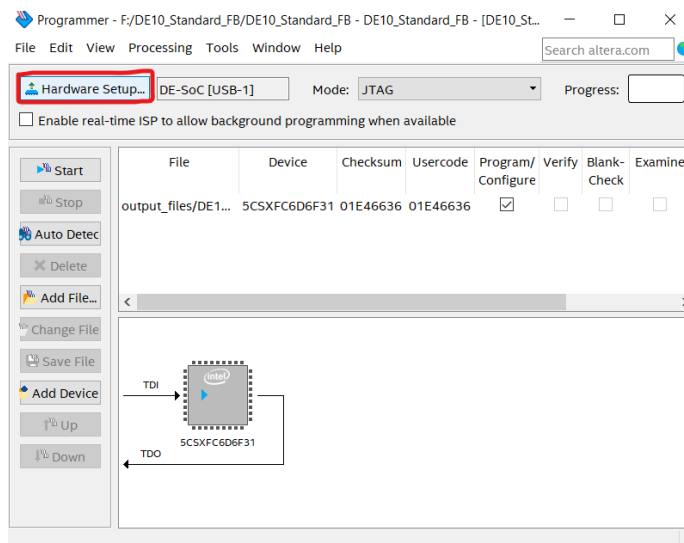


Hình 5.32 Kết nối trên board

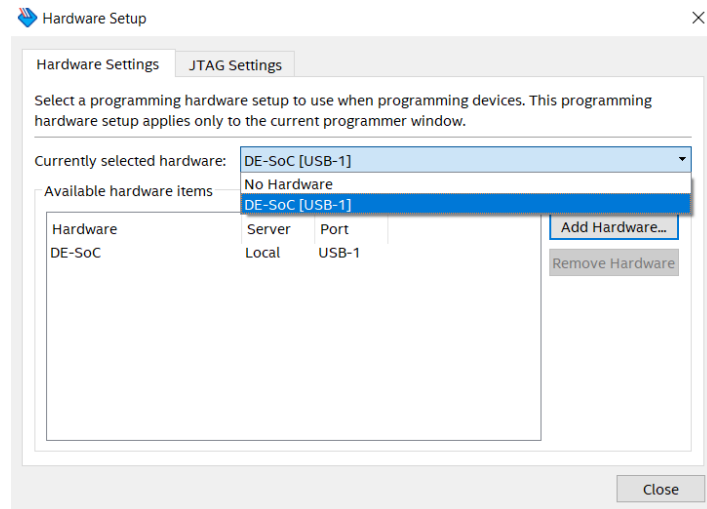


Hình 5.33 Quá trình nạp phần cứng lên board

Tiếp theo, nhấn Hardware Setup để chọn BLASTER USB của DE10 và Close.

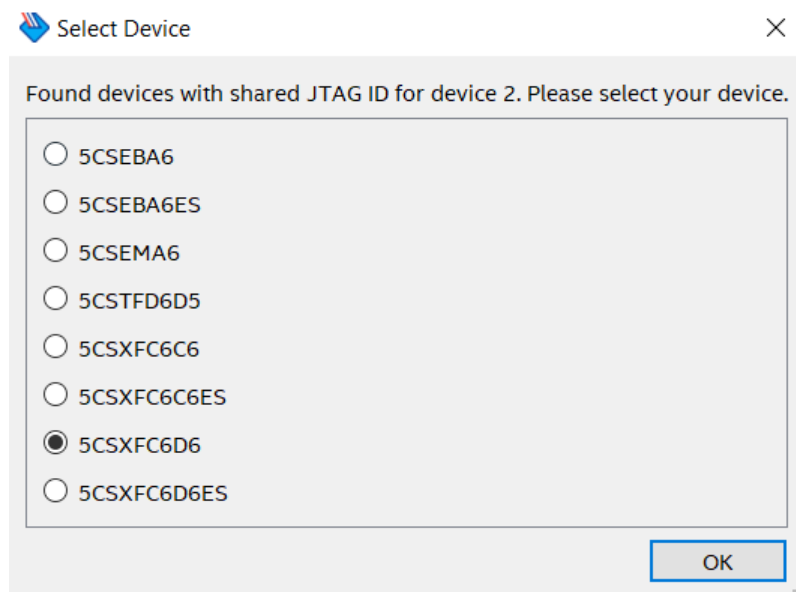


Hình 5.34 Quá trình nạp phần cứng lên board



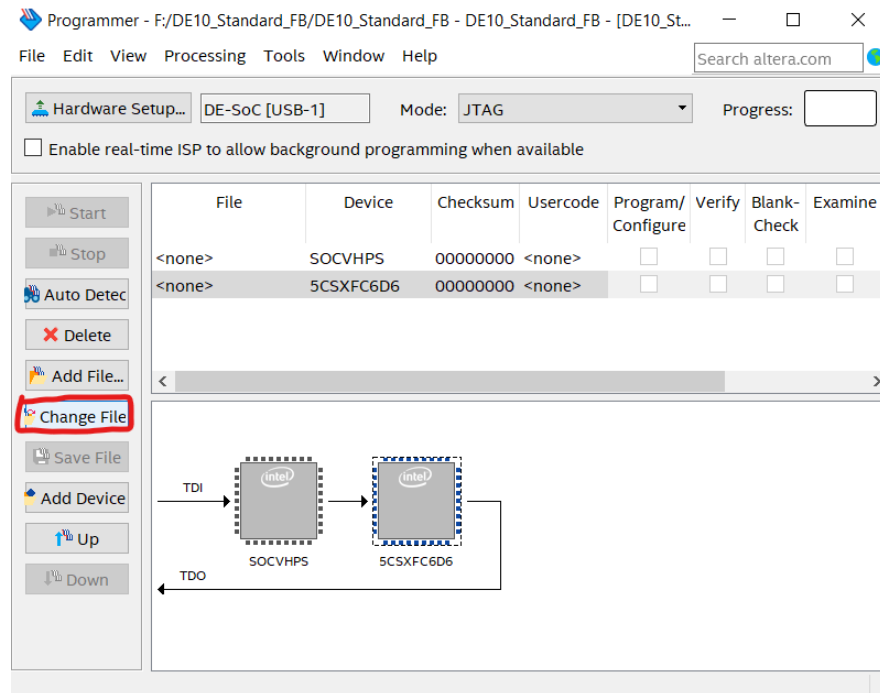
Hình 5.35 Quá trình nạp phần cứng lên board

Kế tiếp, chọn Auto Detect để lựa chọn mã thiết bị, “5CSXFC6D6” đối với DE10- Standard. Sau đó, màn hình sẽ hiện hai thành phần bên trong Cyclone V là HPS và FPGA.



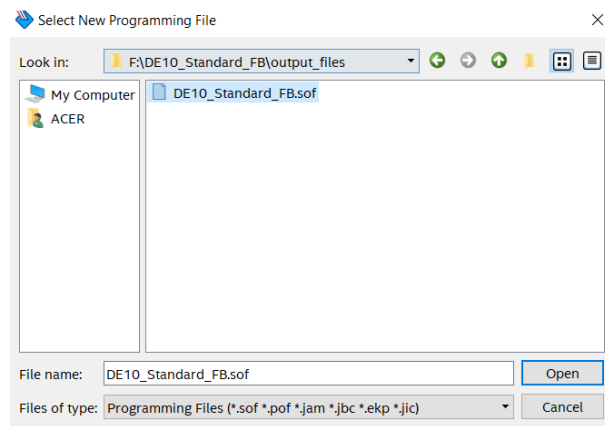
Hình 5.36 Quá trình nạp phần cứng lên board

Tiếp theo, nhấn chuột trái vào Device có tên “5CSXFC6D6” và Change File.

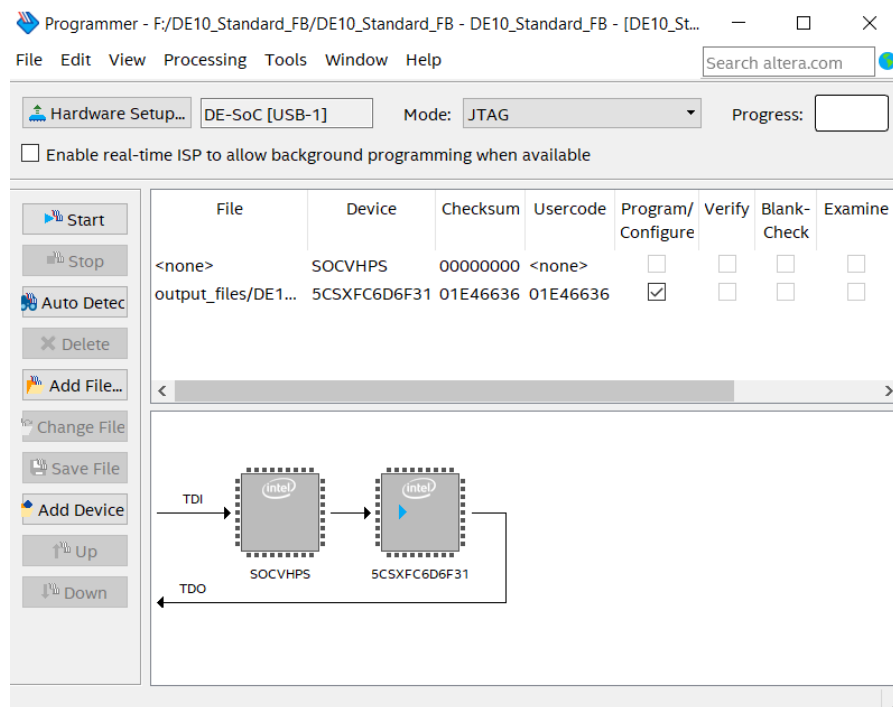


Hình 5.37 Quá trình nạp phần cứng lên board

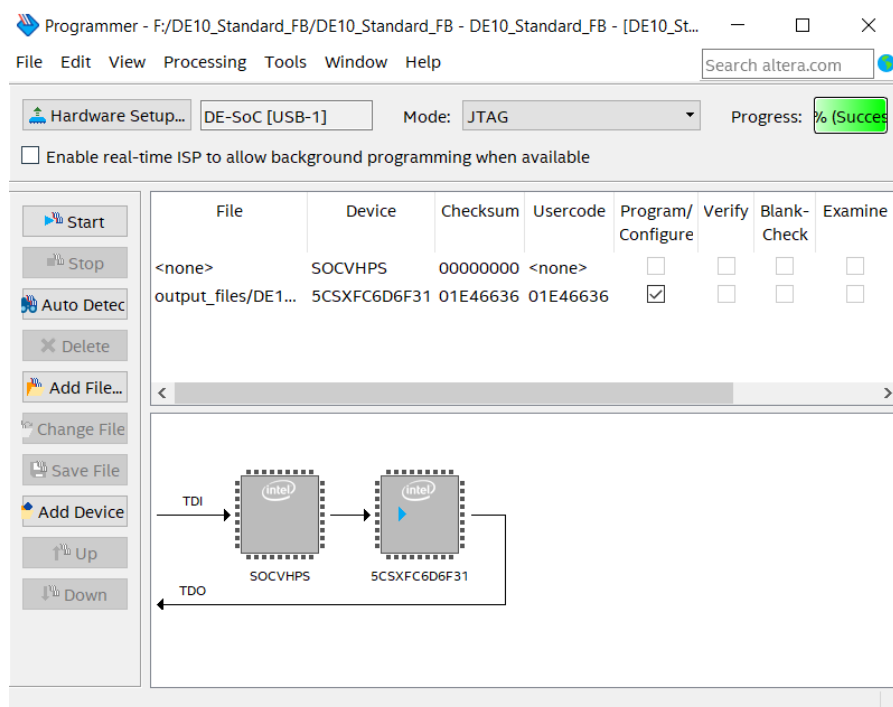
Tại giao diện chọn file nạp, chọn file **.sof** như sau rồi nhấn *Open*. Tiếp đến nhấn *Start*. Nếu thanh *Progress* báo *100% (Successful)* thì việc nạp phần cứng lên board đã thành công.



Hình 5.38 Quá trình nạp phần cứng lên board



Hình 5.39 Quá trình nạp phần cứng lên board



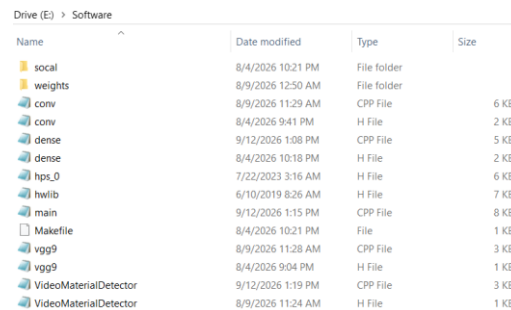
Hình 5.40 Quá trình nạp phần cứng lên board

5.2.2 Vận hành phần mềm trên board DE10

Để vận hành được phần mềm cần đảm bảo thực hiện đúng và đầy đủ mục 5.1 cùng với thực hiện quy trình như bên dưới.

- **Đảm bảo đầy đủ các file cần thiết**

Trước tiên, ta phải chắc chắn các file phần mềm cần thiết cho đề tài đã được khởi tạo đầy đủ vào môi trường Linux đã khởi tạo. Cụ thể, định dạng chúng ta cần như sau: File vận hành/điều khiển phần cứng (.cpp) thực hiện thu nhận hình ảnh bề mặt vật liệu.

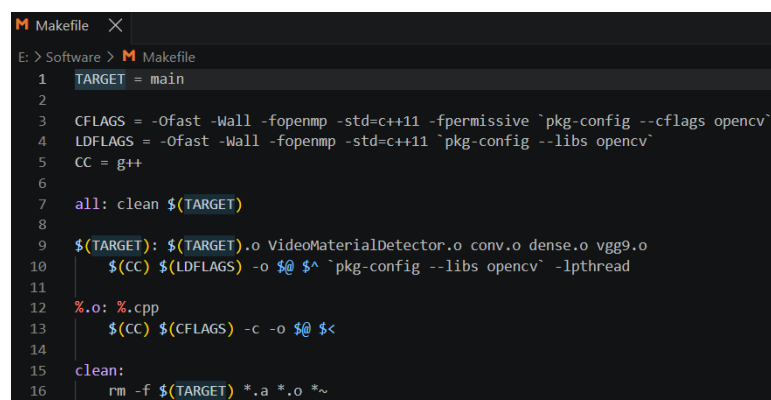


Name	Date modified	Type	Size
social	8/4/2026 10:21 PM	File folder	
weights	8/9/2026 12:50 AM	File folder	
conv	8/9/2026 11:29 AM	CPP File	6 KB
conv	8/4/2026 9:41 PM	H File	2 KB
dense	9/12/2026 1:08 PM	CPP File	5 KB
dense	8/4/2026 10:18 PM	H File	2 KB
hps_0	7/22/2023 3:16 AM	H File	6 KB
hwlib	6/10/2019 8:26 AM	H File	7 KB
main	9/12/2026 1:15 PM	CPP File	8 KB
Makefile	8/4/2026 10:21 PM	File	1 KB
vgg9	8/9/2026 11:28 AM	CPP File	3 KB
vgg9	8/4/2026 9:04 PM	H File	1 KB
VideoMaterialDetector	9/12/2026 1:19 PM	CPP File	3 KB
VideoMaterialDetector	8/9/2026 11:24 AM	H File	1 KB

Hình 5.41 Các file phần mềm vận hành phần cứng

- **Tạo Makefile**

Tại thư mục chứa file phần mềm đã chép vào board, tạo Makefile []. Nội dung của file này chứa các câu lệnh để biên dịch các file code C chứa chương trình dùng để điều khiển phần cứng sang file thực thi được chú thích như hình bên dưới.



```

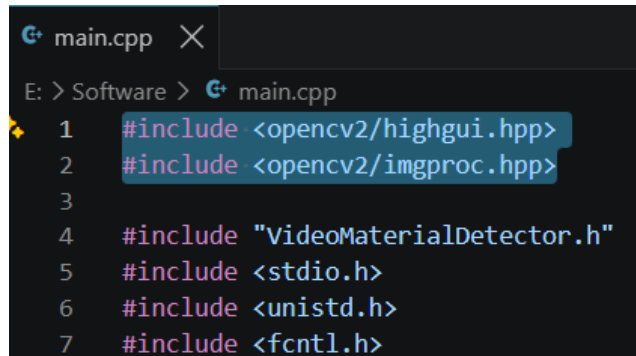
1 TARGET = main
2
3 CFLAGS = -Ofast -Wall -fopenmp -std=c++11 -fpermissive `pkg-config --cflags opencv`
4 LDFLAGS = -Ofast -Wall -fopenmp -std=c++11 `pkg-config --libs opencv`
5 CC = g++
6
7 all: clean $(TARGET)
8
9 $(TARGET): $(TARGET).o VideoMaterialDetector.o conv.o dense.o vgg9.o
10 $(CC) $(LDFLAGS) -o $@ $^ `pkg-config --libs opencv` -lpthread
11
12 %.o: %.cpp
13 $(CC) $(CFLAGS) -c -o $@ $<
14
15 clean:
16 rm -f $(TARGET) *.a *.o *~

```

Hình 5.42 Nội dung Makefile

- **Thư viện OpenCV**

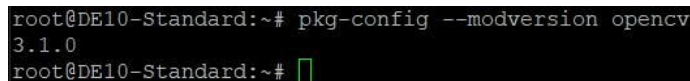
Trong đề tài này, ta cần phải có thư viện OpenCV mới có thể chạy được và nội dung của file code C cũng phải include thư viện OpenCV vào.



```
main.cpp X
E: > Software > main.cpp
1 #include <opencv2/highgui.hpp>
2 #include <opencv2/imgproc.hpp>
3
4 #include "VideoMaterialDetector.h"
5 #include <stdio.h>
6 #include <unistd.h>
7 #include <fcntl.h>
```

Hình 5.43 Include thư viện OpenCV

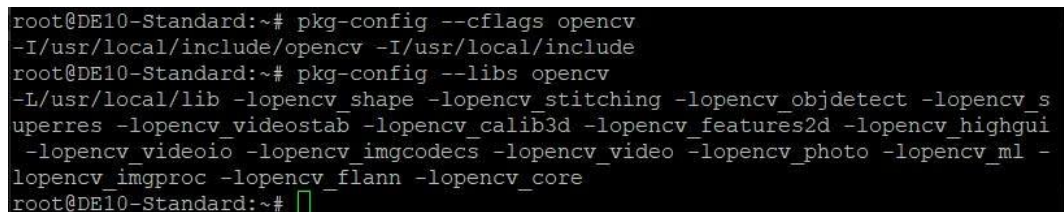
Ngoài ra, phần cài đặt thư viện OpenCV đã được cài đặt sẵn trên board. Ta dùng câu lệnh: **pkg-config --modversion opencv** để xem version của thư viện OpenCV:



```
root@DE10-Standard:~# pkg-config --modversion opencv
3.1.0
root@DE10-Standard:~#
```

Hình 5.44 Version của thư viện OpenCV

Để xem đường dẫn chứa thư viện OpenCV để người dùng có thể include thư viện vào, ta sử dụng câu lệnh: **pkg-config --cflags opencv** và **pkg-config --libs opencv**



```
root@DE10-Standard:~# pkg-config --cflags opencv
-I/usr/local/include/opencv -I/usr/local/include
root@DE10-Standard:~# pkg-config --libs opencv
-L/usr/local/lib -lopencv_shape -lopencv_stitching -lopencv_objdetect -lopencv_s
uperres -lopencv_videostab -lopencv_calib3d -lopencv_features2d -lopencv_highgui
-lopencv_videoio -lopencv_imgcodecs -lopencv_video -lopencv_photo -lopencv_ml -
lopencv_imgproc -lopencv_flann -lopencv_core
root@DE10-Standard:~#
```

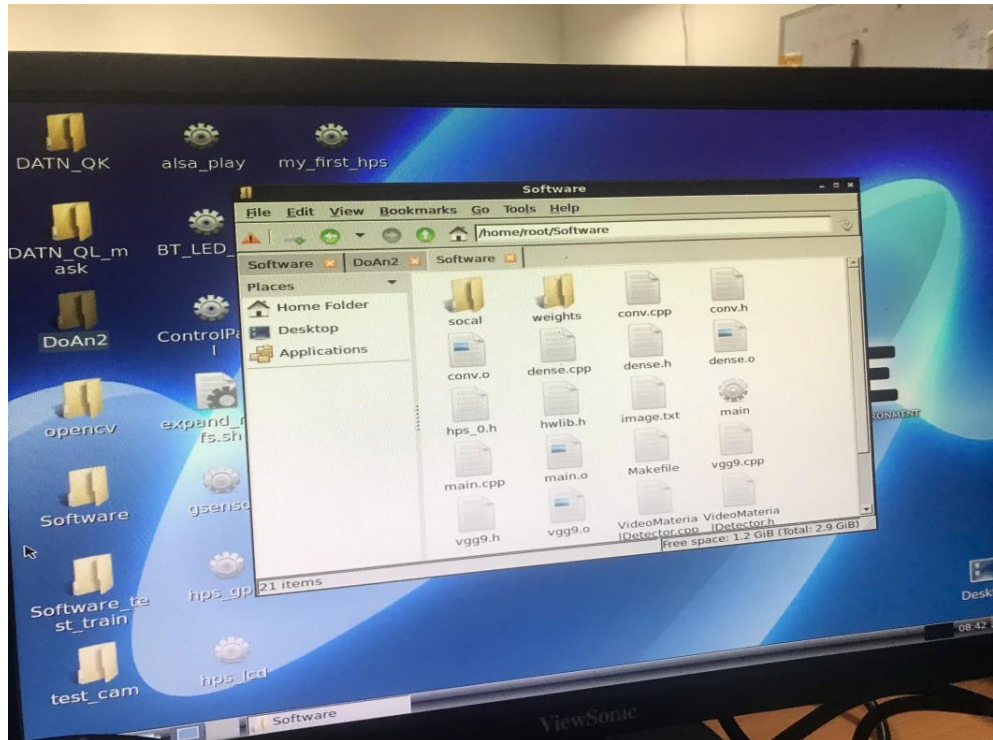
Hình 5.45 Đường dẫn đến thư viện OpenCV

• GNU Toolchain

Để có thể chạy được đề tài này, ta cần phải có file thực thi để chạy được trên môi trường Linux. Để có thể biên dịch từ file code C sang file thực thi để chạy được trên môi trường Linux, ta có thể kết nối giữa laptop với board thông qua phần mềm puTTY. Đồng thời, ARM Cortex-A9 cũng đóng vai trò là GNU Toolchain biên dịch từ file code C sang file thực thi.

- **Biên dịch phần mềm**

Mở puTTY rồi vào thư mục Software.



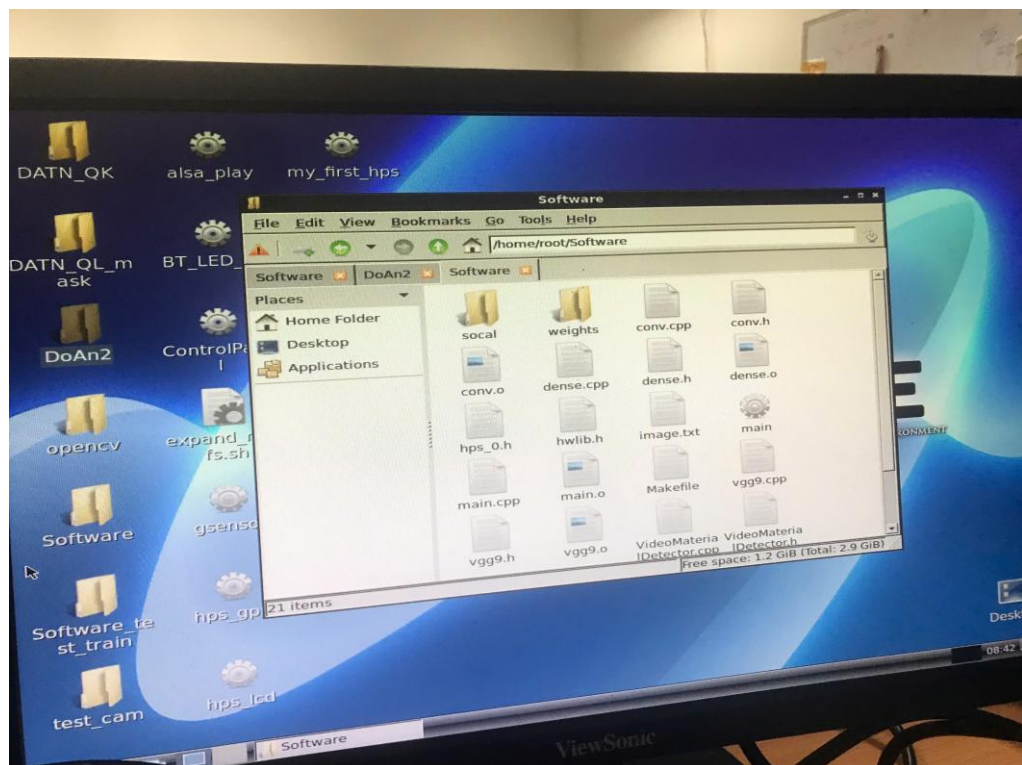
Hình 5.46 Bên trong thư mục Software

Trên giao diện Terminal, dùng lệnh make để hệ thống biên dịch và xuất file thực thi.

```
g++ -Ofast -Wall -fopenmp -std=c++11 -fpermissive `pkg-config --cflags opencv` -c -o vgg9.o vgg9.cpp
vgg9.cpp: In function 'int vgg9 predict(cv::Mat&, long unsigned int*, long unsigned int*, float*, float*)':
vgg9.cpp:16:8: warning: variable 'image' set but not used [-Wunused-but-set-variable]
FILE *image;
^
g++ -Ofast -Wall -fopenmp -std=c++11 `pkg-config --libs opencv` -o main main.o VideoMaterialDetector.o conv.o de
make: warning: Clock skew detected. Your build may be incomplete.
root@DE10-Standard:~/Software#
```

Hình 5.47 Biên dịch và xuất file thực thi

Sau khi chạy lệnh make ta được file thực thi như hình sau:

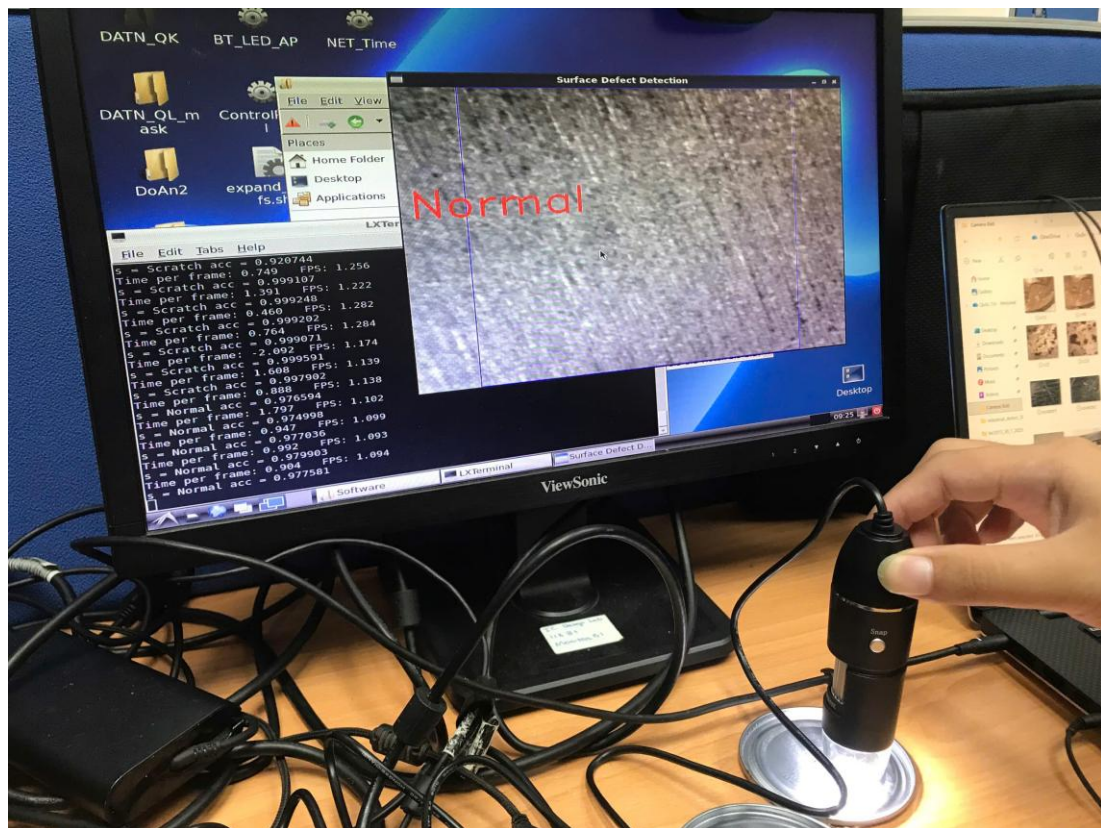


Hình 5.48 File thực thi sau khi chạy Makefile

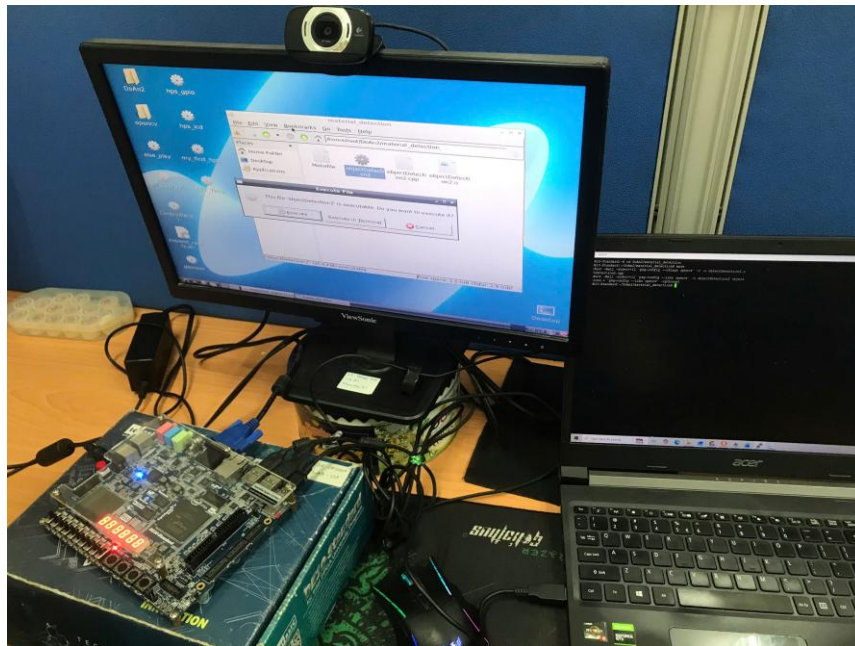
6. KẾT QUẢ THỰC HIỆN

6.1 Kết quả trực quan

Toàn cảnh hệ thống khi thực hiện:

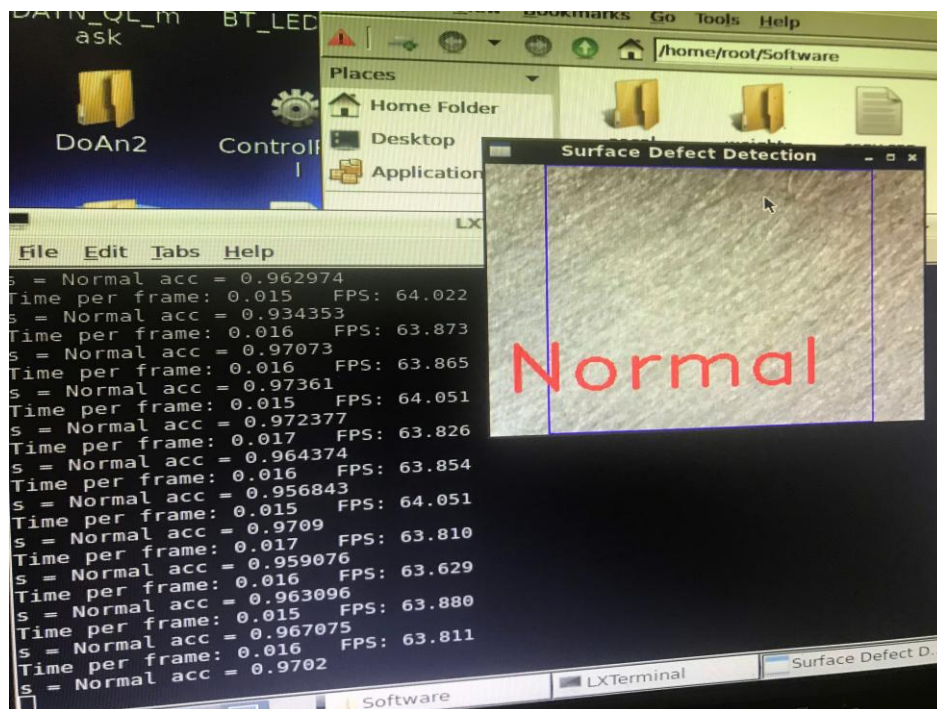


Hình 6.1 Hệ thống phát hiện khiếm khuyết bề mặt vật liệu dựa trên FPGA



Hình 6.2 Hệ thống phát hiện khiếm khuyết bề mặt vật liệu dựa trên FPGA

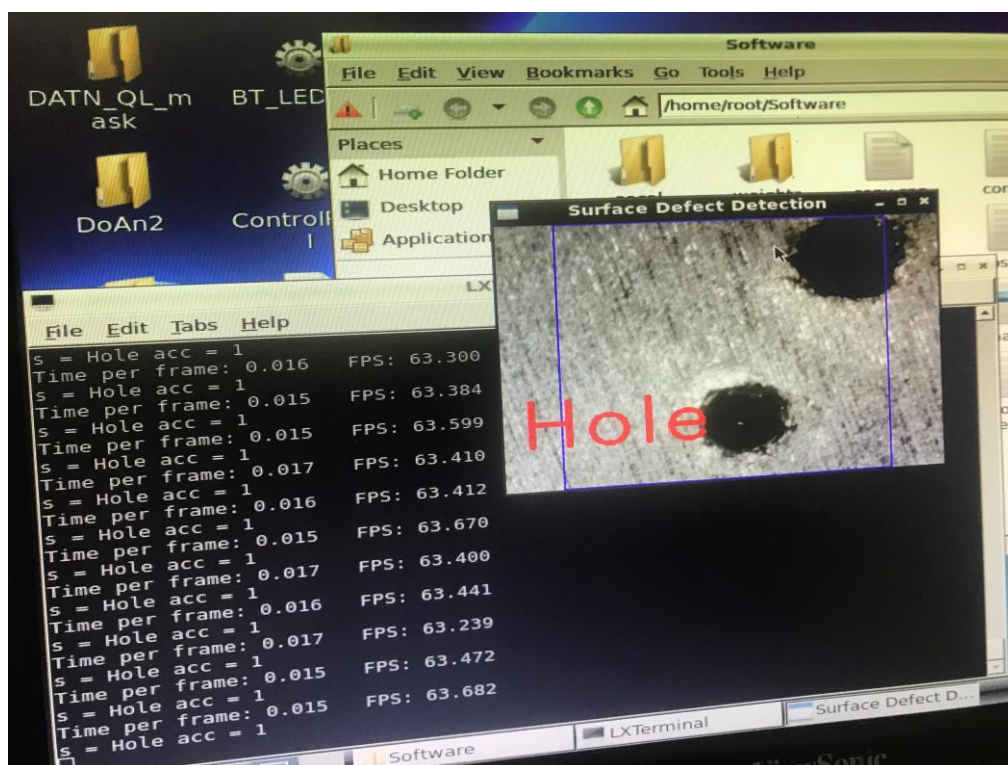
Sau đây là kết quả hiện thực mạng thần kinh tích chập trên board SoC-FPGA thông qua vi xử lý ARM để phát hiện khiếm khuyết bề mặt vật liệu thông qua camera.



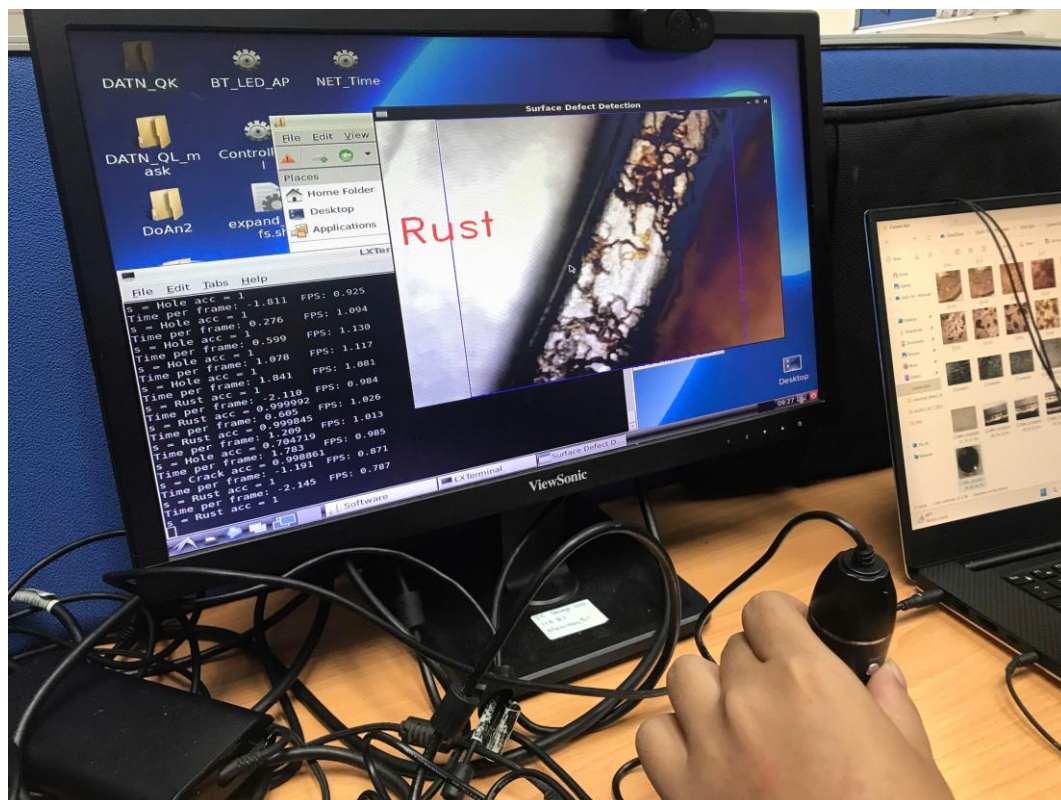
Hình 6.3 Phát hiện không có lỗi



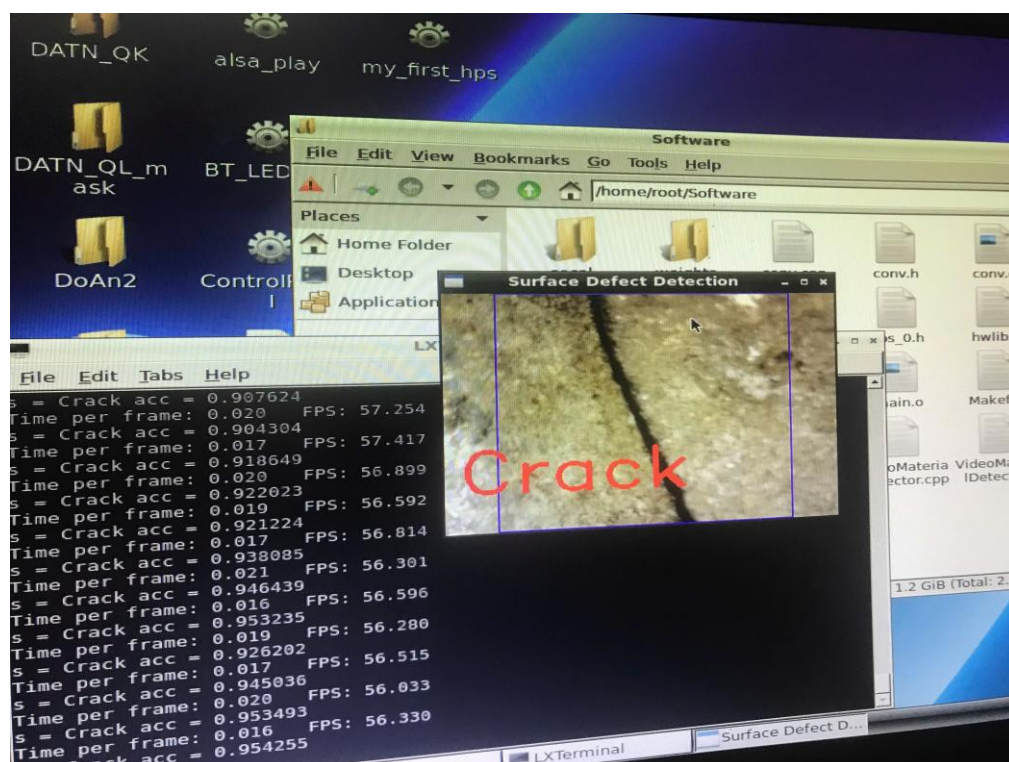
Hình 6.4 Phát hiện bề mặt bị xước



Hình 6.5 Phát hiện bề mặt bị thủng lỗ



Hình 6.6 Phát hiện bề mặt bị rỉ sét



Hình 6.7 Phát hiện bề mặt bị nứt

6.2 Đánh giá hệ thống

Việc đánh giá hệ thống sẽ dựa trên các tiêu chí: đáp ứng thời gian realtime, tài nguyên tiêu tốn, tốc độ khung hình mỗi giây, độ chính xác nhận diện khiếm khuyết, tính thẩm mỹ, tính ứng dụng.

Về đáp ứng thời gian: Hệ thống vận hành với đáp ứng soft-realtime nhờ vào phần cứng hoạt động với xung clock là 80MHz và có thể phân loại được 157 khiếm khuyết bề mặt vật liệu [3]. Thời gian trễ trung bình của một khung hình sau khi được xử lý cả trên phần cứng lẫn phần mềm được tính dựa thông qua hàm `cv::getCPUTickCount()` để có được sự chênh lệch thời gian giữa trước và sau khi xử lý một khung hình. Thời gian này rơi vào khoảng từ 15-34 ms/khung hình.

Về tốc độ khung hình: Hệ thống có thể tùy chỉnh tốc độ FPS giữa việc đánh đổi giữa tốc độ với kích thước khung hình, ví dụ với tốc độ khung hình trung bình rơi vào khoảng 60 (FPS) thì tương ứng kích cỡ hình ảnh thu nhận là 320x240 hoặc FPS thấp hơn, đổi lại kích thước hình ảnh thu nhận sẽ to hơn. Tóm lại, hệ thống sẽ tính tốc độ được tính dựa trên độ trễ của khung hình. Hệ thống có thể đạt được tốc độ khung hình trung bình như trên là do trong chương trình phần mềm chỉ sử dụng 2 hàm để phát hiện khiếm khuyết bề mặt vật liệu. Hàm thứ nhất được dùng cho việc khoanh ROI bề mặt vật liệu cố định trên nền màu RGB có bounding box. Hàm giúp hệ thống tập trung vào vùng cần quan tâm (ROI) nhằm thu hẹp vùng chứa bề mặt vật liệu trên hệ màu xám để giảm dữ liệu xử lý, cải thiện hiệu suất của phần cứng và đây sẽ là nguồn input feature map cho IP phần cứng.

Về tài nguyên tiêu tốn: hệ thống phần cứng thực hiện tích chập trên số nguyên 16 bit với xung clock 80MHz có tài nguyên tiêu tốn được thống kê như trong Bảng 6.1:

Bảng 6.1 Tài nguyên phần cứng tiêu tốn

Loại tài nguyên	Giải thuật đề nghị	Tài nguyên của Cyclone V
ALMs	22,494 (54%)	41,910
DSPs	112 (100%)	112
RAMs	553 (100%)	553
On-chip memory (Kb)	4,313 (76%)	5,662
PLLs	1 (7%)	15
DLLs	1 (25%)	4

Về độ chính xác: hệ thống hoạt động đạt được độ chính xác rơi vào khoảng 96%, được thể hiện thông qua Bảng 6.2 với các công thức tính như sau:

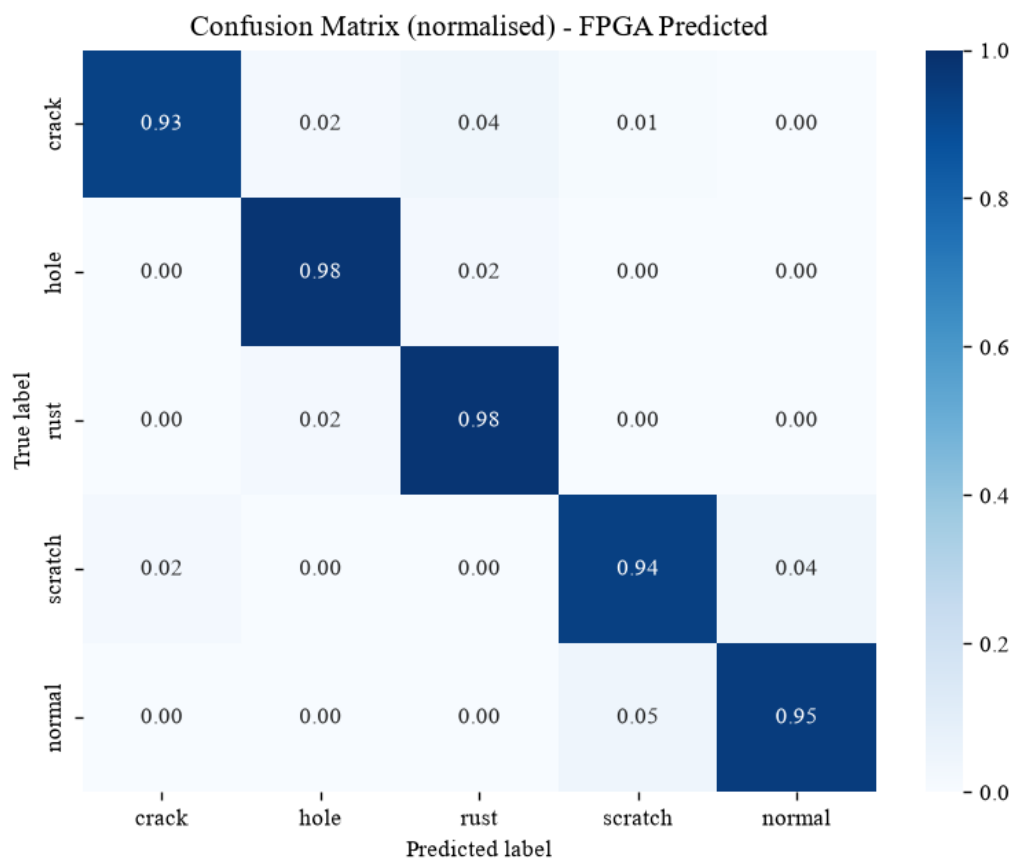
$$F1 - score = \frac{2 * precision * recall}{precision + recall}$$

Trong đó:

$$precision = \frac{true\ positive\ predictions}{positive\ predictions}; \quad recall = \frac{true\ positive\ predictions}{true\ positive\ cases}$$

Bảng 6.2 Các chỉ số đánh giá mô hình nhận diện khiếm khuyết trên Google Colab

	Precision	Recall	F1-Score	Support
crack	0.9836	1.0000	0.9917	300
hole	0.9967	0.9967	0.9967	300
rust	1.0000	0.9667	0.9831	300
scratch	0.9933	0.9900	0.9917	300
normal	0.9804	1.0000	0.9901	300
Accuracy			0.9907	1500



Hình 6.8 Ma trận nhầm lẫn khi chạy tập test trên FPGA.

Hệ thống có thể đạt được độ chính xác như vậy là do cấu trúc mạng thần kinh tích chập tương đối sâu (7 lớp convolution và 2 lớp fully-connected). Cũng như độ chính xác khi huấn luyện mô hình trên Google Colab và triển khai trên DE10 khá tương đồng. Khi hiện thực mô hình trên hệ thống Board DE10-Standard thì kết quả nhận dạng các loại khiếm khuyết như hole, rust, crack tương đối tốt, còn hạn chế khi nhận diện với normal và scratch. Cho thấy việc ứng dụng model AI trên board DE10 có tính khả thi.

Nhưng có thể thấy sự khác biệt giữa độ chính xác khi chạy trên Google Colab so với lúc độ chính xác khi nạp lên FPGA. Lý do cho sự khác biệt này là vì khi đưa xuống FPGA, việc cắt bỏ các phần thập phân siêu nhỏ để ép về định dạng số nguyên int16 chuẩn Q1.15 sẽ làm tích tụ sai số qua 7 lớp tích chập, trong khi trên Google Colab thì dù trọng số bị ép về $(-0.99, 0.99)$ thì định dạng của chúng là float32, độ chính xác cực kỳ cao. Điều này dự kiến kéo Accuracy tổng thể tụt xuống.

Về tính ứng dụng: Hệ thống có khả năng nhận dạng khiếm khuyết bề mặt vật liệu sử dụng mạng thần kinh tích chập trên kit FPGA. Nhờ vào việc tương đồng giữa model AI và triển khai lên phần cứng như vậy đã mở ra tiềm năng phát triển ứng dụng theo nhiều hướng vào nhiều lĩnh vực đời sống khác nhau. Cụ thể với một số ứng dụng như: Kiểm tra tự động trên dây chuyền cán thép và gia công kim loại tấm cũng như Dây chuyền gạch men, kính công nghiệp và vật liệu; Kiểm định chất lượng linh kiện ngành công nghiệp ô tô và cơ khí chính xác,...

Về tính thẩm mỹ: Hệ thống còn khá thô sơ, chỉ cơ bản hoàn thành mục tiêu, hệ thống vẫn chưa có tính thẩm mỹ cao về cả phần cứng.

7. KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN HỆ THỐNG

7.1 Kết luận

Với mục đích thiết kế và hiện thực mạng thần kinh tích chập trên board SoC-FPGA thông qua vi xử lý ARM để thực hiện phát hiện khiếm khuyết bề mặt vật liệu từ việc lấy hình ảnh từ camera đến lúc hiển thị kết quả lên màn hình. Đồ án đã thực hiện được các nhiệm vụ sau:

- Thực hiện thành công kết nối phần cứng cho mạng thần kinh tích chập.
- Huấn luyện thành công mô hình để có trọng số cho mạng thần kinh tích chập.
- Thiết kế và thực hiện thành công chương trình phần mềm cho vi xử lý ARM để nhận hình ảnh từ camera, truyền hình ảnh và trọng số đến bộ tích chập, và điều khiển hoạt động tích chập trên phần cứng.
- Thực hiện thành công quá trình fully-connected của mạng thần kinh tích chập trên vi xử lý ARM.
- Vận hành thành công hệ thống SoC mạng thần kinh tích chập để phát hiện khiếm khuyết bề mặt vật liệu.

Hầu như trong đề tài này đã thực hiện hoàn thành các nhiệm vụ đã giao nhưng vẫn còn hạn chế là độ chính xác nhận dạng khiếm khuyết vẫn chưa cao.

7.2 Hướng phát triển

Trong phần hướng phát triển, có một số cải tiến mà chúng ta có thể thực hiện để nâng cao độ chính xác và tính ứng dụng của hệ thống trong đề tài.

Về độ chính xác: Chúng ta có thể nâng cấp độ chính xác của hệ thống bằng cách thực hiện huấn luyện với một mô hình tối ưu hơn. Điều này đòi hỏi chúng ta cần tìm hiểu và áp dụng các thuật toán và phương pháp khác trong lĩnh vực machine learning. Hơn nữa, chúng ta có thể tinh chỉnh phần cứng tích hợp CNN để đạt được hiệu suất cao hơn.

Về tính ứng dụng: Để nâng cao khả năng triển khai thực tế trên các băng chuyền công nghiệp, hệ thống có thể được tích hợp cơ sở dữ liệu MySQL nhằm tự động lưu trữ nhật ký kiểm định (bao gồm thời gian, phân loại khuyết tật và số lượng phế phẩm). Điều này tạo tiền đề vững chắc cho việc truy xuất nguồn gốc lô hàng và thống kê năng suất dây chuyền. Bên cạnh đó, định hướng phát triển phần mềm cần tập trung xây dựng bảng điều khiển giám sát trung tâm (Web-based Dashboard) hoặc chuẩn hóa giao thức kết nối trực tiếp với bộ điều khiển PLC/hệ thống SCADA của nhà máy. Việc này giúp kỹ sư vận hành dễ dàng theo dõi trực quan trạng thái bề mặt vật liệu và điều khiển các cơ cấu gạt bỏ sản phẩm lỗi theo thời gian thực.

Tóm lại, trọng tâm phát triển tiếp theo của hệ thống cần hướng đến hai mục tiêu: Tiếp tục tinh chỉnh độ chuẩn xác của mô hình AI đối với các khiếm khuyết vi mô, và đồng bộ hóa luồng dữ liệu nhận diện lên hệ thống quản lý tập trung, qua đó đáp ứng trọn vẹn yêu cầu khắt khe của nền sản xuất thông minh.

8. TÀI LIỆU THAM KHẢO

- [1] DE10-Standard User Manual, Hsinchu: Terasic, 2018.
- [2] Avalon Interface Specifications, Intel Corporation, 2022.
- [3] Truong Quang Vinh, Dinh Viet Hai, "Optimizing Convolutional Neural Network Accelerator on Low-Cost FPGA," 2021.
- [4] DE10-Standard Getting Started Guide, Hsinchu: Terasic, 2017.
- [5] robertofem, "CycloneVSoC-examples," GitHub, 18 6 2017. [Online]. Available: <https://github.com/robertofem/CycloneVSoC-examples/blob/master/SD-operatingsystem/Angstrom-v2013.12/README.md>.
- [6] DE10-Standard My First HPS, Hsinchu: Terasic, 2017.
- [7] R. Bacrau, "GSRD - GHRD Overview," 26 6 2014. [Online]. Available: <https://rocketboards.org/foswiki/Documentation/GSRDGhrd>.
- [8] Hanh Phan-Xuan, Thuong Le-Tien, Sy Nguyen-Tan, "FPGA Platform applied for Facial Expression Recognition System using Convolutional Neural Networks," Procedia Computer Science, vol. 151, pp. 651-658, 4 2019.
- [9] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," CoRR, vol. abs/1409.1556, 2014.
- [10] Y. Ma, Y. Cao, S. B. K. Vrudhula, and J. sun Seo, "Optimizing the Convolution Operation to Accelerate Deep Neural Networks on FPGA.

- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv preprint arXiv:1412.6980, 2014.
- [12] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting.,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929- -1958, 2014.
- [13] Phạm Thế Vinh, “Thiết kế cấu trúc phần cứng cho mạng nơron tích chập”, 2020.